

RestN

Создано системой Doxygen 1.16.1

1 Пользовательская документация (RestN)	1
1.1 Обзор проекта	1
1.2 Технологии программирования	2
1.3 Структура проекта	2
1.4 Описание программных модулей	2
1.4.1 Модуль работы с базой данных (DBWorker)	2
1.4.2 Модуль сетевого клиента (TcpClient)	3
1.4.3 Модуль сетевого сервера (TcpServer + ClientConnection)	3
1.4.4 Модуль логирования (Logs)	3
1.4.5 Модуль настроек (Settings)	3
1.4.6 Пользовательский интерфейс (QML)	3
1.5 Интеграция программных модулей	4
1.5.1 Подключаемые внешние модули	4
1.5.2 Интеграция C++ и QML	4
1.5.3 Клиент-серверное взаимодействие	4
1.6 Руководство пользователя	4
1.6.1 Начальный запуск	4
1.6.2 Окно авторизации	4
1.6.3 Диалог настроек	5
1.6.4 Главное окно и выдвижное меню	5
1.6.5 Оформление заказа (страница для официанта)	5
1.6.6 Управление складом (страница для кладовщика)	5
1.6.7 Отчёты (страница для администратора)	6
1.6.8 Активные заказы (страница для официанта и администратора)	6
1.7 Настройки и системные файлы	6
1.8 Сборка проекта (для разработчиков)	6
1.9 Контакты	7
2 Иерархический список классов	9
2.1 Иерархия классов	9
3 Алфавитный указатель классов	11
3.1 Классы	11
4 Список файлов	13
4.1 Файлы	13
5 Классы	15
5.1 Класс ClientConnection	15
5.1.1 Подробное описание	15
5.1.2 Конструктор(ы)	15
5.1.2.1 ClientConnection()	15
5.1.2.2 ~ClientConnection()	16
5.2 Класс DatabaseEx	16

5.2.1 Подробное описание	16
5.2.2 Конструктор(ы)	16
5.2.2.1 DatabaseEx()	16
5.3 Класс DBWorker	16
5.3.1 Подробное описание	18
5.3.2 Конструктор(ы)	18
5.3.2.1 DBWorker()	18
5.3.2.2 ~DBWorker()	19
5.3.3 Методы	19
5.3.3.1 addOrderItem()	19
5.3.3.2 addProductQuantity()	19
5.3.3.3 checkAndConsume()	20
5.3.3.4 checkUser()	20
5.3.3.5 closeOrder()	20
5.3.3.6 createOrder()	21
5.3.3.7 generateOrderNumber()	21
5.3.3.8 getAllOrdersForReport()	21
5.3.3.9 getCheckByOrderId()	21
5.3.3.10 getChecksByDateRange()	22
5.3.3.11 getClosedOrders()	22
5.3.3.12 getDishesList()	22
5.3.3.13 getFoodPopularity()	22
5.3.3.14 getFoods()	22
5.3.3.15 getLocalIps()	23
5.3.3.16 getLowStockWarning()	23
5.3.3.17 getOrderItems()	23
5.3.3.18 getOrders()	23
5.3.3.19 getProducts()	24
5.3.3.20 getReportLowStock()	24
5.3.3.21 getReportRevenue()	24
5.3.3.22 getRevenueByDay()	24
5.3.3.23 getUserId()	25
5.3.3.24 getUserName()	25
5.3.3.25 getUserRole()	25
5.3.3.26 getWaitersList()	25
5.3.3.27 pingServer()	25
5.3.3.28 reconnect()	25
5.3.3.29 refreshAllFoodsAvailability()	26
5.3.3.30 returnIngredients()	26
5.3.3.31 saveReportToCSV()	26
5.3.3.32 syncOrderItems()	27
5.3.3.33 updateConnectionParams()	27

5.3.3.34 updateOrderStatus()	27
5.3.3.35 updateProductQuantity()	28
5.4 Класс Exceptions	28
5.4.1 Подробное описание	28
5.4.2 Конструктор(ы)	29
5.4.2.1 Exceptions()	29
5.4.3 Методы	29
5.4.3.1 what()	29
5.5 Класс FileEx	29
5.5.1 Подробное описание	29
5.5.2 Конструктор(ы)	29
5.5.2.1 FileEx()	29
5.6 Класс Logs	30
5.6.1 Подробное описание	30
5.6.2 Конструктор(ы)	30
5.6.2.1 Logs()	30
5.6.2.2 ~Logs()	30
5.6.3 Методы	30
5.6.3.1 error()	30
5.6.3.2 info()	30
5.6.3.3 setLogPath()	31
5.6.3.4 warning()	31
5.7 Класс NetworkEx	31
5.7.1 Подробное описание	31
5.7.2 Конструктор(ы)	31
5.7.2.1 NetworkEx()	31
5.8 Класс Settings	32
5.8.1 Подробное описание	33
5.8.2 Конструктор(ы)	33
5.8.2.1 Settings()	33
5.8.3 Методы	33
5.8.3.1 dbHost()	33
5.8.3.2 dbName()	33
5.8.3.3 dbPassword()	34
5.8.3.4 dbPort()	34
5.8.3.5 dbUser()	34
5.8.3.6 isServerMode()	34
5.8.3.7 load()	34
5.8.3.8 logPath()	34
5.8.3.9 reportsPath()	34
5.8.3.10 save()	34
5.8.3.11 serverHost()	34

5.8.3.12 serverPort()	34
5.8.3.13 setDbHost()	34
5.8.3.14 setDbName()	35
5.8.3.15 setDbPassword()	35
5.8.3.16 setDbPort()	35
5.8.3.17 setDbUser()	35
5.8.3.18 setIsServerMode()	35
5.8.3.19 setLogPath()	35
5.8.3.20 setReportsPath()	35
5.8.3.21 setServerHost()	35
5.8.3.22 setServerPort()	35
5.8.4 Полный список свойств	36
5.8.4.1 dbHost	36
5.8.4.2 dbName	36
5.8.4.3 dbPassword	36
5.8.4.4 dbPort	36
5.8.4.5 dbUser	36
5.8.4.6 isServerMode	36
5.8.4.7 logPath	36
5.8.4.8 reportsPath	36
5.8.4.9 serverHost	36
5.8.4.10 serverPort	36
5.9 Класс TcpClient	36
5.9.1 Подробное описание	38
5.9.2 Конструктор(ы)	38
5.9.2.1 TcpClient()	38
5.9.2.2 ~TcpClient()	39
5.9.3 Методы	39
5.9.3.1 addOrderItem()	39
5.9.3.2 addProductQuantity()	39
5.9.3.3 checkAndConsume()	40
5.9.3.4 checkUser()	40
5.9.3.5 closeOrder()	40
5.9.3.6 connectToServer()	41
5.9.3.7 createOrder()	41
5.9.3.8 disconnectFromServer()	41
5.9.3.9 getCheckByOrderId()	41
5.9.3.10 getClosedOrders()	42
5.9.3.11 getDishesList()	42
5.9.3.12 getFoodPopularity()	42
5.9.3.13 getFoods()	42
5.9.3.14 getLocalIps()	42

5.9.3.15 getLowStockWarning()	43
5.9.3.16 getOrderItems()	43
5.9.3.17 getOrders()	43
5.9.3.18 getProducts()	43
5.9.3.19 getReportLowStock()	44
5.9.3.20 getReportRevenue()	44
5.9.3.21 getRevenueByDay()	44
5.9.3.22 getUserId()	45
5.9.3.23 getName()	45
5.9.3.24 getUserRole()	45
5.9.3.25 getWaitersList()	45
5.9.3.26 isConnected()	45
5.9.3.27 pingServer()	45
5.9.3.28 reconnect()	46
5.9.3.29 returnIngredients()	46
5.9.3.30 saveReportToCSV()	46
5.9.3.31 syncOrderItems()	47
5.9.3.32 updateOrderStatus()	47
5.9.3.33 updateProductQuantity()	47
5.10 Класс TcpServer	48
5.10.1 Подробное описание	48
5.10.2 Конструктор(ы)	48
5.10.2.1 TcpServer()	48
5.10.2.2 ~TcpServer()	49
5.10.3 Методы	49
5.10.3.1 start()	49
5.10.3.2 stop()	49
5.11 Класс ValidateEx	49
5.11.1 Подробное описание	49
5.11.2 Конструктор(ы)	50
5.11.2.1 ValidateEx()	50
6 Файлы	51
6.1 dbworker.h	51
6.2 exceptions.h	52
6.3 logs.h	53
6.4 settings.h	53
6.5 tcpclient.h	54
6.6 tcpserver.h	56
6.7 dbworker.cpp	56
6.8 exceptions.cpp	72
6.9 logs.cpp	72
6.10 main.cpp	73

6.11 settings.cpp	74
6.12 tcpclient.cpp	75
6.13 tcpserver.cpp	84

Глава 1

Пользовательская документация (RestN)

1.1. Обзор проекта

Название: RestN

Тип: кроссплатформенное приложение для планшетов (Android / Windows) на Qt 6

Назначение: автоматизация приёма заказов, управления складом, формирования отчётности, контроля активных заказов и генерации чеков в ресторане.

Ключевые возможности:

- Авторизация сотрудников (официант, кладовщик, администратор)
- Оформление заказов с автоматическим списанием ингредиентов
- Возврат продуктов при удалении позиции из заказа
- Редактирование существующих заказов (добавление/удаление позиций)
- Управление складскими остатками (добавление, убавка, установка)
- Отчёты по выручке, популярности блюд, дефицитным продуктам и чекам
- Сохранение отчётов в CSV
- Просмотр активных заказов, изменение статуса («Готовится» → «Готов» → «Закрыт»)
- Формирование чека при закрытии заказа и его последующий просмотр
- Ролевое разграничение доступа
- Синхронизация данных по локальной сети (клиент-сервер)

1.2. Технологии программирования

При разработке использовались следующие инструменты и подходы:

- **Язык C++** – основная логика приложения, работа с сетью и базой данных.
- **Qt 6** – фреймворк для графического интерфейса (Qt Quick), сетевых сокетов и доступа к SQL.
- **PostgreSQL** – централизованная СУБД для серверного режима.
- **CMake** – система сборки проекта.
- **Qt Creator** – интегрированная среда разработки.

Архитектурные подходы:

Клиент-серверная архитектура, паттерн Model-View (данные передаются через QVariantList), механизм сигналов и слотов для коммуникации между объектами, а также Singleton для классов [Logs](#) и [Settings](#).

1.3. Структура проекта

```
project/
├── CMakeLists.txt
├── include/                # заголовочные файлы C++
│   ├── dbworker.h
│   ├── logs.h
│   ├── tcpclient.h
│   ├── tcpserver.h
│   ├── settings.h
│   └── exceptions.h
├── src/                   # исходные файлы C++
│   ├── dbworker.cpp
│   ├── logs.cpp
│   ├── tcpclient.cpp
│   ├── tcpserver.cpp
│   ├── settings.cpp
│   ├── exceptions.cpp
│   └── main.cpp
├── resources/             # ресурсы (иконки, стили)
│   ├── qml/               # файлы интерфейса QML
│   │   ├── Main.qml
│   │   ├── LoginPage.qml
│   │   ├── OrderPage.qml
│   │   ├── StockPage.qml
│   │   ├── ReportsPage.qml
│   │   └── ActiveOrdersPage.qml
│   └── icons/
├── tests/                 # модульные тесты
└── build/                 # каталог сборки (игнорируется)
```

1.4. Описание программных модулей

1.4.1. Модуль работы с базой данных (DBWorker)

Класс [DBWorker](#) предназначен для выполнения всех операций с базой данных PostgreSQL. Открывает соединение в конструкторе, поддерживает переподключение через `reconnect`. Основные методы:

- `checkUser(login, password)` – аутентификация с хешированием SHA3-256 + соль.
- `getFoods()` – возвращает список блюд с доступностью (рассчитывается по остаткам) и составом.
- `getProducts()` – список продуктов с остатками и минимальным порогом.
- `updateProductQuantity()` / `addProductQuantity()` – изменение остатков.
- `checkAndConsume(foodId, quantity)` – в транзакции проверяет наличие ингредиентов и списывает их.
- `returnIngredients(foodId, quantity)` – возвращает ингредиенты на склад.
- `createOrder()` / `addOrderItem()` – создание заказа.

- Методы отчётов: `getReportRevenue`, `getRevenueByDay`, `getFoodPopularity`, `getReportLowStock`.
- `saveReportToCSV()` – сохранение отчёта в CSV.
- `getOrders(status)` – возвращает список заказов с фильтром по статусу.
- `updateOrderStatus(orderId, newStatus)` – изменяет статус заказа.
- `getOrderItems(orderId)` – возвращает позиции заказа.
- `closeOrder(orderId, totalSum, items)` – закрывает заказ, формирует чек.
- `getCheckByOrderId(orderId)` – получает данные чека.
- `syncOrderItems(orderId, newItems)` – синхронизирует позиции заказа при редактировании.
- `getChecksByDateRange(from, to)` – формирует список чеков за период.

Все публичные методы обрабатывают исключения ([DatabaseEx](#), [ValidateEx](#)), логируют ошибки и испускают сигнал `errorOccurred` для QML.

1.4.2. Модуль сетевого клиента (TcpClient)

Класс [TcpClient](#) дублирует интерфейс [DBWorker](#), но вместо прямых запросов к БД отправляет JSON-команды на сервер. Соединение устанавливается методом `connectToServer(host, port)`. Протокол обмена: запрос/ответ в виде строк JSON, завершённых символом `\n`. Для синхронного ожидания используется `waitForResponse` с таймаутом 60 секунд. При потере соединения генерируется сигнал `errorOccurred`.

Пример запроса:

```
{"command": "checkUser", "requestId": 1, "data": {"login": "waiter1", "password": "123"}}
```

Ответ сервера:

```
{"response": true, "requestId": 1, "data": {"role": "waiter", "name": "Иванов", "userId": 5}}
```

1.4.3. Модуль сетевого сервера (TcpServer + ClientConnection)

[TcpServer](#) запускает серверный сокет и при подключении клиента создаёт объект [ClientConnection](#). В `ClientConnection::onReadyRead()` данные накапливаются в буфере, извлекаются по разделителю `\n`, парсится JSON и вызывается соответствующий метод [DBWorker](#). Результат отправляется обратно в виде JSON. Сервер способен параллельно обрабатывать несколько клиентов.

1.4.4. Модуль логирования (Logs)

Класс [Logs](#) записывает сообщения с уровнями `INF`, `WARN`, `ERR` в файл и дублирует их в консоль. При недоступности основного файла использует резервный путь в системной папке приложения. Формат строки: `yyyy-MM-dd hh:mm:ss [УРОВЕНЬ] сообщение`.

1.4.5. Модуль настроек (Settings)

Класс [Settings](#) предоставляет `Q_PROPERTY` для доступа из QML. Параметры (хост, порт БД, сервер, режим, пути к логам и отчётам) сохраняются через `QSettings` в системном реестре/конфигурационном файле. Методы `load()` и `save()` вызываются при запуске и при закрытии диалога настроек.

1.4.6. Пользовательский интерфейс (QML)

- **LoginPage.qml** – экран входа. Поля «Логин», «Пароль», кнопки «Войти» и «Настройки». При успехе испускает сигнал `loginSuccess`.
- **Main.qml** – главное окно. Панель инструментов с кнопками меню () и синхронизации (), выдвижное меню (`Drawer`), пункты которого зависят от роли пользователя.

- **OrderPage.qml** – страница заказа. Левая колонка – меню с фильтрами, правая – корзина. Кнопки «Добавить», «Удалить», «Оформить заказ». Поддерживается режим редактирования (при передаче `editMode` и `orderId`).
- **StockPage.qml** – страница склада. Список продуктов, поле ввода количества, кнопки «Добавить», «Убавить», «Установить». Продукты ниже минимального порога подсвечиваются.
- **ReportsPage.qml** – страница отчётов. Выбор типа отчёта (выручка, популярность блюд, малые остатки, чеки), фильтры (даты, блюдо, столик, официант), кнопка «Сформировать», вывод результатов и «Сохранить отчёт» (CSV).
- **ActiveOrdersPage.qml** – страница активных заказов. Отображает заказы, созданные сегодня, с фильтрацией по статусу. Для каждого заказа доступны действия: «Изменить» (открыть в режиме редактирования), «Готов» (изменить статус, только для администратора), «Оплатить» (закрыть заказ и сформировать чек), «Просмотр чека» (показать диалог с деталями).

1.5. Интеграция программных модулей

1.5.1. Подключаемые внешние модули

- Qt 6 (Core, Quick, Sql, Network)
- PostgreSQL client library (libpq) – для драйвера QPSQL.

1.5.2. Интеграция C++ и QML

Объекты регистрируются в контексте через `setContextProperty()`:

```
engine.rootContext()->setContextProperty("dbWorker", worker);
```

В QML методы вызываются напрямую (`dbWorker.checkUser(...)`). Сигналы (например, `errorOccurred`) перехватываются через `Connections`.

1.5.3. Клиент-серверное взаимодействие

Используется TCP. Каждое сообщение – JSON-строка, заканчивающаяся `\n`. Клиент нумерует запросы (`requestId`) и ожидает соответствующий ответ. Таймаут – 60 секунд. Сервер обрабатывает каждого клиента в отдельном объекте, что позволяет параллельно обслуживать множество подключений.

1.6. Руководство пользователя

1.6.1. Начальный запуск

1. Установите приложение на планшет (APK для Android или соберите под Windows).
2. При первом запуске откроется окно авторизации.
3. Если приложение работает в **клиентском режиме**, убедитесь, что **сервер** (другое устройство с программой в режиме сервера) активен и доступен по сети.
4. При необходимости настройте подключение через кнопку «Настройки».

1.6.2. Окно авторизации

- Поле «**Логин**» – введите имя пользователя.
- Поле «**Пароль**» – введите пароль.
- Кнопка «**Войти**» – проверка данных. При успехе открывается главное окно с соответствующими правами.
- Кнопка «**Настройки**» – открывает диалог конфигурации.

1.6.3. Диалог настроек

Группа «База данных» (видна только в серверном режиме): хост, порт, имя БД, пользователь, пароль; кнопка «Проверить подключение».

Группа «Сеть»: IP сервера, порт; флажок «Режим сервера»; кнопки «Проверить сервер», «Показать IP устройства».

Группа «Прочее»: путь к логам, папка для отчётов.

Сохранение – кнопки «ОК» (сохранить и закрыть) и «Отмена».

1.6.4. Главное окно и выдвижное меню

- **Верхняя панель:** кнопка (открыть меню), название «Ресторан», кнопка (синхронизация данных).
- **Выдвижное меню:**
 - Отображается ФИО пользователя.
 - Пункты зависят от роли: официант – «Оформление заказа», «Активные заказы»; кладовщик – «Склад»; администратор – «Оформление заказа», «Склад», «Отчёты», «Активные заказы».
 - Кнопки: «Настройки», «Выйти», «Закрыть приложение».
- Меню закрывается касанием вне его или повторным нажатием кнопки .

1.6.5. Оформление заказа (страница для официанта)

Левая колонка – меню:

- Фильтр по категории (Супы, Горячее, Салаты, Напитки, Все)
- Флажок «Только доступные»
- Список блюд: название, цена, категория, описание, состав. Недоступные блюда помечены красным.

Правая колонка – заказ:

- «Текущий заказ» – корзина с добавленными блюдами.
- Поля: «Столик №» (1–50), «Кол-во» (1–10), «Комментарий».
- Кнопка «Добавить» – проверяет наличие ингредиентов. Если после списания остатки падают ниже порога, появляется предупреждение (добавление разрешается). При нехватке продуктов – ошибка.
- Кнопка «Удалить» – удаляет позицию из корзины и возвращает ингредиенты.
- Кнопка «Оформить заказ» – фиксирует заказ в БД (статус «Готовится»), очищает корзину и обновляет меню.

Режим редактирования: если страница открыта с параметром `editMode: true` и передан `orderId`, загружается существующий заказ. Кнопка «Оформить заказ» заменяется на «Сохранить изменения». При сохранении вызывается `syncOrderItems`, который корректно списывает и возвращает ингредиенты в зависимости от изменений.

1.6.6. Управление складом (страница для кладовщика)

- Список продуктов: название, текущее количество (кг, л, шт.). Продукты ниже минимального порога подсвечены розовым.
- Выбор продукта – касание строки.
- Поле «Количество» (целое число 0–10000).
- Кнопки: «Добавить» (увеличить остаток), «Убавить» (уменьшить, если результат ≥ 0), «Установить» (задать точное значение). Изменения немедленно синхронизируются с сервером.

1.6.7. Отчёты (страница для администратора)

Типы отчётов (выпадающий список):

1. **Выручка** – фильтры: дата «с», «по», блюдо, столик, официант. Выводится общая выручка и разбивка по дням.
2. **Популярность блюд** – фильтры: дата «с», «по». Результат – список блюд с количеством проданных порций.
3. **Малые остатки** – без фильтров. Отображаются продукты, у которых остаток ниже порога.
4. **Чеки** – фильтр по датам («с» и «по»). Отображается таблица с номерами чеков, суммой, номером столика, официантом и датой. Для каждого чека доступна кнопка «Просмотр» с полным содержимым.

Сохранение отчёта: после формирования нажмите **«Сохранить отчёт»**, выберите имя файла. Файл будет создан в папке, указанной в настройках (по умолчанию `reports`). Формат – CSV (разделитель запятой, кодировка UTF-8).

1.6.8. Активные заказы (страница для официанта и администратора)

- Отображаются заказы, созданные сегодня.
- Фильтр по статусу: «Все», «Активный» (статус «Новый»), «Готовится», «Готов», «Закрит».
- Флажок «Скрыть закрытые» – убирает из списка заказы со статусом «Закрит».

Для каждого заказа доступны действия:

- **«Изменить»** – открывает страницу оформления заказа в режиме редактирования (для любых незакрытых заказов).
- **«Готов»** – переводит заказ из статуса «Готовится» в «Готов» (доступно только администратору).
- **«Оплатить»** – закрывает заказ (статус становится «Закрит»), формирует чек и сохраняет его в базе данных (доступно для заказов со статусом «Готов»).
- **«Просмотр чека»** – показывает сохранённый чек для закрытых заказов.

Страница автоматически обновляется при изменении статуса заказа или создании нового заказа.

1.7. Настройки и системные файлы

- **Файл настроек** – хранится через `QSettings` (реестр Windows или конфигурационный файл). Вручную не редактировать.
- **Лог-файл** – по умолчанию `logs.txt` в рабочей папке. При ошибке записи используется системная папка приложения.
- **Папка отчётов** – задаётся в диалоге настроек, используется для сохранения CSV-файлов.

1.8. Сборка проекта (для разработчиков)

Требования:

- Qt 6.8 или новее (Core, Quick, Sql, Network)
- CMake ≥ 3.16
- PostgreSQL (серверный режим) либо клиентская часть без БД.

Шаги сборки:

```
git clone <url>
cd RestN
mkdir build && cd build
cmake ..
cmake --build . --config Release
```

Запуск на Android: выберите в Qt Creator комплект Android, соберите APK, установите на планшет. На серверном устройстве должно быть установлено приложение (режим сервера) или отдельный PostgreSQL.

1.9. Контакты

Автор: Самсыка Родион Михайлович

Группа: И-42

Специальность: 09.02.07 Информационные системы и программирование

Научный руководитель: Фомин Андрей Томасович

Год: 2026

Глава 2

Иерархический список классов

2.1. Иерархия классов

Иерархия классов.

std::exception	
Exceptions	28
DatabaseEx	16
FileEx	29
NetworkEx	31
ValidateEx	49
QObject	
ClientConnection	15
DBWorker	16
Logs	30
Settings	32
TcpClient	36
TcpServer	48

Глава 3

Алфавитный указатель классов

3.1. Классы

Классы с их кратким описанием.

ClientConnection	Обработчик одного клиентского TCP-соединения	15
DatabaseEx	Исключение, связанное с ошибками базы данных	16
DBWorker	Класс для взаимодействия с базой данных PostgreSQL ресторана	16
Exceptions	Базовый класс исключений, принимающий QString	28
FileEx	Исключение, связанное с файловыми операциями	29
Logs	Сервис логирования сообщений в файл	30
NetworkEx	Исключение, связанное с сетевыми ошибками	31
Settings	Класс для хранения и управления настройками приложения	32
TcpClient	Клиент для удалённого взаимодействия с сервером приложения	36
TcpServer	TCP-сервер, принимающий подключения от клиентов	48
ValidateEx	Исключение валидации (некорректные данные, нехватка продуктов и т.п.)	49

Глава 4

Список файлов

4.1. Файлы

Полный список документированных файлов.

C:/Users/rodio/Desktop/W/Diplom/app/include/dbworker.h	51
C:/Users/rodio/Desktop/W/Diplom/app/include/exceptions.h	52
C:/Users/rodio/Desktop/W/Diplom/app/include/logs.h	53
C:/Users/rodio/Desktop/W/Diplom/app/include/settings.h	53
C:/Users/rodio/Desktop/W/Diplom/app/include/tcpclient.h	54
C:/Users/rodio/Desktop/W/Diplom/app/include/tcpserver.h	56
C:/Users/rodio/Desktop/W/Diplom/app/src/dbworker.cpp	56
C:/Users/rodio/Desktop/W/Diplom/app/src/exceptions.cpp	72
C:/Users/rodio/Desktop/W/Diplom/app/src/logs.cpp	72
C:/Users/rodio/Desktop/W/Diplom/app/src/main.cpp	73
C:/Users/rodio/Desktop/W/Diplom/app/src/settings.cpp	74
C:/Users/rodio/Desktop/W/Diplom/app/src/tcpclient.cpp	75
C:/Users/rodio/Desktop/W/Diplom/app/src/tcpserver.cpp	84

Глава 5

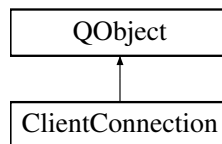
Классы

5.1. Класс ClientConnection

Обработчик одного клиентского TCP-соединения.

```
#include <tcpserver.h>
```

Граф наследования: ClientConnection:



Открытые члены

- [ClientConnection](#) ([Logs](#) *logger, qintptr socketDescriptor, [DBWorker](#) *dbWorker, QObject *parent=nullptr)
Конструктор.

5.1.1. Подробное описание

Обработчик одного клиентского TCP-соединения.

Принимает JSON-команды, вызывает соответствующие методы [DBWorker](#) и отправляет обратно ответ.

См. определение в файле [tcpserver.h](#) строка 19

5.1.2. Конструктор(ы)

5.1.2.1. ClientConnection()

```
ClientConnection::ClientConnection (  
    Logs * logger,  
    qintptr socketDescriptor,  
    DBWorker * dbWorker,  
    QObject * parent = nullptr)
```

Конструктор.

Аргументы

<i>logger</i>	Логгер.
<i>socketDescriptor</i>	Дескриптор сокета.
<i>dbWorker</i>	Указатель на объект работы с БД.
<i>parent</i>	Родительский объект.

См. определение в файле [tcpserver.cpp](#) строка 7

5.1.2.2. ~ClientConnection()

ClientConnection::~~ClientConnection ()

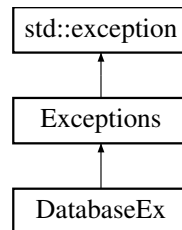
См. определение в файле [tcpserver.cpp](#) строка 16

5.2. Класс DatabaseEx

Исключение, связанное с ошибками базы данных.

#include <exceptions.h>

Граф наследования:DatabaseEx:



Открытые члены

- [DatabaseEx](#) (const QString &message)

Открытые члены унаследованные от [Exceptions](#)

- [Exceptions](#) (const QString &message)
Конструктор.
- virtual const char * [what](#) () const noexcept override

5.2.1. Подробное описание

Исключение, связанное с ошибками базы данных.

См. определение в файле [exceptions.h](#) строка 35

5.2.2. Конструктор(ы)

5.2.2.1. DatabaseEx()

DatabaseEx::DatabaseEx (
const QString & message) [inline], [explicit]

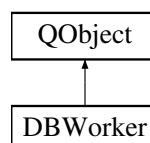
См. определение в файле [exceptions.h](#) строка 38

5.3. Класс DBWorker

Класс для взаимодействия с базой данных PostgreSQL ресторана.

#include <dbworker.h>

Граф наследования:DBWorker:



Сигналы

- void **errorOccurred** (const QString &message)
Сигнал, передающий текст ошибки для отображения в интерфейсе.
- void **foodAvailabilityChanged** ()
Сообщает об изменении доступности блюд (например, после обновления остатков).
- void **ordersChanged** ()

Открытые члены

- DBWorker (Logs *logger, QObject *parent=nullptr)
Конструктор.
- Q_INVOKABLE bool **checkUser** (const QString &login, const QString &password)
Проверяет логин и пароль пользователя.
- Q_INVOKABLE QString **getUserName** () const
Возвращает имя текущего пользователя.
- Q_INVOKABLE QString **getUserRole** () const
Возвращает роль текущего пользователя.
- Q_INVOKABLE int **getUserId** () const
Возвращает идентификатор текущего пользователя.
- Q_INVOKABLE bool **checkAndConsume** (int foodId, int quantity)
Проверяет наличие ингредиентов и списывает их со склада.
- Q_INVOKABLE bool **returnIngredients** (int foodId, int quantity)
Возвращает ингредиенты на склад при отмене позиции заказа.
- Q_INVOKABLE bool **refreshAllFoodsAvailability** ()
Пересчитывает доступность всех блюд на основе остатков продуктов.
- Q_INVOKABLE bool **reconnect** (const QString &host, int port, const QString &dbName, const QString &user, const QString &password)
Переподключает базу данных с новыми параметрами.
- Q_INVOKABLE QStringList **getLocalIps** ()
Возвращает список локальных IPv4-адресов.
- Q_INVOKABLE bool **pingServer** (const QString &host, int port)
Проверяет доступность удалённого сервера (заглушка, всегда true).
- Q_INVOKABLE QVariantList **getFoods** ()
Загружает меню с составом и доступностью.
- Q_INVOKABLE QVariantList **getProducts** ()
Загружает складские остатки продуктов.
- Q_INVOKABLE QVariantList **getDishesList** ()
Возвращает краткий список блюд (ID и название).
- Q_INVOKABLE QVariantList **getWaitersList** ()
Возвращает список официантов.
- Q_INVOKABLE bool **updateProductQuantity** (int productId, int newQuantity)
Устанавливает новое абсолютное количество продукта.
- Q_INVOKABLE bool **addProductQuantity** (int productId, int delta)
Изменяет количество продукта на заданную величину.
- Q_INVOKABLE int **createOrder** (int tableNumber, int userId, const QString &specialRequests, double totalCost)
Создаёт новый заказ.
- Q_INVOKABLE bool **addOrderItem** (int orderId, int foodId, int quantity, double price)
Добавляет позицию в заказ.
- Q_INVOKABLE QVariantMap **getReportRevenue** (const QString &from, const QString &to, int dishId, int tableNumber, int waiterId)

- Возвращает общую выручку за период с применёнными фильтрами.*
- Q_INVOKABLE QVariantList [getRevenueByDay](#) (const QString &from, const QString &to, int dishId, int tableNumber, int waiterId)
- Возвращает выручку, сгруппированную по дням.*
- Q_INVOKABLE QVariantList [getFoodPopularity](#) (const QString &from, const QString &to)
- Получает популярность блюд за период.*
- Q_INVOKABLE QVariantList [getReportLowStock](#) ()
- Возвращает продукты, количество которых ниже минимального порога.*
- Q_INVOKABLE QVariantList [getLowStockWarning](#) (int foodId, int quantity)
- Предупреждает о продуктах, остаток которых станет ниже порога после приготовления блюда.*
- Q_INVOKABLE bool [saveReportToCSV](#) (const QString &reportType, const QString &from, const QString &to, int dishId, int tableNumber, int waiterId, const QString &fileName)
- Сохраняет отчёт в CSV-файл.*
- Q_INVOKABLE QVariantList [getOrders](#) (const QString &status=QString())
- Возвращает список заказов с возможностью фильтрации по статусу.*
- Q_INVOKABLE bool [updateOrderStatus](#) (int orderId, const QString &newStatus)
- Обновляет статус заказа.*
- Q_INVOKABLE QVariantList [getOrderItems](#) (int orderId)
- Возвращает позиции заказа.*
- Q_INVOKABLE bool [closeOrder](#) (int orderId, double totalSum, const QVariantList &items)
- Закрывает заказ, создаёт чек и переводит статус в "Закрыт".*
- Q_INVOKABLE QVariantMap [getCheckByOrderId](#) (int orderId)
- Возвращает чек по ID заказа.*
- Q_INVOKABLE QVariantList [getAllOrdersForReport](#) ()
- Упрощённый список всех заказов (для отчётов).*
- Q_INVOKABLE bool [syncOrderItems](#) (int orderId, const QVariantList &newItems)
- Синхронизирует позиции заказа при редактировании (вычисляет разницу).*
- Q_INVOKABLE QVariantList [getClosedOrders](#) ()
- Возвращает закрытые заказы (статус "Закрыт").*
- Q_INVOKABLE void [updateConnectionParams](#) (const QString &host, int port, const QString &dbName, const QString &user, const QString &password)
- Обновляет параметры подключения к БД без переподключения.*
- Q_INVOKABLE QVariantList [getChecksByDateRange](#) (const QString &from, const QString &to)
- Возвращает чеки за период с деталями заказа.*
- int [generateOrderNumber](#) ()
- Генерирует уникальный номер заказа для текущего дня (начинается с 1 каждый день).*

5.3.1. Подробное описание

Класс для взаимодействия с базой данных PostgreSQL ресторана.

Предоставляет методы аутентификации, работы с меню, складом, заказами и отчётами. Хранит информацию о текущем пользователе.

См. определение в файле [dbworker.h](#) строка 24

5.3.2. Конструктор(ы)

5.3.2.1. DBWorker()

```
DBWorker::DBWorker (
    Logs * logger,
    QObject * parent = nullptr)
```

Конструктор.

Аргументы

<i>logger</i>	Указатель на объект логирования.
<i>parent</i>	Родительский объект (по умолчанию nullptr).

См. определение в файле [dbworker.cpp](#) строка 9

5.3.2.2. ~DBWorker()

`DBWorker::~DBWorker ()`

См. определение в файле [dbworker.cpp](#) строка 32

5.3.3. Методы**5.3.3.1. addOrderItem()**

```
bool DBWorker::addOrderItem (  
    int orderId,  
    int foodId,  
    int quantity,  
    double price)
```

Добавляет позицию в заказ.

Аргументы

<i>orderId</i>	ID заказа.
<i>foodId</i>	ID блюда.
<i>quantity</i>	Количество.
<i>price</i>	Цена за единицу.

Возвращает

`true` при успешной вставке.

См. определение в файле [dbworker.cpp](#) строка 256

5.3.3.2. addProductQuantity()

```
bool DBWorker::addProductQuantity (  
    int productId,  
    int delta)
```

Изменяет количество продукта на заданную величину.

Аргументы

<i>productId</i>	ID продукта.
<i>delta</i>	Изменение (может быть отрицательным).

Возвращает

`true` при успехе.

См. определение в файле [dbworker.cpp](#) строка 199

5.3.3.3. checkAndConsume()

```
bool DBWorker::checkAndConsume (  
    int foodId,  
    int quantity)
```

Проверяет наличие ингредиентов и списывает их со склада.

Аргументы

<i>foodId</i>	ID блюда.
<i>quantity</i>	Количество порций.

Возвращает

`true`, если списание прошло успешно.

См. определение в файле [dbworker.cpp](#) строка 364

5.3.3.4. checkUser()

```
bool DBWorker::checkUser (  
    const QString & login,  
    const QString & password)
```

Проверяет логин и пароль пользователя.

Аргументы

<i>login</i>	Логин.
<i>password</i>	Пароль.

Возвращает

`true` при успешной аутентификации.

См. определение в файле [dbworker.cpp](#) строка 38

5.3.3.5. closeOrder()

```
bool DBWorker::closeOrder (  
    int orderId,  
    double totalSum,  
    const QVariantList & items)
```

Закрывает заказ, создаёт чек и переводит статус в "Закрыт".

Аргументы

<i>orderId</i>	ID заказа.
<i>totalSum</i>	Итоговая сумма заказа.
<i>items</i>	Список позиций заказа (каждый элемент: {name, quantity, price}).

Возвращает

`true` при успешном закрытии.

См. определение в файле [dbworker.cpp](#) строка 1050

5.3.3.6. createOrder()

```
int DBWorker::createOrder (
    int tableNumber,
    int userId,
    const QString & specialRequests,
    double totalCost)
```

Создаёт новый заказ.

Аргументы

<i>tableNumber</i>	Номер столика.
<i>userId</i>	ID официанта.
<i>specialRequests</i>	Особые пожелания.
<i>totalCost</i>	Итоговая стоимость.

Возвращает

ID созданного заказа или -1 при ошибке.

См. определение в файле [dbworker.cpp](#) строка 223

5.3.3.7. generateOrderNumber()

```
int DBWorker::generateOrderNumber ()
```

Генерирует уникальный номер заказа для текущего дня (начинается с 1 каждый день).

См. определение в файле [dbworker.cpp](#) строка 922

5.3.3.8. getAllOrdersForReport()

```
QVariantList DBWorker::getAllOrdersForReport ()
```

Упрощённый список всех заказов (для отчётов).

Возвращает

Результат вызова [getOrders\(\)](#) без фильтра.

См. определение в файле [dbworker.cpp](#) строка 1254

5.3.3.9. getCheckByOrderId()

```
QVariantMap DBWorker::getCheckByOrderId (
    int orderId)
```

Возвращает чек по ID заказа.

Аргументы

<i>orderId</i>	ID заказа.
----------------	------------

Возвращает

Словарь с ключами `checkId`, `checkNumber`, `orderNumber`, `totalSum`, `checkData`, `createdAt`.

См. определение в файле [dbworker.cpp](#) строка 1115

5.3.3.10. getChecksByDateRange()

```
QVariantList DBWorker::getChecksByDateRange (
    const QString & from,
    const QString & to)
```

Возвращает чеки за период с деталями заказа.

Аргументы

<i>from</i>	Начальная дата (ГГГГ-ММ-ДД).
<i>to</i>	Конечная дата.

Возвращает

Список словарей с полями checkNumber, orderNumber, totalSum, createdAt, tableNumber, waiterName, orderId.

См. определение в файле [dbworker.cpp](#) строка 1274

5.3.3.11. getClosedOrders()

```
QVariantList DBWorker::getClosedOrders ()
```

Возвращает закрытые заказы (статус "Закрыт").

Возвращает

Список заказов.

См. определение в файле [dbworker.cpp](#) строка 1259

5.3.3.12. getDishesList()

```
QVariantList DBWorker::getDishesList ()
```

Возвращает краткий список блюд (ID и название).
См. определение в файле [dbworker.cpp](#) строка 613

5.3.3.13. getFoodPopularity()

```
QVariantList DBWorker::getFoodPopularity (
    const QString & from,
    const QString & to)
```

Получает популярность блюд за период.

Аргументы

<i>from</i>	Начальная дата.
<i>to</i>	Конечная дата.

Возвращает

Список словарей с ключами "foodName" и "count".

См. определение в файле [dbworker.cpp](#) строка 314

5.3.3.14. getFoods()

```
QVariantList DBWorker::getFoods ()
```

Загружает меню с составом и доступностью.
См. определение в файле [dbworker.cpp](#) строка 81

5.3.3.15. getLocalIps()

```
QStringList DBWorker::getLocalIps ()
```

Возвращает список локальных IPv4-адресов.

См. определение в файле [dbworker.cpp](#) строка 824

5.3.3.16. getLowStockWarning()

```
QVariantList DBWorker::getLowStockWarning (  
    int foodId,  
    int quantity)
```

Предупреждает о продуктах, остаток которых станет ниже порога после приготовления блюда.

Аргументы

<i>foodId</i>	ID блюда.
<i>quantity</i>	Количество порций.

Возвращает

Список предупреждений с полями `productName`, `currentQuantity`, `newQuantity`, `minThreshold`.

См. определение в файле [dbworker.cpp](#) строка 564

5.3.3.17. getOrderItems()

```
QVariantList DBWorker::getOrderItems (  
    int orderId)
```

Возвращает позиции заказа.

Аргументы

<i>orderId</i>	ID заказа.
----------------	------------

Возвращает

Список словарей с полями `orderFoodId`, `foodId`, `name`, `quantity`, `price`.

См. определение в файле [dbworker.cpp](#) строка 1015

5.3.3.18. getOrders()

```
QVariantList DBWorker::getOrders (  
    const QString & status = QString())
```

Возвращает список заказов с возможностью фильтрации по статусу.

Аргументы

<i>status</i>	Статус заказа (пустая строка – все заказы).
---------------	---

Возвращает

Список словарей с полями `orderId`, `orderNumber`, `tableNumber`, `orderDate`, `specialRequests`, `totalCost`, `status`, `waiterName`.

См. определение в файле [dbworker.cpp](#) строка 947

5.3.3.19. getProducts()

`QVariantList DBWorker::getProducts ()`

Загружает складские остатки продуктов.

См. определение в файле [dbworker.cpp](#) строка 141

5.3.3.20. getReportLowStock()

`QVariantList DBWorker::getReportLowStock ()`

Возвращает продукты, количество которых ниже минимального порога.

См. определение в файле [dbworker.cpp](#) строка 283

5.3.3.21. getReportRevenue()

`QVariantMap DBWorker::getReportRevenue (`

`const QString & from,`

`const QString & to,`

`int dishId,`

`int tableNumber,`

`int waiterId)`

Возвращает общую выручку за период с применёнными фильтрами.

Аргументы

<i>from</i>	Начальная дата (ГГГГ-ММ-ДД).
<i>to</i>	Конечная дата.
<i>dishId</i>	ID блюда (0 – без фильтра).
<i>tableNumber</i>	Номер столика (0 – без фильтра).
<i>waiterId</i>	ID официанта (0 – без фильтра).

Возвращает

Словарь с ключом "totalRevenue" или "error".

См. определение в файле [dbworker.cpp](#) строка 673

5.3.3.22. getRevenueByDay()

`QVariantList DBWorker::getRevenueByDay (`

`const QString & from,`

`const QString & to,`

`int dishId,`

`int tableNumber,`

`int waiterId)`

Возвращает выручку, сгруппированную по дням.

Аргументы

<i>from</i>	Начальная дата.
<i>to</i>	Конечная дата.
<i>dishId</i>	Фильтр по блюду (0 – без фильтра).
<i>tableNumber</i>	Фильтр по столику (0 – без фильтра).
<i>waiterId</i>	Фильтр по официанту (0 – без фильтра).

Возвращает

Список словарей с полями "date" и "total".

См. определение в файле [dbworker.cpp](#) строка 737

5.3.3.23. getUserId()

```
Q_INVOKABLE int DBWorker::getUserId () const [inline]
```

Возвращает идентификатор текущего пользователя.

См. определение в файле [dbworker.h](#) строка 49

5.3.3.24. getUserName()

```
Q_INVOKABLE QString DBWorker::getUserName () const [inline]
```

Возвращает имя текущего пользователя.

См. определение в файле [dbworker.h](#) строка 45

5.3.3.25. getUserRole()

```
Q_INVOKABLE QString DBWorker::getUserRole () const [inline]
```

Возвращает роль текущего пользователя.

См. определение в файле [dbworker.h](#) строка 47

5.3.3.26. getWaitersList()

```
QVariantList DBWorker::getWaitersList ()
```

Возвращает список официантов.

См. определение в файле [dbworker.cpp](#) строка 643

5.3.3.27. pingServer()

```
bool DBWorker::pingServer (  
    const QString & host,  
    int port)
```

Проверяет доступность удалённого сервера (заглушка, всегда true).

Аргументы

<i>host</i>	Адрес.
<i>port</i>	Порт.

Возвращает

true.

См. определение в файле [dbworker.cpp](#) строка 840

5.3.3.28. reconnect()

```
bool DBWorker::reconnect (  
    const QString & host,  
    int port,  
    const QString & dbName,  
    const QString & user,  
    const QString & password)
```

Переподключает базу данных с новыми параметрами.

Аргументы

<i>host</i>	Хост.
<i>port</i>	Порт.
<i>dbName</i>	Имя базы данных.
<i>user</i>	Пользователь.
<i>password</i>	Пароль.

Возвращает

`true` при успешном подключении.

См. определение в файле [dbworker.cpp](#) строка 801

5.3.3.29. refreshAllFoodsAvailability()

```
bool DBWorker::refreshAllFoodsAvailability ()
```

Пересчитывает доступность всех блюд на основе остатков продуктов.

См. определение в файле [dbworker.cpp](#) строка 497

5.3.3.30. returnIngredients()

```
bool DBWorker::returnIngredients (
    int foodId,
    int quantity)
```

Возвращает ингредиенты на склад при отмене позиции заказа.

Аргументы

<i>foodId</i>	ID блюда.
<i>quantity</i>	Количество порций.

Возвращает

`true` в случае успеха.

См. определение в файле [dbworker.cpp](#) строка 441

5.3.3.31. saveReportToCSV()

```
bool DBWorker::saveReportToCSV (
    const QString & reportType,
    const QString & from,
    const QString & to,
    int dishId,
    int tableNumber,
    int waiterId,
    const QString & fileName)
```

Сохраняет отчёт в CSV-файл.

Аргументы

<i>reportType</i>	Тип отчёта ("revenue", "popularity", "lowstock").
<i>from</i>	Начальная дата.
<i>to</i>	Конечная дата.

<i>dishId</i>	Фильтр по блюду.
<i>tableNumber</i>	Фильтр по столику.
<i>waiterId</i>	Фильтр по официанту.
<i>fileName</i>	Полный путь к выходному файлу.

Возвращает

true при успешном сохранении.

См. определение в файле [dbworker.cpp](#) строка 847

5.3.3.32. syncOrderItems()

```
bool DBWorker::syncOrderItems (
    int orderId,
    const QVariantList & newItems)
```

Синхронизирует позиции заказа при редактировании (вычисляет разницу).

Аргументы

<i>orderId</i>	ID заказа.
<i>newItems</i>	Новый список позиций (каждый элемент: {foodId, quantity, price, name}).

Возвращает

true при успешной синхронизации.

См. определение в файле [dbworker.cpp](#) строка 1147

5.3.3.33. updateConnectionParams()

```
void DBWorker::updateConnectionParams (
    const QString & host,
    int port,
    const QString & dbName,
    const QString & user,
    const QString & password)
```

Обновляет параметры подключения к БД без переподключения.

Аргументы

<i>host</i>	Хост.
<i>port</i>	Порт.
<i>dbName</i>	Имя БД.
<i>user</i>	Пользователь.
<i>password</i>	Пароль.

См. определение в файле [dbworker.cpp](#) строка 1264

5.3.3.34. updateOrderStatus()

```
bool DBWorker::updateOrderStatus (
    int orderId,
    const QString & newStatus)
```

Обновляет статус заказа.

Аргументы

<i>orderId</i>	ID заказа.
<i>newStatus</i>	Новый статус ("Новый", "Готовится", "Готов", "Закрыт").

Возвращает

`true` при успешном обновлении.

См. определение в файле [dbworker.cpp](#) строка 990

5.3.3.35. updateProductQuantity()

```
bool DBWorker::updateProductQuantity (
    int productId,
    int newQuantity)
```

Устанавливает новое абсолютное количество продукта.

Аргументы

<i>productId</i>	ID продукта.
<i>newQuantity</i>	Новое количество.

Возвращает

`true` при успешном обновлении.

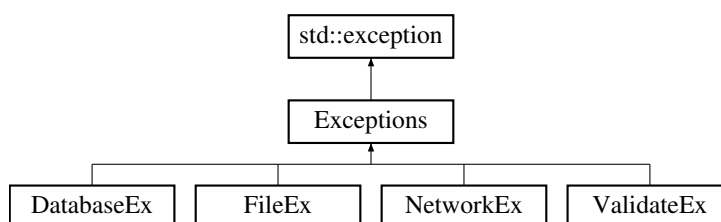
См. определение в файле [dbworker.cpp](#) строка 175

5.4. Класс Exceptions

Базовый класс исключений, принимающий QString.

```
#include <exceptions.h>
```

Граф наследования: Exceptions:



Открытые члены

- [Exceptions](#) (const QString &message)
Конструктор.
- virtual const char * [what](#) () const noexcept override

5.4.1. Подробное описание

Базовый класс исключений, принимающий QString.

Хранит сообщение в `std::string` для совместимости с `std::exception`. Все специализированные исключения наследуют этот класс.

См. определение в файле [exceptions.h](#) строка 13

5.4.2. Конструктор(ы)

5.4.2.1. Exceptions()

```
Exceptions::Exceptions (
    const QString & message) [explicit]
```

Конструктор.

Аргументы

<i>message</i>	Описание ошибки.
----------------	------------------

См. определение в файле [exceptions.cpp](#) строка 3

5.4.3. Методы

5.4.3.1. what()

```
const char * Exceptions::what () const [override], [virtual], [noexcept]
```

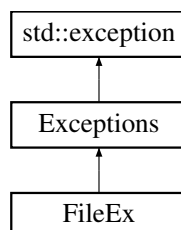
См. определение в файле [exceptions.cpp](#) строка 8

5.5. Класс FileEx

Исключение, связанное с файловыми операциями.

```
#include <exceptions.h>
```

Граф наследования:FileEx:



Открытые члены

- [FileEx](#) (const QString &message)

Открытые члены унаследованные от [Exceptions](#)

- [Exceptions](#) (const QString &message)
Конструктор.
- virtual const char * [what](#) () const noexcept override

5.5.1. Подробное описание

Исключение, связанное с файловыми операциями.

См. определение в файле [exceptions.h](#) строка 49

5.5.2. Конструктор(ы)

5.5.2.1. FileEx()

```
FileEx::FileEx (
    const QString & message) [inline], [explicit]
```

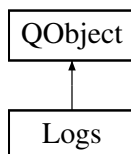
См. определение в файле [exceptions.h](#) строка 52

5.6. Класс Logs

Сервис логирования сообщений в файл.

```
#include <logs.h>
```

Граф наследования:Logs:



Открытые члены

- [Logs](#) (QObject *parent=nullptr)
- Q_INVOKABLE void [info](#) (const QString &msg)
Записывает информационное сообщение.
- Q_INVOKABLE void [warning](#) (const QString &msg)
Записывает предупреждение.
- Q_INVOKABLE void [error](#) (const QString &msg)
Записывает сообщение об ошибке.
- Q_INVOKABLE void [setLogPath](#) (const QString &path)
Устанавливает путь к лог-файлу и перепускает его.

5.6.1. Подробное описание

Сервис логирования сообщений в файл.

Записывает сообщения с меткой времени и уровнем (INF, WARN, ERR). Если основной файл не может быть открыт, использует резервный путь в директории AppLocalDataLocation.

См. определение в файле [logs.h](#) строка 16

5.6.2. Конструктор(ы)

5.6.2.1. Logs()

```
Logs::Logs (
    QObject * parent = nullptr) [explicit]
```

См. определение в файле [logs.cpp](#) строка 7

5.6.2.2. ~Logs()

```
Logs::~Logs ()
```

См. определение в файле [logs.cpp](#) строка 12

5.6.3. Методы

5.6.3.1. error()

```
void Logs::error (
    const QString & msg)
```

Записывает сообщение об ошибке.

См. определение в файле [logs.cpp](#) строка 74

5.6.3.2. info()

```
void Logs::info (
    const QString & msg)
```

Записывает информационное сообщение.

См. определение в файле [logs.cpp](#) строка 72

5.6.3.3. setLogPath()

```
void Logs::setLogPath (
    const QString & path)
```

Устанавливает путь к лог-файлу и переоткрывает его.

Аргументы

<i>path</i>	Новый путь.
-------------	-------------

См. определение в файле [logs.cpp](#) строка 18

5.6.3.4. warning()

```
void Logs::warning (
    const QString & msg)
```

Записывает предупреждение.

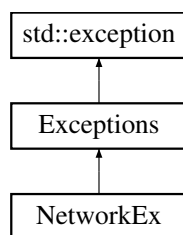
См. определение в файле [logs.cpp](#) строка 73

5.7. Класс NetworkEx

Исключение, связанное с сетевыми ошибками.

```
#include <exceptions.h>
```

Граф наследования:NetworkEx:



Открытые члены

- [NetworkEx](#) (const QString &message)

Открытые члены унаследованные от [Exceptions](#)

- [Exceptions](#) (const QString &message)
Конструктор.
- virtual const char * [what](#) () const noexcept override

5.7.1. Подробное описание

Исключение, связанное с сетевыми ошибками.

См. определение в файле [exceptions.h](#) строка 42

5.7.2. Конструктор(ы)

5.7.2.1. NetworkEx()

```
NetworkEx::NetworkEx (
    const QString & message) [inline], [explicit]
```

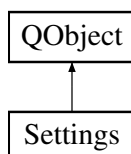
См. определение в файле [exceptions.h](#) строка 45

5.8. Класс Settings

Класс для хранения и управления настройками приложения.

```
#include <settings.h>
```

Граф наследования: Settings:



Сигналы

- void **dbHostChanged** ()
- void **dbPortChanged** ()
- void **dbNameChanged** ()
- void **dbUserChanged** ()
- void **dbPasswordChanged** ()
- void **isServerModeChanged** ()
- void **logPathChanged** ()
- void **serverHostChanged** ()
- void **serverPortChanged** ()
- void **reportsPathChanged** ()

Открытые члены

- [Settings](#) ([Logs](#) *logger, QObject *parent=nullptr)
Конструктор. Автоматически вызывает [load\(\)](#).
- QString [dbHost](#) () const
- void [setDbHost](#) (const QString &host)
Устанавливает хост БД.
- int [dbPort](#) () const
- void [setDbPort](#) (int port)
Устанавливает порт БД.
- QString [dbName](#) () const
- void [setDbName](#) (const QString &name)
Устанавливает имя БД.
- QString [dbUser](#) () const
- void [setDbUser](#) (const QString &user)
Устанавливает пользователя БД.
- QString [dbPassword](#) () const
- void [setDbPassword](#) (const QString &password)
Устанавливает пароль БД.
- bool [isServerMode](#) () const
- void [setIsServerMode](#) (bool mode)
Включает/выключает серверный режим.
- QString [logPath](#) () const
- void [setLogPath](#) (const QString &path)
Задаёт путь к лог-файлу.
- QString [serverHost](#) () const
- void [setServerHost](#) (const QString &host)
Устанавливает IP-адрес сервера.
- int [serverPort](#) () const

- void [setServerPort](#) (int port)
Устанавливает порт сервера.
- QString [reportsPath](#) () const
- void [setReportsPath](#) (const QString &path)
Задаёт каталог для сохранения отчётов.
- Q_INVOKABLE void [load](#) ()
Загружает все настройки из QSettings.
- Q_INVOKABLE void [save](#) ()
Сохраняет текущие настройки в QSettings.

Свойства

- QString [dbHost](#)
- int [dbPort](#)
- QString [dbName](#)
- QString [dbUser](#)
- QString [dbPassword](#)
- bool [isServerMode](#)
- QString [logPath](#)
- QString [serverHost](#)
- int [serverPort](#)
- QString [reportsPath](#)

5.8.1. Подробное описание

Класс для хранения и управления настройками приложения.

Обёртка над QSettings, предоставляющая свойства QML для доступа к параметрам базы данных, сети и путям.

См. определение в файле [settings.h](#) строка 18

5.8.2. Конструктор(ы)

5.8.2.1. Settings()

```
Settings::Settings (
    Logs * logger,
    QObject * parent = nullptr) [explicit]
```

Конструктор. Автоматически вызывает [load\(\)](#).

Аргументы

<i>logger</i>	Указатель на логгер.
<i>parent</i>	Родительский объект.

См. определение в файле [settings.cpp](#) строка 4

5.8.3. Методы

5.8.3.1. dbHost()

```
QString Settings::dbHost () const [inline]
```

См. определение в файле [settings.h](#) строка 41

5.8.3.2. dbName()

```
QString Settings::dbName () const [inline]
```

См. определение в файле [settings.h](#) строка 47

5.8.3.3. dbPassword()

```
QString Settings::dbPassword () const [inline]
```

См. определение в файле [settings.h](#) строка 53

5.8.3.4. dbPort()

```
int Settings::dbPort () const [inline]
```

См. определение в файле [settings.h](#) строка 44

5.8.3.5. dbUser()

```
QString Settings::dbUser () const [inline]
```

См. определение в файле [settings.h](#) строка 50

5.8.3.6. isServerMode()

```
bool Settings::isServerMode () const [inline]
```

См. определение в файле [settings.h](#) строка 56

5.8.3.7. load()

```
void Settings::load ()
```

Загружает все настройки из QSettings.

См. определение в файле [settings.cpp](#) строка 68

5.8.3.8. logPath()

```
QString Settings::logPath () const [inline]
```

См. определение в файле [settings.h](#) строка 59

5.8.3.9. reportsPath()

```
QString Settings::reportsPath () const [inline]
```

См. определение в файле [settings.h](#) строка 68

5.8.3.10. save()

```
void Settings::save ()
```

Сохраняет текущие настройки в QSettings.

См. определение в файле [settings.cpp](#) строка 51

5.8.3.11. serverHost()

```
QString Settings::serverHost () const [inline]
```

См. определение в файле [settings.h](#) строка 62

5.8.3.12. serverPort()

```
int Settings::serverPort () const [inline]
```

См. определение в файле [settings.h](#) строка 65

5.8.3.13. setDbHost()

```
void Settings::setDbHost (  
    const QString & host)
```

Устанавливает хост БД.

См. определение в файле [settings.cpp](#) строка 9

5.8.3.14. setDbName()

```
void Settings::setDbName (  
    const QString & name)
```

Устанавливает имя БД.

См. определение в файле [settings.cpp](#) строка 23

5.8.3.15. setDbPassword()

```
void Settings::setDbPassword (  
    const QString & password)
```

Устанавливает пароль БД.

См. определение в файле [settings.cpp](#) строка 37

5.8.3.16. setDbPort()

```
void Settings::setDbPort (  
    int port)
```

Устанавливает порт БД.

См. определение в файле [settings.cpp](#) строка 16

5.8.3.17. setDbUser()

```
void Settings::setDbUser (  
    const QString & user)
```

Устанавливает пользователя БД.

См. определение в файле [settings.cpp](#) строка 30

5.8.3.18. setIsServerMode()

```
void Settings::setIsServerMode (  
    bool mode)
```

Включает/выключает серверный режим.

См. определение в файле [settings.cpp](#) строка 84

5.8.3.19. setLogPath()

```
void Settings::setLogPath (  
    const QString & path)
```

Задаёт путь к лог-файлу.

См. определение в файле [settings.cpp](#) строка 91

5.8.3.20. setReportsPath()

```
void Settings::setReportsPath (  
    const QString & path)
```

Задаёт каталог для сохранения отчётов.

См. определение в файле [settings.cpp](#) строка 44

5.8.3.21. setServerHost()

```
void Settings::setServerHost (  
    const QString & host)
```

Устанавливает IP-адрес сервера.

См. определение в файле [settings.cpp](#) строка 98

5.8.3.22. setServerPort()

```
void Settings::setServerPort (  
    int port)
```

Устанавливает порт сервера.

См. определение в файле [settings.cpp](#) строка 104

5.8.4. Полный список свойств

5.8.4.1. dbHost

QString Settings::dbHost [read], [write]

См. определение в файле [settings.h](#) строка 21

5.8.4.2. dbName

QString Settings::dbName [read], [write]

См. определение в файле [settings.h](#) строка 23

5.8.4.3. dbPassword

QString Settings::dbPassword [read], [write]

См. определение в файле [settings.h](#) строка 25

5.8.4.4. dbPort

int Settings::dbPort [read], [write]

См. определение в файле [settings.h](#) строка 22

5.8.4.5. dbUser

QString Settings::dbUser [read], [write]

См. определение в файле [settings.h](#) строка 24

5.8.4.6. isServerMode

bool Settings::isServerMode [read], [write]

См. определение в файле [settings.h](#) строка 26

5.8.4.7. logPath

QString Settings::logPath [read], [write]

См. определение в файле [settings.h](#) строка 27

5.8.4.8. reportsPath

QString Settings::reportsPath [read], [write]

См. определение в файле [settings.h](#) строка 30

5.8.4.9. serverHost

QString Settings::serverHost [read], [write]

См. определение в файле [settings.h](#) строка 28

5.8.4.10. serverPort

int Settings::serverPort [read], [write]

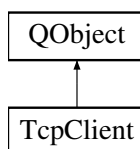
См. определение в файле [settings.h](#) строка 29

5.9. Класс TcpClient

Клиент для удалённого взаимодействия с сервером приложения.

```
#include <tcpclient.h>
```

Граф наследования: TcpClient:



Сигналы

- void **errorOccurred** (const QString &message)
Сообщение об ошибке для UI.
- void **dataChanged** ()
Уведомление от сервера об изменении данных (data_changed).
- void **responseReceived** (int requestId, bool success, const QVariantMap &data)
Пришёл ответ на ранее отправленный запрос.
- void **ordersChanged** ()
Сигнал об изменении списка заказов (создание, обновление статуса, закрытие).

Открытые члены

- **TcpClient** (Logs *logger, QObject *parent=nullptr)
Конструктор.
- bool **connectToServer** (const QString &host, quint16 port)
Подключается к серверу.
- void **disconnectFromServer** ()
Закрывает соединение и очищает буфер.
- bool **isConnected** () const
- Q_INVOKABLE bool **checkUser** (const QString &login, const QString &password)
Отправляет на сервер запрос аутентификации.
- Q_INVOKABLE QString **getUserName** () const
- Q_INVOKABLE QString **getUserRole** () const
- Q_INVOKABLE int **getUserId** () const
- Q_INVOKABLE QVariantList **getFoods** ()
Запрашивает меню с сервера.
- Q_INVOKABLE QVariantList **getProducts** ()
Запрашивает список продуктов.
- Q_INVOKABLE bool **updateProductQuantity** (int productId, int newQuantity)
Отправляет команду установки абсолютного количества продукта.
- Q_INVOKABLE bool **addProductQuantity** (int productId, int delta)
Отправляет команду изменения количества продукта на дельту.
- Q_INVOKABLE int **createOrder** (int tableNumber, int userId, const QString &specialRequests, double totalCost)
Создаёт заказ на сервере.
- Q_INVOKABLE bool **addOrderItem** (int orderId, int foodId, int quantity, double price)
Добавляет позицию заказа.
- Q_INVOKABLE bool **checkAndConsume** (int foodId, int quantity)
Запрашивает проверку и списание ингредиентов.
- Q_INVOKABLE bool **returnIngredients** (int foodId, int quantity)
Запрашивает возврат ингредиентов на склад.
- Q_INVOKABLE QVariantList **getLowStockWarning** (int foodId, int quantity)
Получает предупреждения о снижении остатков ниже порога.
- Q_INVOKABLE QVariantMap **getReportRevenue** (const QString &from, const QString &to, int dishId, int tableNumber, int waiterId)

- Запрашивает отчёт по общей выручке.*
- Q_INVOKABLE QVariantList [getRevenueByDay](#) (const QString &from, const QString &to, int dishId, int tableNumber, int waiterId)
- Запрашивает выручку по дням.*
- Q_INVOKABLE QVariantList [getFoodPopularity](#) (const QString &from, const QString &to)
- Запрашивает популярность блюд.*
- Q_INVOKABLE QVariantList [getReportLowStock](#) ()
- Запрашивает отчёт по продуктам с низким остатком.*
- Q_INVOKABLE QVariantList [getDishesList](#) ()
- Запрашивает краткий список блюд (ID и название).*
- Q_INVOKABLE QVariantList [getWaitersList](#) ()
- Запрашивает список официантов.*
- Q_INVOKABLE QStringList [getLocalIps](#) ()
- Возвращает IP-адреса локального устройства.*
- Q_INVOKABLE bool [pingServer](#) (const QString &host, int port)
- Проверяет доступность удалённого хоста через пробное TCP-подключение.*
- Q_INVOKABLE bool [reconnect](#) (const QString &host, quint16 port)
- Переподключается к серверу (сначала отключается, затем подключается заново).*
- Q_INVOKABLE bool [saveReportToCSV](#) (const QString &reportType, const QString &from, const QString &to, int dishId, int tableNumber, int waiterId, const QString &fileName)
- Отправляет запрос на сохранение отчёта в CSV на сервер.*
- Q_INVOKABLE QVariantList [getOrders](#) (const QString &status=QString())
- Возвращает список заказов с возможной фильтрацией по статусу.*
- Q_INVOKABLE bool [updateOrderStatus](#) (int orderId, const QString &newStatus)
- Обновляет статус заказа.*
- Q_INVOKABLE QVariantList [getOrderItems](#) (int orderId)
- Возвращает позиции заказа.*
- Q_INVOKABLE bool [closeOrder](#) (int orderId, double totalSum, const QVariantList &items)
- Закрывает заказ, создаёт чек и переводит статус в "Закрыт".*
- Q_INVOKABLE QVariantMap [getCheckByOrderId](#) (int orderId)
- Возвращает чек по ID заказа.*
- Q_INVOKABLE bool [syncOrderItems](#) (int orderId, const QVariantList &newItems)
- Синхронизирует позиции заказа при редактировании (вычисляет разницу).*
- Q_INVOKABLE QVariantList [getClosedOrders](#) ()
- Возвращает закрытые заказы (статус "Закрыт").*

5.9.1. Подробное описание

Клиент для удалённого взаимодействия с сервером приложения.

Устанавливает TCP-соединение, обменивается JSON-сообщениями. Дублирует интерфейс [DBWorker](#) для прозрачной работы из QML независимо от режима (локальный или клиент-сервер). При потере связи генерирует сигнал `errorOccurred`.

См. определение в файле [tcpclient.h](#) строка 19

5.9.2. Конструктор(ы)

5.9.2.1. TcpClient()

```
TcpClient::TcpClient (
    Logs * logger,
    QObject * parent = nullptr) [explicit]
```

Конструктор.

Аргументы

<i>logger</i>	Указатель на логгер.
<i>parent</i>	Родительский объект.

См. определение в файле [tcpclient.cpp](#) строка 10

5.9.2.2. ~TcpClient()

`TcpClient::~~TcpClient ()`

См. определение в файле [tcpclient.cpp](#) строка 16

5.9.3. Методы**5.9.3.1. addOrderItem()**

```
bool TcpClient::addOrderItem (  
    int orderId,  
    int foodId,  
    int quantity,  
    double price)
```

Добавляет позицию заказа.

Аргументы

<i>orderId</i>	ID заказа.
<i>foodId</i>	ID блюда.
<i>quantity</i>	Количество.
<i>price</i>	Цена.

Возвращает

`true` при успехе.

См. определение в файле [tcpclient.cpp](#) строка 298

5.9.3.2. addProductQuantity()

```
bool TcpClient::addProductQuantity (  
    int productId,  
    int delta)
```

Отправляет команду изменения количества продукта на дельту.

Аргументы

<i>productId</i>	ID продукта.
<i>delta</i>	Изменение (может быть отрицательным).

Возвращает

`true` при успехе.

См. определение в файле [tcpclient.cpp](#) строка 249

5.9.3.3. checkAndConsume()

```
bool TcpClient::checkAndConsume (  
    int foodId,  
    int quantity)
```

Запрашивает проверку и списание ингредиентов.

Аргументы

<i>foodId</i>	ID блюда.
<i>quantity</i>	Количество порций.

Возвращает

`true`, если списание разрешено и выполнено.

См. определение в файле [tcpclient.cpp](#) строка 321

5.9.3.4. checkUser()

```
bool TcpClient::checkUser (  
    const QString & login,  
    const QString & password)
```

Отправляет на сервер запрос аутентификации.

Аргументы

<i>login</i>	Логин.
<i>password</i>	Пароль.

Возвращает

`true` при успешной проверке.

См. определение в файле [tcpclient.cpp](#) строка 150

5.9.3.5. closeOrder()

```
bool TcpClient::closeOrder (  
    int orderId,  
    double totalSum,  
    const QVariantList & items)
```

Закрывает заказ, создаёт чек и переводит статус в "Закрыт".

Аргументы

<i>orderId</i>	ID заказа.
<i>totalSum</i>	Итоговая сумма заказа.
<i>items</i>	Список позиций заказа (каждый элемент: {name, quantity, price}).

Возвращает

`true` при успешном закрытии.

См. определение в файле [tcpclient.cpp](#) строка 649

5.9.3.6. connectToServer()

```
bool TcpClient::connectToServer (
    const QString & host,
    quint16 port)
```

Подключается к серверу.

Аргументы

<i>host</i>	Адрес сервера.
<i>port</i>	Порт.

Возвращает

`true`, если соединение установлено.

См. определение в файле [tcpclient.cpp](#) строка 22

5.9.3.7. createOrder()

```
int TcpClient::createOrder (
    int tableNumber,
    int userId,
    const QString & specialRequests,
    double totalCost)
```

Создаёт заказ на сервере.

Аргументы

<i>tableNumber</i>	Номер столика.
<i>userId</i>	ID пользователя.
<i>specialRequests</i>	Комментарий.
<i>totalCost</i>	Общая стоимость.

Возвращает

ID созданного заказа или -1.

См. определение в файле [tcpclient.cpp](#) строка 270

5.9.3.8. disconnectFromServer()

```
void TcpClient::disconnectFromServer ()
```

Закрывает соединение и очищает буфер.

См. определение в файле [tcpclient.cpp](#) строка 89

5.9.3.9. getCheckByOrderId()

```
QVariantMap TcpClient::getCheckByOrderId (
    int orderId)
```

Возвращает чек по ID заказа.

Аргументы

<i>orderId</i>	ID заказа.
----------------	------------

Возвращает

Словарь с ключами `checkId`, `checkNumber`, `orderNumber`, `totalSum`, `checkData`, `createdAt`.

См. определение в файле [tcpclient.cpp](#) строка 670

5.9.3.10. `getClosedOrders()`

`QVariantList TcpClient::getClosedOrders ()`

Возвращает закрытые заказы (статус "Закрыт").

Возвращает

Список заказов.

См. определение в файле [tcpclient.cpp](#) строка 709

5.9.3.11. `getDishesList()`

`QVariantList TcpClient::getDishesList ()`

Запрашивает краткий список блюд (ID и название).

См. определение в файле [tcpclient.cpp](#) строка 486

5.9.3.12. `getFoodPopularity()`

`QVariantList TcpClient::getFoodPopularity (`

`const QString & from,`

`const QString & to)`

Запрашивает популярность блюд.

Аргументы

<i>from</i>	Начальная дата.
<i>to</i>	Конечная дата.

Возвращает

Список словарей с ключами "foodName" и "count".

См. определение в файле [tcpclient.cpp](#) строка 441

5.9.3.13. `getFoods()`

`QVariantList TcpClient::getFoods ()`

Запрашивает меню с сервера.

Возвращает

Список блюд (`QVariantList`).

См. определение в файле [tcpclient.cpp](#) строка 182

5.9.3.14. `getLocalIps()`

`QStringList TcpClient::getLocalIps ()`

Возвращает IP-адреса локального устройства.

См. определение в файле [tcpclient.cpp](#) строка 528

5.9.3.15. getLowStockWarning()

```
QVariantList TcpClient::getLowStockWarning (
    int foodId,
    int quantity)
```

Получает предупреждения о снижении остатков ниже порога.

Аргументы

<i>foodId</i>	ID блюда.
<i>quantity</i>	Количество порций.

Возвращает

Список предупреждений.

См. определение в файле [tcpclient.cpp](#) строка 363

5.9.3.16. getOrderItems()

```
QVariantList TcpClient::getOrderItems (
    int orderId)
```

Возвращает позиции заказа.

Аргументы

<i>orderId</i>	ID заказа.
----------------	------------

Возвращает

Список словарей с полями orderId, foodId, name, quantity, price.

См. определение в файле [tcpclient.cpp](#) строка 630

5.9.3.17. getOrders()

```
QVariantList TcpClient::getOrders (
    const QString & status = QString())
```

Возвращает список заказов с возможной фильтрацией по статусу.

Аргументы

<i>status</i>	Статус заказа (пустая строка – все заказы).
---------------	---

Возвращает

Список словарей с полями orderId, orderNumber, tableNumber, orderDate, specialRequests, totalCost, status, waiterName.

См. определение в файле [tcpclient.cpp](#) строка 590

5.9.3.18. getProducts()

```
QVariantList TcpClient::getProducts ()
```

Запрашивает список продуктов.

Возвращает

Список продуктов.

См. определение в файле [tcpclient.cpp](#) строка 205

5.9.3.19. getReportLowStock()

QVariantList TcpClient::getReportLowStock ()
Запрашивает отчёт по продуктам с низким остатком.
См. определение в файле [tcpclient.cpp](#) строка 465

5.9.3.20. getReportRevenue()

QVariantMap TcpClient::getReportRevenue (
 const QString & *from*,
 const QString & *to*,
 int *dishId*,
 int *tableNumber*,
 int *waiterId*)

Запрашивает отчёт по общей выручке.

Аргументы

<i>from</i>	Начальная дата.
<i>to</i>	Конечная дата.
<i>dishId</i>	ID блюда (0 – все).
<i>tableNumber</i>	№ столика (0 – все).
<i>waiterId</i>	ID официанта (0 – все).

Возвращает

Словарь с ключом "totalRevenue" или "error".

См. определение в файле [tcpclient.cpp](#) строка 387

5.9.3.21. getRevenueByDay()

QVariantList TcpClient::getRevenueByDay (
 const QString & *from*,
 const QString & *to*,
 int *dishId*,
 int *tableNumber*,
 int *waiterId*)

Запрашивает выручку по дням.

Аргументы

<i>from</i>	Начальная дата.
<i>to</i>	Конечная дата.
<i>dishId</i>	ID блюда.
<i>tableNumber</i>	№ столика.
<i>waiterId</i>	ID официанта.

Возвращает

Список словарей с ключами "date" и "total".

См. определение в файле [tcpclient.cpp](#) строка 414

5.9.3.22. getUserId()

```
int TcpClient::getUserId () const
```

Возвращает

ID текущего пользователя.

См. определение в файле [tcpclient.cpp](#) строка 180

5.9.3.23. getUserName()

```
QString TcpClient::getUserName () const
```

Возвращает

Имя текущего пользователя.

См. определение в файле [tcpclient.cpp](#) строка 178

5.9.3.24. getUserRole()

```
QString TcpClient::getUserRole () const
```

Возвращает

Роль текущего пользователя.

См. определение в файле [tcpclient.cpp](#) строка 179

5.9.3.25. getWaitersList()

```
QVariantList TcpClient::getWaitersList ()
```

Запрашивает список официантов.

См. определение в файле [tcpclient.cpp](#) строка 507

5.9.3.26. isConnected()

```
bool TcpClient::isConnected () const
```

Возвращает

true, если сокет находится в состоянии ConnectedState.

См. определение в файле [tcpclient.cpp](#) строка 101

5.9.3.27. pingServer()

```
bool TcpClient::pingServer (  
    const QString & host,  
    int port)
```

Проверяет доступность удалённого хоста через пробное TCP-подключение.

Аргументы

<i>host</i>	Адрес.
<i>port</i>	Порт.

Возвращает

true, если соединение удалось установить за 2 с.

См. определение в файле [tcpclient.cpp](#) строка 545

5.9.3.28. reconnect()

```
bool TcpClient::reconnect (
    const QString & host,
    quint16 port)
```

Переподключается к серверу (сначала отключается, затем подключается заново).

Аргументы

<i>host</i>	Новый адрес.
<i>port</i>	Новый порт.

Возвращает

`true` при успешном подключении.

См. определение в файле [tcpclient.cpp](#) строка 555

5.9.3.29. returnIngredients()

```
bool TcpClient::returnIngredients (
    int foodId,
    int quantity)
```

Запрашивает возврат ингредиентов на склад.

Аргументы

<i>foodId</i>	ID блюда.
<i>quantity</i>	Количество порций.

Возвращает

`true` при успехе.

См. определение в файле [tcpclient.cpp](#) строка 342

5.9.3.30. saveReportToCSV()

```
bool TcpClient::saveReportToCSV (
    const QString & reportType,
    const QString & from,
    const QString & to,
    int dishId,
    int tableNumber,
    int waiterId,
    const QString & fileName)
```

Отправляет запрос на сохранение отчёта в CSV на сервер.

Аргументы

<i>reportType</i>	Тип отчёта.
<i>from</i>	Начальная дата.
<i>to</i>	Конечная дата.
<i>dishId</i>	ID блюда (0 – все).
<i>tableNumber</i>	№ столика (0 – все).

<i>waiterId</i>	ID официанта (0 – все).
<i>fileName</i>	Полный путь к сохраняемому файлу.

Возвращает

`true`, если сервер успешно создал файл.

См. определение в файле [tcpclient.cpp](#) строка 561

5.9.3.31. syncOrderItems()

```
bool TcpClient::syncOrderItems (  
    int orderId,  
    const QVariantList & newItems)
```

Синхронизирует позиции заказа при редактировании (вычисляет разницу).

Аргументы

<i>orderId</i>	ID заказа.
<i>newItems</i>	Новый список позиций (каждый элемент: {foodId, quantity, price, name}).

Возвращает

`true` при успешной синхронизации.

См. определение в файле [tcpclient.cpp](#) строка 689

5.9.3.32. updateOrderStatus()

```
bool TcpClient::updateOrderStatus (  
    int orderId,  
    const QString & newStatus)
```

Обновляет статус заказа.

Аргументы

<i>orderId</i>	ID заказа.
<i>newStatus</i>	Новый статус ("Новый", "Готовится", "Готов", "Закрыт").

Возвращает

`true` при успешном обновлении.

См. определение в файле [tcpclient.cpp](#) строка 610

5.9.3.33. updateProductQuantity()

```
bool TcpClient::updateProductQuantity (  
    int productId,  
    int newQuantity)
```

Отправляет команду установки абсолютного количества продукта.

Аргументы

<i>productId</i>	ID продукта.
------------------	--------------

<code>newQuantity</code>	Новое количество.
--------------------------	-------------------

Возвращает

`true` при успехе.

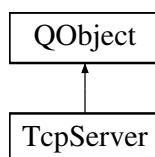
См. определение в файле [tcpclient.cpp](#) строка 228

5.10. Класс TcpServer

TCP-сервер, принимающий подключения от клиентов.

`#include <tcpserver.h>`

Граф наследования: TcpServer:



Открытые члены

- [TcpServer](#) ([Logs](#) *logger, [DBWorker](#) *dbWorker, `QObject *parent=nullptr`)
Конструктор.
- `bool start` (`quint16 port`)
Запускает прослушивание порта.
- `void stop` ()
Останавливает сервер и удаляет все соединения.

5.10.1. Подробное описание

TCP-сервер, принимающий подключения от клиентов.

При новом подключении создаёт [ClientConnection](#) и передаёт ему [DBWorker](#).

См. определение в файле [tcpserver.h](#) строка 57

5.10.2. Конструктор(ы)

5.10.2.1. TcpServer()

```

TcpServer::TcpServer (
    Logs * logger,
    DBWorker * dbWorker,
    QObject * parent = nullptr) [explicit]
  
```

Конструктор.

Аргументы

<code>logger</code>	Логгер.
<code>dbWorker</code>	Указатель на DBWorker .
<code>parent</code>	Родительский объект.

См. определение в файле [tcpserver.cpp](#) строка 299

5.10.2.2. ~TcpServer()

TcpServer::~~TcpServer ()

См. определение в файле [tcpserver.cpp](#) строка 305

5.10.3. Методы

5.10.3.1. start()

```
bool TcpServer::start (
    quint16 port)
```

Запускает прослушивание порта.

Аргументы

<i>port</i>	Порт.
-------------	-------

Возвращает

true при успехе.

См. определение в файле [tcpserver.cpp](#) строка 311

5.10.3.2. stop()

```
void TcpServer::stop ()
```

Останавливает сервер и удаляет все соединения.

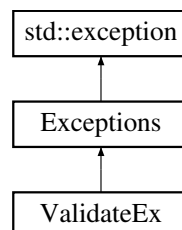
См. определение в файле [tcpserver.cpp](#) строка 329

5.11. Класс ValidateEx

Исключение валидации (некорректные данные, нехватка продуктов и т.п.).

```
#include <exceptions.h>
```

Граф наследования: ValidateEx:



Открытые члены

- [ValidateEx](#) (const QString &message)

Открытые члены унаследованные от [Exceptions](#)

- [Exceptions](#) (const QString &message)
Конструктор.
- virtual const char * [what](#) () const noexcept override

5.11.1. Подробное описание

Исключение валидации (некорректные данные, нехватка продуктов и т.п.).

См. определение в файле [exceptions.h](#) строка 28

5.11.2. Конструктор(ы)

5.11.2.1. ValidateEx()

```
ValidateEx::ValidateEx (  
    const QString & message) [inline], [explicit]
```

См. определение в файле [exceptions.h](#) строка 31

Глава 6

Файлы

6.1. dbworker.h

```
00001 #ifndef DBWORKER_H
00002 #define DBWORKER_H
00003
00004 #include <QObject>
00005 #include <QSqlDatabase>
00006 #include <QSqlQuery>
00007 #include <QSqlError>
00008 #include <QVariant>
00009 #include <QVariantList>
00010 #include <QVariantMap>
00011 #include <QStringConverter>
00012 #include <QJsonDocument>
00013 #include <QJsonArray>
00014 #include <QJsonObject>
00015 #include "logs.h"
00016 #include "exceptions.h"
00017
00024 class DBWorker : public QObject
00025 {
00026     Q_OBJECT
00027 public:
00033     DBWorker(Logs *logger, QObject *parent = nullptr);
00034     ~DBWorker();
00035
00042     Q_INVOKABLE bool checkUser(const QString &login, const QString &password);
00043
00045     Q_INVOKABLE QString getUsername() const { return userName; }
00047     Q_INVOKABLE QString getUserRole() const { return userRole; }
00049     Q_INVOKABLE int getUserId() const { return userId; }
00050
00057     Q_INVOKABLE bool checkAndConsume(int foodId, int quantity);
00058
00065     Q_INVOKABLE bool returnIngredients(int foodId, int quantity);
00066
00068     Q_INVOKABLE bool refreshAllFoodsAvailability();
00069
00079     Q_INVOKABLE bool reconnect(const QString &host, int port, const QString &dbName, const QString
&user, const QString &password);
00080
00082     Q_INVOKABLE QStringList getLocalIps();
00083
00090     Q_INVOKABLE bool pingServer(const QString &host, int port);
00091
00093     Q_INVOKABLE QVariantList getFoods();
00094
00096     Q_INVOKABLE QVariantList getProducts();
00097
00099     Q_INVOKABLE QVariantList getDishesList();
00100
00102     Q_INVOKABLE QVariantList getWaitersList();
00103
00110     Q_INVOKABLE bool updateProductQuantity(int productId, int newQuantity);
00111
00118     Q_INVOKABLE bool addProductQuantity(int productId, int delta);
00119
00128     Q_INVOKABLE int createOrder(int tableNumber, int userId, const QString &specialRequests, double
totalCost);
00129
00138     Q_INVOKABLE bool addOrderItem(int orderId, int foodId, int quantity, double price);
00139
00149     Q_INVOKABLE QVariantMap getReportRevenue(const QString &from, const QString &to,
```

```

00150                                     int dishId, int tableNumber, int waiterId);
00151
00161     Q_INVOKABLE QVariantList getRevenueByDay(const QString &from, const QString &to,
00162                                             int dishId, int tableNumber, int waiterId);
00163
00170     Q_INVOKABLE QVariantList getFoodPopularity(const QString &from, const QString &to);
00171
00173     Q_INVOKABLE QVariantList getReportLowStock();
00174
00181     Q_INVOKABLE QVariantList getLowStockWarning(int foodId, int quantity);
00182
00194     Q_INVOKABLE bool saveReportToCSV(const QString &reportType,
00195                                     const QString &from,
00196                                     const QString &to,
00197                                     int dishId,
00198                                     int tableNumber,
00199                                     int waiterId,
00200                                     const QString &fileName);
00201
00208     Q_INVOKABLE QVariantList getOrders(const QString &status = QString());
00209
00216     Q_INVOKABLE bool updateOrderStatus(int orderId, const QString &newStatus);
00217
00223     Q_INVOKABLE QVariantList getOrderItems(int orderId);
00224
00232     Q_INVOKABLE bool closeOrder(int orderId, double totalSum, const QVariantList &items);
00233
00239     Q_INVOKABLE QVariantMap getCheckByOrderId(int orderId);
00240
00245     Q_INVOKABLE QVariantList getAllOrdersForReport();
00246
00253     Q_INVOKABLE bool syncOrderItems(int orderId, const QVariantList &newItems);
00254
00259     Q_INVOKABLE QVariantList getClosedOrders();
00260
00269     Q_INVOKABLE void updateConnectionParams(const QString &host, int port, const QString &dbName,
const QString &user, const QString &password);
00270
00277     Q_INVOKABLE QVariantList getChecksByDateRange(const QString &from, const QString &to);
00278
00280     int generateOrderNumber();
00281
00282 signals:
00284     void errorOccurred(const QString &message);
00286     void foodAvailabilityChanged();
00287     /*! Сигнал об изменении списка заказов (создание, обновление статуса, закрытие).
00288     void ordersChanged();
00289
00290 private:
00291     QSqlDatabase db;
00292     QString userName;
00293     QString userRole;
00294     int userId;
00295     Logs *logger;
00296 };
00297
00298 #endif // DBWORKER_H

```

6.2. exceptions.h

```

00001 #ifndef EXCEPTIONS_H
00002 #define EXCEPTIONS_H
00003
00004 #include <QString>
00005 #include <stdexcept>
00006
00013 class Exceptions : public std::exception
00014 {
00015 public:
00020     explicit Exceptions(const QString &message);
00021     virtual const char* what() const noexcept override;
00022
00023 private:
00024     std::string msg;
00025 };
00026
00028 class ValidateEx : public Exceptions
00029 {
00030 public:
00031     explicit ValidateEx(const QString &message) : Exceptions(message) {}
00032 };
00033
00035 class DatabaseEx : public Exceptions
00036 {

```

```

00037 public:
00038     explicit DatabaseEx(const QString &message) : Exceptions(message) {}
00039 };
00040
00042 class NetworkEx : public Exceptions
00043 {
00044 public:
00045     explicit NetworkEx(const QString &message) : Exceptions(message) {}
00046 };
00047
00049 class FileEx : public Exceptions
00050 {
00051 public:
00052     explicit FileEx(const QString &message) : Exceptions(message) {}
00053 };
00054
00055 #endif // EXCEPTIONS_H

```

6.3. logs.h

```

00001 #ifndef LOGS_H
00002 #define LOGS_H
00003
00004 #include <QObject>
00005 #include <QFile>
00006 #include <QTextStream>
00007 #include "exceptions.h"
00008
00016 class Logs : public QObject
00017 {
00018     Q_OBJECT
00019 public:
00020     explicit Logs(QObject *parent = nullptr);
00021     ~Logs();
00022
00024     Q_INVOKABLE void info(const QString &msg);
00026     Q_INVOKABLE void warning(const QString &msg);
00028     Q_INVOKABLE void error(const QString &msg);
00033     Q_INVOKABLE void setLogPath(const QString &path);
00034
00035 private:
00036     void writeLog(const QString &level, const QString &message);
00037     void openLogFile();
00038
00039     QFile logFile;
00040     QString m_logPath;
00041     QTextStream out;
00042 };
00043
00044 #endif // LOGS_H

```

6.4. settings.h

```

00001 #ifndef SETTINGS_H
00002 #define SETTINGS_H
00003
00004 #include <QObject>
00005 #include <QString>
00006 #include <QDir>
00007 #include <QStandardPaths>
00008 #include <QCoreApplication>
00009 #include "logs.h"
00010 #include "exceptions.h"
00011
00018 class Settings : public QObject
00019 {
00020     Q_OBJECT
00021     Q_PROPERTY(QString dbHost READ dbHost WRITE setDbHost NOTIFY dbHostChanged)
00022     Q_PROPERTY(int dbPort READ dbPort WRITE setDbPort NOTIFY dbPortChanged)
00023     Q_PROPERTY(QString dbName READ dbName WRITE setDbName NOTIFY dbNameChanged)
00024     Q_PROPERTY(QString dbUser READ dbUser WRITE setDbUser NOTIFY dbUserChanged)
00025     Q_PROPERTY(QString dbPassword READ dbPassword WRITE setDbPassword NOTIFY dbPasswordChanged)
00026     Q_PROPERTY(bool isServerMode READ isServerMode WRITE setIsServerMode NOTIFY isServerModeChanged)
00027     Q_PROPERTY(QString logPath READ logPath WRITE setLogPath NOTIFY logPathChanged)
00028     Q_PROPERTY(QString serverHost READ serverHost WRITE setServerHost NOTIFY serverHostChanged)
00029     Q_PROPERTY(int serverPort READ serverPort WRITE setServerPort NOTIFY serverPortChanged)
00030     Q_PROPERTY(QString reportsPath READ reportsPath WRITE setReportsPath NOTIFY reportsPathChanged)
00031
00032 public:
00033     explicit Settings(Logs *logger, QObject *parent = nullptr);
00039

```

```

00040 // Геттеры и сеттеры с краткими пояснениями
00041 QString dbHost() const { return m_dbHost; }
00042 void setDbHost(const QString &host);
00043
00044 int dbPort() const { return m_dbPort; }
00045 void setDbPort(int port);
00046
00047 QString dbName() const { return m_dbName; }
00048 void setDbName(const QString &name);
00049
00050 QString dbUser() const { return m_dbUser; }
00051 void setDbUser(const QString &user);
00052
00053 QString dbPassword() const { return m_dbPassword; }
00054 void setDbPassword(const QString &password);
00055
00056 bool isServerMode() const { return m_isServerMode; }
00057 void setIsServerMode(bool mode);
00058
00059 QString logPath() const { return m_logPath; }
00060 void setLogPath(const QString &path);
00061
00062 QString serverHost() const { return m_serverHost; }
00063 void setServerHost(const QString &host);
00064
00065 int serverPort() const { return m_serverPort; }
00066 void setServerPort(int port);
00067
00068 QString reportsPath() const { return m_reportsPath; }
00069 void setReportsPath(const QString &path);
00070
00072 Q_INVOKABLE void load();
00074 Q_INVOKABLE void save();
00075
00076 signals:
00077     void dbHostChanged();
00078     void dbPortChanged();
00079     void dbNameChanged();
00080     void dbUserChanged();
00081     void dbPasswordChanged();
00082     void isServerModeChanged();
00083     void logPathChanged();
00084     void serverHostChanged();
00085     void serverPortChanged();
00086     void reportsPathChanged();
00087
00088 private:
00089     QString m_dbHost;
00090     int m_dbPort;
00091     QString m_dbName;
00092     QString m_dbUser;
00093     QString m_dbPassword;
00094     bool m_isServerMode;
00095     QString m_logPath;
00096     Logs *logger;
00097     QString m_serverHost;
00098     int m_serverPort;
00099     QString m_reportsPath;
00100     QString getSettingsFilePath() const;
00101 };
00102
00103 #endif // SETTINGS_H

```

6.5. tcpclient.h

```

00001 #ifndef TCPCLIENT_H
00002 #define TCPCLIENT_H
00003
00004 #include <QObject>
00005 #include <QTcpSocket>
00006 #include <QVariantList>
00007 #include <QVariantMap>
00008 #include "logs.h"
00009 #include "exceptions.h"
00010
00019 class TcpClient : public QObject
00020 {
00021     Q_OBJECT
00022 public:
00028     explicit TcpClient(Logs *logger, QObject *parent = nullptr);
00029     ~TcpClient();
00030
00037     bool connectToServer(const QString &host, quint16 port);
00039     void disconnectFromServer();

```

```

00041     bool isConnected() const;
00042
00049     Q_INVOKABLE bool checkUser(const QString &login, const QString &password);
00050
00052     Q_INVOKABLE QString getUsername() const;
00054     Q_INVOKABLE QString getUserRole() const;
00056     Q_INVOKABLE int getUserId() const;
00057
00062     Q_INVOKABLE QVariantList getFoods();
00063
00068     Q_INVOKABLE QVariantList getProducts();
00069
00076     Q_INVOKABLE bool updateProductQuantity(int productId, int newQuantity);
00077
00084     Q_INVOKABLE bool addProductQuantity(int productId, int delta);
00085
00094     Q_INVOKABLE int createOrder(int tableNumber, int userId, const QString &specialRequests, double
totalCost);
00095
00104     Q_INVOKABLE bool addOrderItem(int orderId, int foodId, int quantity, double price);
00105
00112     Q_INVOKABLE bool checkAndConsume(int foodId, int quantity);
00113
00120     Q_INVOKABLE bool returnIngredients(int foodId, int quantity);
00121
00128     Q_INVOKABLE QVariantList getLowStockWarning(int foodId, int quantity);
00129
00139     Q_INVOKABLE QVariantMap getReportRevenue(const QString &from, const QString &to, int dishId, int
tableNumber, int waiterId);
00140
00150     Q_INVOKABLE QVariantList getRevenueByDay(const QString &from, const QString &to, int dishId, int
tableNumber, int waiterId);
00151
00158     Q_INVOKABLE QVariantList getFoodPopularity(const QString &from, const QString &to);
00159
00161     Q_INVOKABLE QVariantList getReportLowStock();
00162
00164     Q_INVOKABLE QVariantList getDishesList();
00165
00167     Q_INVOKABLE QVariantList getWaitersList();
00168
00170     Q_INVOKABLE QStringList getLocalIps();
00171
00178     Q_INVOKABLE bool pingServer(const QString &host, int port);
00179
00186     Q_INVOKABLE bool reconnect(const QString &host, quint16 port);
00187
00199     Q_INVOKABLE bool saveReportToCSV(const QString &reportType,
const QString &from,
const QString &to,
int dishId,
int tableNumber,
int waiterId,
const QString &fileName);
00200
00201
00202
00203
00204
00205
00206
00213     Q_INVOKABLE QVariantList getOrders(const QString &status = QString());
00214
00221     Q_INVOKABLE bool updateOrderStatus(int orderId, const QString &newStatus);
00222
00228     Q_INVOKABLE QVariantList getOrderItems(int orderId);
00229
00237     Q_INVOKABLE bool closeOrder(int orderId, double totalSum, const QVariantList &items);
00238
00244     Q_INVOKABLE QVariantMap getCheckByOrderId(int orderId);
00245
00252     Q_INVOKABLE bool syncOrderItems(int orderId, const QVariantList &newItems);
00253
00258     Q_INVOKABLE QVariantList getClosedOrders();
00259
00260 signals:
00262     void errorOccurred(const QString &message);
00264     void dataChanged();
00266     void responseReceived(int requestId, bool success, const QVariantMap &data);
00268     void ordersChanged();
00269
00270 private:
00277     int sendRequest(const QString &command, const QVariantMap &data = QVariantMap());
00278
00285     bool waitForResponse(int requestId, QVariantMap &resultData);
00286
00287     QTcpSocket *socket;
00288     QByteArray buffer;
00289     int nextRequestId;
00290     QMap<int, QVariantMap> pendingResponses;
00291
00292     QString userName;
00293     QString userRole;

```

```

00294     int userId;
00295     Logs *logger;
00296 };
00297
00298 #endif // TCPCLIENT_H

```

6.6. tcpserver.h

```

00001 #ifndef TCPSERVER_H
00002 #define TCPSERVER_H
00003
00004 #include <QObject>
00005 #include <QTcpServer>
00006 #include <QTcpSocket>
00007 #include <QMap>
00008 #include "logs.h"
00009 #include "exceptions.h"
00010
00011 class DBWorker;
00012
00019 class ClientConnection : public QObject
00020 {
00021     Q_OBJECT
00022 public:
00030     ClientConnection(Logs *logger, qintptr socketDescriptor, DBWorker *dbWorker, QObject *parent =
        nullptr);
00031     ~ClientConnection();
00032
00033 private slots:
00034     void onReadyRead();
00035     void onDisconnected();
00036
00037 private:
00044     void sendResponse(int requestId, bool success, const QVariantMap &data = QVariantMap());
00045
00046     QTcpSocket *socket;
00047     DBWorker *dbWorker;
00048     QByteArray buffer;
00049     Logs *logger;
00050 };
00051
00057 class TcpServer : public QObject
00058 {
00059     Q_OBJECT
00060 public:
00067     explicit TcpServer(Logs *logger, DBWorker *dbWorker, QObject *parent = nullptr);
00068     ~TcpServer();
00069
00075     bool start(quint16 port);
00077     void stop();
00078
00079 private slots:
00080     void onNewConnection();
00081
00082 private:
00083     QTcpServer *server;
00084     DBWorker *dbWorker;
00085     QList<ClientConnection*> connections;
00086     Logs *logger;
00087 };
00088
00089 #endif // TCPSERVER_H

```

6.7. dbworker.cpp

```

00001 #include "dbworker.h"
00002 #include <QCryptographicHash>
00003 #include <QRandomGenerator>
00004 #include <QSqlRecord>
00005 #include <qdatetime.h>
00006 #include <QNetworkInterface>
00007 #include <QAbstractSocket>
00008
00009 DBWorker::DBWorker(Logs *logger, QObject *parent) : QObject(parent), logger(logger)
00010 {
00011     db = QSqlDatabase::addDatabase("QPSQL");
00012     db.setHostName("127.0.0.1");
00013     db.setPort(5432);
00014     db.setDatabaseName("RestN");
00015     db.setUserName("R");
00016     db.setPassword("");

```



```

00017     db.setConnectOptions("client_encoding=UTF8");
00018
00019     if (!db.open()) {
00020         QString err = db.lastError().text();
00021         logger->warning("Ошибка подключения к БД: " + err);
00022         emit errorOccurred("Не удалось подключиться к базе данных: " + err);
00023     } else {
00024         logger->info("База данных подключена успешно!");
00025     }
00026
00027     userId = -1;
00028     userName = "Гость";
00029     userRole = "Гость";
00030 }
00031
00032 DBWorker::~DBWorker()
00033 {
00034     if (db.isOpen())
00035         db.close();
00036 }
00037
00038 bool DBWorker::checkUser(const QString &login, const QString &password)
00039 {
00040     logger->info("Проверка пользователя: "+login);
00041     try {
00042         if (!db.isOpen())
00043             throw DatabaseEx("База данных не открыта");
00044
00045         QSqlQuery query;
00046         query.prepare("SELECT UserId, FullName, Role, Salt, PasswordHash FROM Users WHERE Login =
:login");
00047         query.bindValue(":login", login);
00048
00049         if (!query.exec()) {
00050             throw DatabaseEx("Ошибка запроса checkUser: " + query.lastError().text());
00051         }
00052
00053         if (query.next()) {
00054             QString salt = query.value("Salt").toString();
00055             QByteArray storedHash = query.value("PasswordHash").toByteArray();
00056
00057             QString salted = password + salt;
00058             QByteArray computedHash = QCryptographicHash::hash(salted.toUtf8(),
QCryptographicHash::Sha3_256);
00059
00060             if (computedHash == storedHash) {
00061                 userId = query.value("UserId").toInt();
00062                 userName = query.value("FullName").toString();
00063                 userRole = query.value("Role").toString();
00064                 logger->info("Пользователь найден: " + userName);
00065                 return true;
00066             }
00067         }
00068         logger->warning("Ошибка авторизации");
00069         return false;
00070     } catch (const DatabaseEx &e) {
00071         logger->error(QString("checkUser: ") + e.what());
00072         emit errorOccurred(e.what());
00073         return false;
00074     } catch (const std::exception &e) {
00075         logger->error(QString("checkUser: неизвестная ошибка - ") + e.what());
00076         emit errorOccurred("Внутренняя ошибка при авторизации");
00077         return false;
00078     }
00079 }
00080
00081 QVariantList DBWorker::getFoods()
00082 {
00083     logger->info("Получение списка блюд");
00084     try {
00085         if (!db.isOpen())
00086             throw DatabaseEx("База данных не открыта");
00087
00088         QVariantList result;
00089         QSqlQuery query;
00090         query.prepare(
00091             "SELECT f.FoodId, f.FullName, f.Description, f.Price, f.Category, f.Available, "
00092             "CASE "
00093             "    WHEN f.Available = false THEN false "
00094             "    ELSE (SELECT bool_and(p.Quantity >= fc.Quantity) "
00095             "        FROM FoodComposition fc "
00096             "        JOIN Product p ON fc.ProductId = p.ProductId "
00097             "        WHERE fc.FoodId = f.FoodId) "
00098             "END AS available_by_stock "
00099             "FROM Food f "
00100             "ORDER BY f.FullName"
00101         );

```

```

00102         if (!query.exec())
00103             throw DatabaseEx("Ошибка getFoods: " + query.lastError().text());
00104
00105         while (query.next()) {
00106             QVariantMap item;
00107             item["foodId"] = query.value("FoodId").toInt();
00108             item["name"] = query.value("FullName").toString();
00109             item["description"] = query.value("Description").toString();
00110             item["price"] = query.value("Price").toDouble();
00111             item["category"] = query.value("Category").toString();
00112             item["available"] = query.value("available_by_stock").toBool();
00113
00114             QSqlQuery compQuery;
00115             compQuery.prepare("SELECT p.FullName, fc.Quantity FROM FoodComposition fc "
00116                               "JOIN Product p ON fc.ProductId = p.ProductId "
00117                               "WHERE fc.FoodId = :foodId");
00118             compQuery.bindValue(":foodId", query.value("FoodId").toInt());
00119             QStringList composition;
00120             if (compQuery.exec()) {
00121                 while (compQuery.next()) {
00122                     composition << QString("%1 (%2)").arg(compQuery.value(0).toString())
00123                                     .arg(compQuery.value(1).toString());
00124                 }
00125             }
00126             item["composition"] = composition.join(", ");
00127             result.append(item);
00128         }
00129         return result;
00130     } catch (const DatabaseEx &e) {
00131         logger->error(QString("getFoods: ") + e.what());
00132         emit errorOccurred(e.what());
00133         return QVariantList();
00134     } catch (const std::exception &e) {
00135         logger->error(QString("getFoods: неизвестная ошибка - ") + e.what());
00136         emit errorOccurred("Внутренняя ошибка при загрузке меню");
00137         return QVariantList();
00138     }
00139 }
00140
00141 QVariantList DBWorker::getProducts()
00142 {
00143     logger->info("Получение списка продуктов");
00144     try {
00145         if (!db.isOpen())
00146             throw DatabaseEx("База данных не открыта");
00147
00148         QVariantList result;
00149         QSqlQuery query;
00150         query.prepare("SELECT ProductId, FullName, Unit, Quantity, MinThreshold FROM Product ORDER BY
00151 FullName");
00152         if (!query.exec())
00153             throw DatabaseEx("Ошибка получения продуктов: " + query.lastError().text());
00154
00155         while (query.next()) {
00156             QVariantMap item;
00157             item["productId"] = query.value("ProductId").toInt();
00158             item["name"] = query.value("FullName").toString();
00159             item["unit"] = query.value("Unit").toString();
00160             item["quantity"] = query.value("Quantity").toInt();
00161             item["minThreshold"] = query.value("MinThreshold").toInt();
00162             result.append(item);
00163         }
00164         return result;
00165     } catch (const DatabaseEx &e) {
00166         logger->error(QString("getProducts: ") + e.what());
00167         emit errorOccurred(e.what());
00168         return QVariantList();
00169     } catch (const std::exception &e) {
00170         logger->error(QString("getProducts: неизвестная ошибка - ") + e.what());
00171         emit errorOccurred("Внутренняя ошибка при загрузке склада");
00172         return QVariantList();
00173     }
00174 }
00175 bool DBWorker::updateProductQuantity(int productId, int newQuantity)
00176 {
00177     try {
00178         if (!db.isOpen())
00179             throw DatabaseEx("База данных не открыта");
00180
00181         QSqlQuery query;
00182         query.prepare("UPDATE Product SET Quantity = :q WHERE ProductId = :id");
00183         query.bindValue(":id", productId);
00184         query.bindValue(":q", newQuantity);
00185         if (!query.exec())
00186             throw DatabaseEx("Ошибка обновления количества продуктов: " + query.lastError().text());
00187         return true;

```

```

00188     } catch (const DatabaseEx &e) {
00189         logger->error(QString("updateProductQuantity: ") + e.what());
00190         emit errorOccurred(e.what());
00191         return false;
00192     } catch (const std::exception &e) {
00193         logger->error(QString("updateProductQuantity: неизвестная ошибка - ") + e.what());
00194         emit errorOccurred("Внутренняя ошибка при обновлении остатка");
00195         return false;
00196     }
00197 }
00198
00199 bool DBWorker::addProductQuantity(int productId, int delta)
00200 {
00201     try {
00202         if (!db.isOpen())
00203             throw DatabaseEx("База данных не открыта");
00204
00205         QSqlQuery query;
00206         query.prepare("UPDATE Product SET Quantity = Quantity + :delta WHERE ProductId = :id");
00207         query.bindValue(":id", productId);
00208         query.bindValue(":delta", delta);
00209         if (!query.exec())
00210             throw DatabaseEx("Ошибка addProductQuantity: " + query.lastError().text());
00211         return true;
00212     } catch (const DatabaseEx &e) {
00213         logger->error(QString("addProductQuantity: ") + e.what());
00214         emit errorOccurred(e.what());
00215         return false;
00216     } catch (const std::exception &e) {
00217         logger->error(QString("addProductQuantity: неизвестная ошибка - ") + e.what());
00218         emit errorOccurred("Внутренняя ошибка при изменении остатка");
00219         return false;
00220     }
00221 }
00222
00223 int DBWorker::createOrder(int tableNumber, int userId, const QString &specialRequests, double
totalCost)
00224 {
00225     try {
00226         if (!db.isOpen()) throw DatabaseEx("База данных не открыта");
00227         int orderNumber = generateOrderNumber();
00228         if (orderNumber == -1) throw DatabaseEx("Не удалось сгенерировать номер заказа");
00229
00230         QSqlQuery query;
00231         query.prepare("INSERT INTO Orders (UserId, TableNumber, SpecialRequests, TotalCost,
order_number, status) "
00232             "VALUES (:uid, :table, :req, :total, :onum, 'Готовится') RETURNING OrderId");
00233         query.bindValue(":uid", userId);
00234         query.bindValue(":table", tableNumber);
00235         query.bindValue(":req", specialRequests);
00236         query.bindValue(":total", totalCost);
00237         query.bindValue(":onum", orderNumber);
00238         if (!query.exec())
00239             throw DatabaseEx("Ошибка createOrder: " + query.lastError().text());
00240
00241         if (query.next())
00242             return query.value(0).toInt();
00243         else
00244             throw DatabaseEx("Не удалось получить OrderId после вставки");
00245     } catch (const DatabaseEx &e) {
00246         logger->error(QString("createOrder: ") + e.what());
00247         emit errorOccurred(e.what());
00248         return -1;
00249     } catch (const std::exception &e) {
00250         logger->error(QString("createOrder: неизвестная ошибка - ") + e.what());
00251         emit errorOccurred("Внутренняя ошибка при создании заказа");
00252         return -1;
00253     }
00254 }
00255
00256 bool DBWorker::addOrderItem(int OrderId, int foodId, int quantity, double price)
00257 {
00258     try {
00259         if (!db.isOpen())
00260             throw DatabaseEx("База данных не открыта");
00261
00262         QSqlQuery query;
00263         query.prepare("INSERT INTO OrderFood (OrderId, FoodId, Quantity, Price) "
00264             "VALUES (:oid, :fid, :qty, :price)");
00265         query.bindValue(":oid", OrderId);
00266         query.bindValue(":fid", foodId);
00267         query.bindValue(":qty", quantity);
00268         query.bindValue(":price", price);
00269         if (!query.exec())
00270             throw DatabaseEx("Ошибка addOrderItem: " + query.lastError().text());
00271         return true;
00272     } catch (const DatabaseEx &e) {

```

```

00273         logger->error(QString("addOrderItem: ") + e.what());
00274         emit errorOccurred(e.what());
00275         return false;
00276     } catch (const std::exception &e) {
00277         logger->error(QString("addOrderItem: неизвестная ошибка - ") + e.what());
00278         emit errorOccurred("Внутренняя ошибка при добавлении позиции заказа");
00279         return false;
00280     }
00281 }
00282
00283 QVariantList DBWorker::getReportLowStock()
00284 {
00285     try {
00286         if (!db.isOpen())
00287             throw DatabaseEx("База данных не открыта");
00288
00289         QVariantList result;
00290         QSqlQuery query;
00291         query.prepare("SELECT FullName, Quantity, MinThreshold FROM Product WHERE Quantity <
MinThreshold");
00292         if (!query.exec())
00293             throw DatabaseEx("Ошибка getReportLowStock: " + query.lastError().text());
00294
00295         while (query.next()) {
00296             QVariantMap item;
00297             item["name"] = query.value(0).toString();
00298             item["quantity"] = query.value(1).toInt();
00299             item["minThreshold"] = query.value(2).toInt();
00300             result.append(item);
00301         }
00302         return result;
00303     } catch (const DatabaseEx &e) {
00304         logger->error(QString("getReportLowStock: ") + e.what());
00305         emit errorOccurred(e.what());
00306         return QVariantList();
00307     } catch (const std::exception &e) {
00308         logger->error(QString("getReportLowStock: неизвестная ошибка - ") + e.what());
00309         emit errorOccurred("Внутренняя ошибка при формировании отчёта");
00310         return QVariantList();
00311     }
00312 }
00313
00314 QVariantList DBWorker::getFoodPopularity(const QString &from, const QString &to)
00315 {
00316     try {
00317         if (!db.isOpen())
00318             throw DatabaseEx("База данных не открыта");
00319
00320         QVariantList result;
00321         QDate fromDate = QDate::fromString(from, "yyyy-MM-dd");
00322         QDate toDate = QDate::fromString(to, "yyyy-MM-dd");
00323         if (!fromDate.isValid() || !toDate.isValid())
00324             throw ValidateEx("Неверный формат даты. Используйте ГГГГ-ММ-ДД");
00325
00326         if (db.isOpen()) db.rollback();
00327
00328         QString sql = QString(
00329             "SELECT f.FullName, COALESCE(SUM(of.Quantity), 0) as total "
00330             "FROM OrderFood of "
00331             "JOIN Orders o ON of.OrderId = o.OrderId "
00332             "JOIN Food f ON of.FoodId = f.FoodId "
00333             "WHERE o.OrderDate >= '%1' AND o.OrderDate <= '%2' "
00334             "GROUP BY f.FullName "
00335             "ORDER BY total DESC"
00336             ).arg(fromDate.toString("yyyy-MM-dd")).arg(toDate.toString("yyyy-MM-dd"));
00337
00338         QSqlQuery query;
00339         if (!query.exec(sql))
00340             throw DatabaseEx("Ошибка getFoodPopularity: " + query.lastError().text());
00341
00342         while (query.next()) {
00343             QVariantMap item;
00344             item["foodName"] = query.value(0).toString();
00345             item["count"] = query.value(1).toInt();
00346             result.append(item);
00347         }
00348         return result;
00349     } catch (const DatabaseEx &e) {
00350         logger->error(QString("getFoodPopularity: ") + e.what());
00351         emit errorOccurred(e.what());
00352         return QVariantList();
00353     } catch (const ValidateEx &e) {
00354         logger->warning(QString("getFoodPopularity: ") + e.what());
00355         emit errorOccurred(e.what());
00356         return QVariantList();
00357     } catch (const std::exception &e) {
00358         logger->error(QString("getFoodPopularity: неизвестная ошибка - ") + e.what());

```

```

00359         emit errorOccurred("Внутренняя ошибка при формировании отчёта популярности");
00360         return QVariantList();
00361     }
00362 }
00363
00364 bool DBWorker::checkAndConsume(int foodId, int quantity)
00365 {
00366     try {
00367         if (!db.isOpen())
00368             throw DatabaseEx("База данных не открыта");
00369
00370         QSqlQuery compQuery;
00371         compQuery.prepare("SELECT ProductId, Quantity FROM FoodComposition WHERE FoodId = :foodId");
00372         compQuery.bindValue(":foodId", foodId);
00373         if (!compQuery.exec())
00374             throw DatabaseEx("Ошибка получения состава блюда: " + compQuery.lastError().text());
00375
00376         QList<QPair<int, double>> composition;
00377         while (compQuery.next()) {
00378             int productId = compQuery.value(0).toInt();
00379             double need = compQuery.value(1).toDouble() * quantity;
00380             composition.append(QMakePair(productId, need));
00381         }
00382         if (composition.isEmpty())
00383             throw ValidateEx("Блюдо не имеет состава");
00384
00385         for (const auto &pair : composition) {
00386             int productId = pair.first;
00387             double need = pair.second;
00388
00389             QSqlQuery qtyQuery;
00390             qtyQuery.prepare("SELECT Quantity FROM Product WHERE ProductId = :pid");
00391             qtyQuery.bindValue(":pid", productId);
00392             if (!qtyQuery.exec() || !qtyQuery.next())
00393                 throw DatabaseEx(QString("Ошибка получения остатка продукта %1").arg(productId));
00394
00395             double current = qtyQuery.value(0).toDouble();
00396             if (current < need - 0.0001) {
00397                 throw ValidateEx(QString("Не хватает продукта id=%1, нужно %2, есть %3")
00398                     .arg(productId).arg(need).arg(current));
00399             }
00400         }
00401
00402         if (!db.transaction())
00403             throw DatabaseEx("Не удалось начать транзакцию");
00404
00405         for (const auto &pair : composition) {
00406             int productId = pair.first;
00407             double need = pair.second;
00408             QSqlQuery updateQuery;
00409             updateQuery.prepare("UPDATE Product SET Quantity = Quantity - :need WHERE ProductId = :pid
AND Quantity >= :need");
00410             updateQuery.bindValue(":need", need);
00411             updateQuery.bindValue(":pid", productId);
00412             if (!updateQuery.exec()) {
00413                 db.rollback();
00414                 throw DatabaseEx(QString("Ошибка UPDATE при списании:
%1").arg(updateQuery.lastError().text()));
00415             }
00416             if (updateQuery.numRowsAffected() == 0) {
00417                 db.rollback();
00418                 throw ValidateEx(QString("Недостаточно товара для списания (параллельное изменение?)
productId=%1").arg(productId));
00419             }
00420         }
00421         if (!db.commit()) {
00422             db.rollback();
00423             throw DatabaseEx("Ошибка коммита транзакции");
00424         }
00425         return true;
00426     } catch (const DatabaseEx &e) {
00427         logger->error(QString("checkAndConsume: ") + e.what());
00428         emit errorOccurred(e.what());
00429         return false;
00430     } catch (const ValidateEx &e) {
00431         logger->warning(QString("checkAndConsume: ") + e.what());
00432         emit errorOccurred(e.what());
00433         return false;
00434     } catch (const std::exception &e) {
00435         logger->error(QString("checkAndConsume: неизвестная ошибка - ") + e.what());
00436         emit errorOccurred("Внутренняя ошибка при добавлении блюда");
00437         return false;
00438     }
00439 }
00440
00441 bool DBWorker::returnIngredients(int foodId, int quantity)
00442 {

```

```

00443     try {
00444         if (!db.isOpen())
00445             throw DatabaseEx("База данных не открыта");
00446
00447         QSqlQuery compQuery;
00448         compQuery.prepare("SELECT ProductId, Quantity FROM FoodComposition WHERE FoodId = :foodId");
00449         compQuery.bindValue(":foodId", foodId);
00450         if (!compQuery.exec())
00451             throw DatabaseEx("Ошибка получения состава для возврата: " +
compQuery.lastError().text());
00452
00453         QList<QPair<int, double> composition;
00454         while (compQuery.next()) {
00455             int productId = compQuery.value(0).toInt();
00456             double amount = compQuery.value(1).toDouble() * quantity;
00457             composition.append(QMakePair(productId, amount));
00458         }
00459         if (composition.isEmpty())
00460             throw ValidateEx("Блюдо не имеет состава, возврат невозможен");
00461
00462         if (!db.transaction())
00463             throw DatabaseEx("Не удалось начать транзакцию для возврата");
00464
00465         for (const auto &pair : composition) {
00466             int productId = pair.first;
00467             double amount = pair.second;
00468             QSqlQuery updateQuery;
00469             updateQuery.prepare("UPDATE Product SET Quantity = Quantity + :amount WHERE ProductId =
:pid");
00470             updateQuery.bindValue(":amount", amount);
00471             updateQuery.bindValue(":pid", productId);
00472             if (!updateQuery.exec()) {
00473                 db.rollback();
00474                 throw DatabaseEx(QString("Ошибка UPDATE при возврате:
%1").arg(updateQuery.lastError().text()));
00475             }
00476         }
00477         if (!db.commit()) {
00478             db.rollback();
00479             throw DatabaseEx("Ошибка коммита при возврате");
00480         }
00481         return true;
00482     } catch (const DatabaseEx &e) {
00483         logger->error(QString("returnIngredients: ") + e.what());
00484         emit errorOccurred(e.what());
00485         return false;
00486     } catch (const ValidateEx &e) {
00487         logger->warning(QString("returnIngredients: ") + e.what());
00488         emit errorOccurred(e.what());
00489         return false;
00490     } catch (const std::exception &e) {
00491         logger->error(QString("returnIngredients: неизвестная ошибка - ") + e.what());
00492         emit errorOccurred("Внутренняя ошибка при возврате продуктов");
00493         return false;
00494     }
00495 }
00496
00497 bool DBWorker::refreshAllFoodsAvailability()
00498 {
00499     try {
00500         if (!db.isOpen())
00501             throw DatabaseEx("База данных не открыта");
00502
00503         QSqlQuery foodsQuery;
00504         foodsQuery.prepare("SELECT FoodId FROM Food");
00505         if (!foodsQuery.exec())
00506             throw DatabaseEx("Ошибка получения списка блюд: " + foodsQuery.lastError().text());
00507
00508         QSqlDatabase::database().transaction();
00509         bool allOk = true;
00510         while (foodsQuery.next()) {
00511             int foodId = foodsQuery.value(0).toInt();
00512
00513             QSqlQuery compQuery;
00514             compQuery.prepare("SELECT ProductId, Quantity FROM FoodComposition WHERE FoodId =
:foodId");
00515             compQuery.bindValue(":foodId", foodId);
00516             if (!compQuery.exec())
00517                 throw DatabaseEx("Ошибка состава блюда: " + compQuery.lastError().text());
00518
00519             bool available = true;
00520             while (compQuery.next()) {
00521                 int productId = compQuery.value(0).toInt();
00522                 double needed = compQuery.value(1).toDouble();
00523
00524                 QSqlQuery stockQuery;
00525                 stockQuery.prepare("SELECT Quantity FROM Product WHERE ProductId = :pid");

```



```

00526         stockQuery.bindValue(":pid", productId);
00527         if (!stockQuery.exec() || !stockQuery.next())
00528             throw DatabaseEx("Ошибка остатка продукта: " + stockQuery.lastError().text());
00529
00530         double current = stockQuery.value(0).toDouble();
00531         if (current < needed - 0.0001) {
00532             available = false;
00533             break;
00534         }
00535     }
00536
00537     QSqlQuery updateQuery;
00538     updateQuery.prepare("UPDATE Food SET Available = :avail WHERE FoodId = :foodId");
00539     updateQuery.bindValue(":avail", available);
00540     updateQuery.bindValue(":foodId", foodId);
00541     if (!updateQuery.exec())
00542         throw DatabaseEx("Ошибка обновления доступности блюда: " +
updateQuery.lastError().text());
00543     }
00544
00545     if (allOk) {
00546         QSqlDatabase::database().commit();
00547         emit foodAvailabilityChanged();
00548         return true;
00549     } else {
00550         QSqlDatabase::database().rollback();
00551         return false;
00552     }
00553 } catch (const DatabaseEx &e) {
00554     logger->error(QString("refreshAllFoodsAvailability: ") + e.what());
00555     emit errorOccurred(e.what());
00556     return false;
00557 } catch (const std::exception &e) {
00558     logger->error(QString("refreshAllFoodsAvailability: неизвестная ошибка - ") + e.what());
00559     emit errorOccurred("Внутренняя ошибка при обновлении доступности");
00560     return false;
00561 }
00562 }
00563
00564 QVariantList DBWorker::getLowStockWarning(int foodId, int quantity)
00565 {
00566     try {
00567         if (!db.isOpen())
00568             throw DatabaseEx("База данных не открыта");
00569
00570         QVariantList result;
00571         QSqlQuery compQuery;
00572         compQuery.prepare("SELECT ProductId, Quantity FROM FoodComposition WHERE FoodId = :foodId");
00573         compQuery.bindValue(":foodId", foodId);
00574         if (!compQuery.exec())
00575             throw DatabaseEx("Ошибка получения состава для предупреждения: " +
compQuery.lastError().text());
00576
00577         while (compQuery.next()) {
00578             int productId = compQuery.value(0).toInt();
00579             double need = compQuery.value(1).toDouble() * quantity;
00580
00581             QSqlQuery prodQuery;
00582             prodQuery.prepare("SELECT FullName, Quantity, MinThreshold FROM Product WHERE ProductId =
:pid");
00583             prodQuery.bindValue(":pid", productId);
00584             if (!prodQuery.exec() || !prodQuery.next())
00585                 continue;
00586
00587             QString name = prodQuery.value(0).toString();
00588             double current = prodQuery.value(1).toDouble();
00589             double minThr = prodQuery.value(2).toDouble();
00590             double newQty = current - need;
00591
00592             if (newQty < minThr) {
00593                 QVariantMap warning;
00594                 warning["productName"] = name;
00595                 warning["currentQuantity"] = current;
00596                 warning["newQuantity"] = newQty;
00597                 warning["minThreshold"] = minThr;
00598                 result.append(warning);
00599             }
00600         }
00601         return result;
00602     } catch (const DatabaseEx &e) {
00603         logger->error(QString("getLowStockWarning: ") + e.what());
00604         emit errorOccurred(e.what());
00605         return QVariantList();
00606     } catch (const std::exception &e) {
00607         logger->error(QString("getLowStockWarning: неизвестная ошибка - ") + e.what());
00608         emit errorOccurred("Внутренняя ошибка при проверке остатков");
00609         return QVariantList();

```

```

00610     }
00611 }
00612
00613 QVariantList DBWorker::getDishesList()
00614 {
00615     try {
00616         if (!db.isOpen())
00617             throw DatabaseEx("База данных не открыта");
00618
00619         QVariantList result;
00620         QSqlQuery query;
00621         query.prepare("SELECT FoodId, FullName FROM Food ORDER BY FullName");
00622         if (!query.exec())
00623             throw DatabaseEx("Ошибка getDishesList: " + query.lastError().text());
00624
00625         while (query.next()) {
00626             QVariantMap item;
00627             item["id"] = query.value(0).toInt();
00628             item["name"] = query.value(1).toString();
00629             result.append(item);
00630         }
00631         return result;
00632     } catch (const DatabaseEx &e) {
00633         logger->error(QString("getDishesList: ") + e.what());
00634         emit errorOccurred(e.what());
00635         return QVariantList();
00636     } catch (const std::exception &e) {
00637         logger->error(QString("getDishesList: неизвестная ошибка - ") + e.what());
00638         emit errorOccurred("Внутренняя ошибка при загрузке списка блюд");
00639         return QVariantList();
00640     }
00641 }
00642
00643 QVariantList DBWorker::getWaitersList()
00644 {
00645     try {
00646         if (!db.isOpen())
00647             throw DatabaseEx("База данных не открыта");
00648
00649         QVariantList result;
00650         QSqlQuery query;
00651         query.prepare("SELECT UserId, FullName FROM Users WHERE Role = 'waiter' ORDER BY FullName");
00652         if (!query.exec())
00653             throw DatabaseEx("Ошибка getWaitersList: " + query.lastError().text());
00654
00655         while (query.next()) {
00656             QVariantMap item;
00657             item["id"] = query.value(0).toInt();
00658             item["name"] = query.value(1).toString();
00659             result.append(item);
00660         }
00661         return result;
00662     } catch (const DatabaseEx &e) {
00663         logger->error(QString("getWaitersList: ") + e.what());
00664         emit errorOccurred(e.what());
00665         return QVariantList();
00666     } catch (const std::exception &e) {
00667         logger->error(QString("getWaitersList: неизвестная ошибка - ") + e.what());
00668         emit errorOccurred("Внутренняя ошибка при загрузке списка официантов");
00669         return QVariantList();
00670     }
00671 }
00672
00673 QVariantMap DBWorker::getReportRevenue(const QString &from, const QString &to,
00674                                         int dishId, int tableNumber, int waiterId)
00675 {
00676     try {
00677         if (!db.isOpen())
00678             throw DatabaseEx("База данных не открыта");
00679
00680         QVariantMap result;
00681         QDate fromDate = QDate::fromString(from, "yyyy-MM-dd");
00682         QDate toDate = QDate::fromString(to, "yyyy-MM-dd");
00683         if (!fromDate.isValid() || !toDate.isValid()) {
00684             result["error"] = "Неверный формат даты. Используйте ГГГГ-ММ-ДД";
00685             return result;
00686         }
00687
00688         if (db.isOpen()) db.rollback();
00689
00690         QString sql = "SELECT COALESCE(SUM(o.TotalCost), 0) FROM Orders o ";
00691         QString where = "WHERE o.OrderDate >= :from AND o.OrderDate <= :to";
00692
00693         if (dishId > 0) {
00694             sql += "JOIN OrderFood of ON o.OrderId = of.OrderId ";
00695             where += " AND of.FoodId = :dishId";
00696         }

```



```

00697         if (tableNumber > 0) {
00698             where += " AND o.TableNumber = :table";
00699         }
00700         if (waiterId > 0) {
00701             where += " AND o.UserId = :waiter";
00702         }
00703         sql += where;
00704
00705         QSqlQuery query;
00706         query.prepare(sql);
00707         query.bindValue(":from", fromDate);
00708         query.bindValue(":to", toDate);
00709         if (dishId > 0) query.bindValue(":dishId", dishId);
00710         if (tableNumber > 0) query.bindValue(":table", tableNumber);
00711         if (waiterId > 0) query.bindValue(":waiter", waiterId);
00712
00713         if (!query.exec())
00714             throw DatabaseEx("Ошибка getReportRevenue: " + query.lastError().text());
00715
00716         if (query.next()) {
00717             result["totalRevenue"] = query.value(0).toDouble();
00718         } else {
00719             result["totalRevenue"] = 0.0;
00720         }
00721         return result;
00722     } catch (const DatabaseEx &e) {
00723         logger->error(QString("getReportRevenue: ") + e.what());
00724         emit errorOccurred(e.what());
00725         QVariantMap err;
00726         err["error"] = e.what();
00727         return err;
00728     } catch (const std::exception &e) {
00729         logger->error(QString("getReportRevenue: неизвестная ошибка - ") + e.what());
00730         emit errorOccurred("Внутренняя ошибка при формировании отчёта");
00731         QVariantMap err;
00732         err["error"] = "Внутренняя ошибка";
00733         return err;
00734     }
00735 }
00736
00737 QVariantList DBWorker::getRevenueByDay(const QString &from, const QString &to,
00738                                       int dishId, int tableNumber, int waiterId)
00739 {
00740     try {
00741         if (!db.isOpen())
00742             throw DatabaseEx("База данных не открыта");
00743
00744         QVariantList result;
00745         QDate fromDate = QDate::fromString(from, "yyyy-MM-dd");
00746         QDate toDate = QDate::fromString(to, "yyyy-MM-dd");
00747         if (!fromDate.isValid() || !toDate.isValid())
00748             throw ValidateEx("Неверный формат даты");
00749
00750         if (db.isOpen()) db.rollback();
00751
00752         QString sql = "SELECT DATE(o.OrderDate) as day, COALESCE(SUM(o.TotalCost), 0) as total "
00753                     "FROM Orders o ";
00754         QString where = "WHERE o.OrderDate >= :from AND o.OrderDate <= :to";
00755
00756         if (dishId > 0) {
00757             sql += "JOIN OrderFood of ON o.OrderId = of.OrderId ";
00758             where += " AND of.FoodId = :dishId";
00759         }
00760         if (tableNumber > 0) {
00761             where += " AND o.TableNumber = :table";
00762         }
00763         if (waiterId > 0) {
00764             where += " AND o.UserId = :waiter";
00765         }
00766         sql += where + " GROUP BY DATE(o.OrderDate) ORDER BY day";
00767
00768         QSqlQuery query;
00769         query.prepare(sql);
00770         query.bindValue(":from", fromDate);
00771         query.bindValue(":to", toDate);
00772         if (dishId > 0) query.bindValue(":dishId", dishId);
00773         if (tableNumber > 0) query.bindValue(":table", tableNumber);
00774         if (waiterId > 0) query.bindValue(":waiter", waiterId);
00775
00776         if (!query.exec())
00777             throw DatabaseEx("Ошибка getRevenueByDay: " + query.lastError().text());
00778
00779         while (query.next()) {
00780             QVariantMap dayReport;
00781             dayReport["date"] = query.value(0).toDate().toString("yyyy-MM-dd");
00782             dayReport["total"] = query.value(1).toDouble();
00783             result.append(dayReport);

```

```

00784     }
00785     return result;
00786 } catch (const DatabaseEx &e) {
00787     logger->error(QString("getRevenueByDay: ") + e.what());
00788     emit errorOccurred(e.what());
00789     return QVariantList();
00790 } catch (const ValidateEx &e) {
00791     logger->warning(QString("getRevenueByDay: ") + e.what());
00792     emit errorOccurred(e.what());
00793     return QVariantList();
00794 } catch (const std::exception &e) {
00795     logger->error(QString("getRevenueByDay: неизвестная ошибка - ") + e.what());
00796     emit errorOccurred("Внутренняя ошибка при формировании отчёта по дням");
00797     return QVariantList();
00798 }
00799 }
00800
00801 bool DBWorker::reconnect(const QString &host, int port, const QString &dbName, const QString &user,
const QString &password)
00802 {
00803     try {
00804         if (db.isOpen()) db.close();
00805         db.setHostName(host);
00806         db.setPort(port);
00807         db.setDatabaseName(dbName);
00808         db.setUserName(user);
00809         db.setPassword(password);
00810         if (!db.open())
00811             throw DatabaseEx("Ошибка подключения к БД: " + db.lastError().text());
00812         return true;
00813     } catch (const DatabaseEx &e) {
00814         logger->error(QString("reconnect: ") + e.what());
00815         emit errorOccurred(e.what());
00816         return false;
00817     } catch (const std::exception &e) {
00818         logger->error(QString("reconnect: неизвестная ошибка - ") + e.what());
00819         emit errorOccurred("Не удалось переподключиться к базе данных");
00820         return false;
00821     }
00822 }
00823
00824 QStringList DBWorker::getLocalIps()
00825 {
00826     QStringList result;
00827     const QList<QNetworkInterface> interfaces = QNetworkInterface::allInterfaces();
00828     for (const QNetworkInterface &iface : interfaces) {
00829         if (!(iface.flags() & QNetworkInterface::IsUp) || (iface.flags() &
QNetworkInterface::IsLoopBack))
00830             continue;
00831         for (const QNetworkAddressEntry &entry : iface.addressEntries()) {
00832             if (entry.ip().protocol() == QAbstractSocket::IPv4Protocol) {
00833                 result « entry.ip().toString();
00834             }
00835         }
00836     }
00837     return result;
00838 }
00839
00840 bool DBWorker::pingServer(const QString &host, int port)
00841 {
00842     Q_UNUSED(host);
00843     Q_UNUSED(port);
00844     return true;
00845 }
00846
00847 bool DBWorker::saveReportToCSV(const QString &reportType,
const QString &from,
const QString &to,
int dishId,
int tableNumber,
int waiterId,
const QString &fileName)
00848 {
00849     try {
00850         if (!db.isOpen())
00851             throw DatabaseEx("База данных не открыта");
00852         QFile file(fileName);
00853         if (!file.open(QIODevice::WriteOnly | QIODevice::Text)) {
00854             throw FileEx("Не удалось открыть файл для записи: " + fileName);
00855         }
00856         QTextStream out(&file);
00857         out.setEncoding(QStringConverter::Utf8);
00858         if (reportType == "revenue") {
00859             // Отчёт по выручке
00860             QVariantMap total = getReportRevenue(from, to, dishId, tableNumber, waiterId);

```

```

00869         QVariantList days = getRevenueByDay(from, to, dishId, tableNumber, waiterId);
00870
00871         out << "Общая выручка," << total["totalRevenue"].toDouble() << "\n";
00872         out << "\nДата, Выручка\n";
00873         for (const QVariant &day : days) {
00874             QVariantMap d = day.toMap();
00875             out << d["date"].toString() << ", " << d["total"].toDouble() << "\n";
00876         }
00877     }
00878     else if (reportType == "popularity") {
00879         QVariantList pop = getFoodPopularity(from, to);
00880         out << "Блюда, Количество порций\n";
00881         for (const QVariant &item : pop) {
00882             QVariantMap m = item.toMap();
00883             out << m["foodName"].toString() << ", " << m["count"].toInt() << "\n";
00884         }
00885     }
00886     else if (reportType == "lowstock") {
00887         QVariantList low = getReportLowStock();
00888         out << "Продукт, Текущее количество, Минимальный порог\n";
00889         for (const QVariant &item : low) {
00890             QVariantMap m = item.toMap();
00891             out << m["name"].toString() << ", "
00892                 << m["quantity"].toInt() << ", "
00893                 << m["minThreshold"].toInt() << "\n";
00894         }
00895     }
00896     else {
00897         throw ValidateEx("Неизвестный тип отчёта");
00898     }
00899
00900     file.close();
00901     logger->info("Отчёт сохранён в файл: " + fileName);
00902     return true;
00903 }
00904 catch (const DatabaseEx &e) {
00905     logger->error("saveReportToCSV: " + QString(e.what()));
00906     emit errorOccurred(e.what());
00907     return false;
00908 }
00909 catch (const FileEx &e) {
00910     logger->error("saveReportToCSV: " + QString(e.what()));
00911     emit errorOccurred(e.what());
00912     return false;
00913 }
00914 catch (const std::exception &e) {
00915     logger->error("saveReportToCSV: " + QString(e.what()));
00916     emit errorOccurred("Внутренняя ошибка при сохранении отчёта");
00917     return false;
00918 }
00919 }
00920
00921 // ===== Генератор номера заказа (ежедневный сброс) =====
00922 int DBWorker::generateOrderNumber()
00923 {
00924     try {
00925         if (!db.isOpen()) throw DatabaseEx("База данных не открыта");
00926         QSqlQuery query;
00927         // Получаем максимальный номер заказа за сегодня
00928         query.prepare("SELECT COALESCE(MAX(order_number), 0) + 1 FROM Orders WHERE DATE(OrderDate) =
CURRENT_DATE");
00929         if (!query.exec())
00930             throw DatabaseEx("Ошибка при генерации номера заказа: " + query.lastError().text());
00931         if (query.next())
00932             return query.value(0).toInt();
00933         else
00934             return 1;
00935     } catch (const DatabaseEx &e) {
00936         logger->error(QString("generateOrderNumber: ") + e.what());
00937         emit errorOccurred(e.what());
00938         return -1;
00939     } catch (const std::exception &e) {
00940         logger->error(QString("generateOrderNumber: неизвестная ошибка - ") + e.what());
00941         emit errorOccurred("Внутренняя ошибка при генерации номера заказа");
00942         return -1;
00943     }
00944 }
00945
00946 // ===== Получение списка заказов =====
00947 QVariantList DBWorker::getOrders(const QString &status)
00948 {
00949     try {
00950         if (!db.isOpen()) throw DatabaseEx("База данных не открыта");
00951         QVariantList result;
00952         QString sql = "SELECT o.OrderId, o.order_number, o.TableNumber, o.OrderDate, "
00953             "o.SpecialRequests, o.TotalCost, o.status, u.FullName AS waiterName "
00954             "FROM Orders o "

```

```

00955         "LEFT JOIN Users u ON o.UserId = u.UserId ";
00956     if (!status.isEmpty())
00957         sql += "WHERE status = '" + status + "' ";
00958     sql += "ORDER BY o.OrderDate DESC";
00959
00960     QSqlQuery query;
00961     if (!query.exec(sql))
00962         throw DatabaseEx("Ошибка getOrders: " + query.lastError().text());
00963
00964     while (query.next()) {
00965         QVariantMap order;
00966         order["orderId"] = query.value("OrderId").toInt();           // было OrderId ->
00967         order["orderNumber"] = query.value("order_number").toInt();
00968         order["tableNumber"] = query.value("TableNumber").toInt();   // TableNumber ->
00969         order["orderDate"] = query.value("OrderDate").toDate();
00970         order["specialRequests"] = query.value("SpecialRequests").toString();
00971         order["totalCost"] = query.value("TotalCost").toDouble();     // TotalCost ->
00972         order["status"] = query.value("status").toString();
00973         order["waiterName"] = query.value("waiterName").toString();  // новое поле
00974         result.append(order);
00975     }
00976     logger->info("getOrders returned " + QString::number(result.size()) + " orders");
00977     return result;
00978 } catch (const DatabaseEx &e) {
00979     logger->error(QString("getOrders: ") + e.what());
00980     emit errorOccurred(e.what());
00981     return QVariantList();
00982 } catch (const std::exception &e) {
00983     logger->error(QString("getOrders: неизвестная ошибка - ") + e.what());
00984     emit errorOccurred("Внутренняя ошибка при загрузке заказов");
00985     return QVariantList();
00986 }
00987 }
00988
00989 // ===== Обновление статуса заказа =====
00990 bool DBWorker::updateOrderStatus(int OrderId, const QString &newStatus)
00991 {
00992     try {
00993         if (!db.isOpen()) throw DatabaseEx("База данных не открыта");
00994         QSqlQuery query;
00995         query.prepare("UPDATE Orders SET status = :status WHERE OrderId = :id");
00996         query.bindValue(":status", newStatus);
00997         query.bindValue(":id", OrderId);
00998         if (!query.exec())
00999             throw DatabaseEx("Ошибка updateOrderStatus: " + query.lastError().text());
01000         logger->info("Статус заказа " + QString::number(OrderId) + " изменён на " + newStatus);
01001         emit ordersChanged();
01002         return true;
01003     } catch (const DatabaseEx &e) {
01004         logger->error(QString("updateOrderStatus: ") + e.what());
01005         emit errorOccurred(e.what());
01006         return false;
01007     } catch (const std::exception &e) {
01008         logger->error(QString("updateOrderStatus: неизвестная ошибка - ") + e.what());
01009         emit errorOccurred("Ошибка при изменении статуса заказа");
01010         return false;
01011     }
01012 }
01013
01014 // ===== Получение позиций заказа =====
01015 QVariantList DBWorker::getOrderItems(int OrderId)
01016 {
01017     try {
01018         if (!db.isOpen()) throw DatabaseEx("База данных не открыта");
01019         QVariantList result;
01020         QSqlQuery query;
01021         query.prepare("SELECT of.OrderFoodId, of.FoodId, f.FullName, of.Quantity, of.Price "
01022             "FROM OrderFood of "
01023             "JOIN Food f ON of.FoodId = f.FoodId "
01024             "WHERE of.OrderId = :OrderId");
01025         query.bindValue(":OrderId", OrderId);
01026         if (!query.exec())
01027             throw DatabaseEx("Ошибка getOrderItems: " + query.lastError().text());
01028         while (query.next()) {
01029             QVariantMap item;
01030             item["orderFoodId"] = query.value("OrderFoodId").toInt();
01031             item["foodId"] = query.value("FoodId").toInt();
01032             item["name"] = query.value("FullName").toString();
01033             item["quantity"] = query.value("Quantity").toInt();
01034             item["price"] = query.value("Price").toDouble();
01035             result.append(item);
01036         }
01037         return result;
01038     } catch (const DatabaseEx &e) {

```

```

01039         logger->error(QString("getOrderItems: ") + e.what());
01040         emit errorOccurred(e.what());
01041         return QVariantList();
01042     } catch (const std::exception &e) {
01043         logger->error(QString("getOrderItems: неизвестная ошибка - ") + e.what());
01044         emit errorOccurred("Ошибка загрузки позиций заказа");
01045         return QVariantList();
01046     }
01047 }
01048
01049 // ===== Закрытие заказа и создание чека =====
01050 bool DBWorker::closeOrder(int OrderId, double totalSum, const QVariantList &items)
01051 {
01052     try {
01053         if (!db.isOpen()) throw DatabaseEx("База данных не открыта");
01054
01055         // 1. Получить номер заказа и дату
01056         QSqlQuery orderQuery;
01057         orderQuery.prepare("SELECT order_number, OrderDate FROM Orders WHERE OrderId = :id");
01058         orderQuery.bindValue(":id", OrderId);
01059         if (!orderQuery.exec() || !orderQuery.next())
01060             throw DatabaseEx("Не удалось получить данные заказа");
01061         int orderNumber = orderQuery.value("order_number").toInt();
01062         QDate orderDate = orderQuery.value("OrderDate").toDate();
01063
01064         // 2. Сформировать номер чека: ЧЕК-ГГГГММДД-XXX
01065         QString checkNumber = QString("ЧЕК-%1-%2")
01066             .arg(orderDate.toString("yyyyMMdd"))
01067             .arg(orderNumber, 3, 10, QChar('0'));
01068
01069         // 3. Подготовить JSON с позициями
01070         QJsonArray itemsArray;
01071         for (const QVariant &v : items) {
01072             QVariantMap m = v.toMap();
01073             QJsonObject obj;
01074             obj["name"] = m["name"].toString();
01075             obj["quantity"] = m["quantity"].toInt();
01076             obj["price"] = m["price"].toDouble();
01077             itemsArray.append(obj);
01078         }
01079         QJsonDocument doc(itemsArray);
01080         QString checkData = QString::fromUtf8(doc.toJson(QJsonDocument::Compact));
01081
01082         // 4. Вставить запись в Checks
01083         QSqlQuery insertCheck;
01084         insertCheck.prepare("INSERT INTO Checks (order_id, check_number, order_number, total_sum,
01085             check_data) "
01086             "VALUES (:oid, :cnum, :onum, :sum, :data)");
01087         insertCheck.bindValue(":oid", OrderId);
01088         insertCheck.bindValue(":cnum", checkNumber);
01089         insertCheck.bindValue(":onum", orderNumber);
01090         insertCheck.bindValue(":sum", totalSum);
01091         insertCheck.bindValue(":data", checkData);
01092         if (!insertCheck.exec())
01093             throw DatabaseEx("Ошибка сохранения чека: " + insertCheck.lastError().text());
01094
01095         QSqlQuery updateOrder;
01096         updateOrder.prepare("UPDATE Orders SET status = 'Закрыт', TotalCost = :total WHERE OrderId =
01097             :id");
01098         updateOrder.bindValue(":total", totalSum);
01099         updateOrder.bindValue(":id", OrderId);
01100         if (!updateOrder.exec())
01101             throw DatabaseEx("Ошибка обновления статуса заказа: " + updateOrder.lastError().text());
01102
01103         logger->info("Заказ " + QString::number(OrderId) + " закрыт, чек " + checkNumber);
01104         emit ordersChanged();
01105         return true;
01106     } catch (const DatabaseEx &e) {
01107         logger->error(QString("closeOrder: ") + e.what());
01108         emit errorOccurred(e.what());
01109         return false;
01110     } catch (const std::exception &e) {
01111         logger->error(QString("closeOrder: неизвестная ошибка - ") + e.what());
01112         emit errorOccurred("Ошибка при закрытии заказа");
01113         return false;
01114     }
01115 }
01116
01117 QVariantMap DBWorker::getCheckByOrderId(int OrderId)
01118 {
01119     try {
01120         if (!db.isOpen()) throw DatabaseEx("База данных не открыта");
01121         QSqlQuery query;
01122         query.prepare("SELECT check_id, check_number, order_number, total_sum, check_data, created_at
01123             "
01124             "FROM Checks WHERE order_id = :oid ORDER BY created_at DESC LIMIT 1");
01125         query.bindValue(":oid", OrderId);

```

```

01123         if (!query.exec())
01124             throw DatabaseEx("Ошибка getCheckByOrderId: " + query.lastError().text());
01125         if (query.next()) {
01126             QVariantMap check;
01127             check["checkId"] = query.value("check_id").toInt();
01128             check["checkNumber"] = query.value("check_number").toString();
01129             check["orderNumber"] = query.value("order_number").toInt();
01130             check["totalSum"] = query.value("total_sum").toDouble();
01131             check["checkData"] = query.value("check_data").toString();
01132             check["createdAt"] = query.value("created_at").toDate();
01133             return check;
01134         }
01135         return QVariantMap();
01136     } catch (const DatabaseEx &e) {
01137         logger->error(QString("getCheckByOrderId: ") + e.what());
01138         emit errorOccurred(e.what());
01139         return QVariantMap();
01140     } catch (const std::exception &e) {
01141         logger->error(QString("getCheckByOrderId: неизвестная ошибка - ") + e.what());
01142         emit errorOccurred("Ошибка получения чека");
01143         return QVariantMap();
01144     }
01145 }
01146
01147 bool DBWorker::syncOrderItems(int OrderId, const QVariantList &newItems)
01148 {
01149     try {
01150         if (!db.isOpen()) throw DatabaseEx("База данных не открыта");
01151
01152         if (OrderId <= 0) {
01153             logger->error("syncOrderItems: invalid orderId " + QString::number(OrderId));
01154             emit errorOccurred("Недействительный идентификатор заказа");
01155             return false;
01156         }
01157
01158         QVariantList oldItems = getOrderItems(OrderId);
01159         if (oldItems.isEmpty() && !newItems.isEmpty()) {
01160             // Если старых нет, просто добавляем новые - но обычно их нет.
01161         }
01162
01163         QMap<int, QVariantMap> oldMap;
01164         for (const QVariant &v : oldItems) {
01165             QVariantMap m = v.toMap();
01166             oldMap[m["foodId"].toInt()] = m;
01167         }
01168         QMap<int, QVariantMap> newMap;
01169         for (const QVariant &v : newItems) {
01170             QVariantMap m = v.toMap();
01171             newMap[m["foodId"].toInt()] = m;
01172         }
01173
01174         for (auto it = newMap.begin(); it != newMap.end(); ++it) {
01175             int foodId = it.key();
01176             int newQty = it.value()["quantity"].toInt();
01177             if (!oldMap.contains(foodId)) {
01178                 if (!checkAndConsume(foodId, newQty))
01179                     throw ValidateEx("Не удалось списать ингредиенты для блюда " +
01180                                     QString::number(foodId));
01181                 QSqlQuery insert;
01182                 insert.prepare("INSERT INTO OrderFood (OrderId, FoodId, Quantity, Price) VALUES (:oid,
01183                                     :fid, :qty, :price)");
01184                 insert.bindValue(":oid", OrderId);
01185                 insert.bindValue(":fid", foodId);
01186                 insert.bindValue(":qty", newQty);
01187                 insert.bindValue(":price", it.value()["price"].toDouble());
01188                 if (!insert.exec())
01189                     throw DatabaseEx("Ошибка добавления позиции: " + insert.lastError().text());
01190             } else {
01191                 int oldQty = oldMap[foodId]["quantity"].toInt();
01192                 if (newQty != oldQty) {
01193                     if (!returnIngredients(foodId, oldQty))
01194                         throw ValidateEx("Ошибка возврата ингредиентов");
01195                     if (!checkAndConsume(foodId, newQty))
01196                         throw ValidateEx("Ошибка списания ингредиентов при изменении количества");
01197                     QSqlQuery update;
01198                     update.prepare("UPDATE OrderFood SET Quantity = :qty WHERE OrderFoodId = :id");
01199                     update.bindValue(":qty", newQty);
01200                     update.bindValue(":id", oldMap[foodId]["orderFoodId"].toInt());
01201                     if (!update.exec())
01202                         throw DatabaseEx("Ошибка обновления количества: " +
01203                                         update.lastError().text());
01204                 }
01205             }
01206         }
01207
01208         for (auto it = oldMap.begin(); it != oldMap.end(); ++it) {
01209             int foodId = it.key();

```



```

01207         if (!newMap.contains(foodId)) {
01208             int oldQty = it.value()["quantity"].toInt();
01209             if (!returnIngredients(foodId, oldQty))
01210                 throw ValidateEx("Ошибка возврата ингредиентов при удалении блюда");
01211             QSqlQuery del;
01212             del.prepare("DELETE FROM OrderFood WHERE OrderFoodId = :id");
01213             del.bindValue(":id", it.value()["orderFoodId"].toInt());
01214             if (!del.exec())
01215                 throw DatabaseEx("Ошибка удаления позиции: " + del.lastError().text());
01216         }
01217     }
01218
01219     QSqlQuery sumQuery;
01220     sumQuery.prepare("SELECT COALESCE(SUM(Quantity * Price), 0) FROM OrderFood WHERE OrderId =
:oid");
01221     sumQuery.bindValue(":oid", OrderId);
01222     if (!sumQuery.exec() || !sumQuery.next())
01223         throw DatabaseEx("Ошибка пересчёта суммы");
01224     double newTotal = sumQuery.value(0).toDouble();
01225     QSqlQuery updateTotal;
01226     updateTotal.prepare("UPDATE Orders SET TotalCost = :total WHERE OrderId = :id");
01227     updateTotal.bindValue(":total", newTotal);
01228     updateTotal.bindValue(":id", OrderId);
01229     if (!updateTotal.exec())
01230         throw DatabaseEx("Ошибка обновления суммы заказа");
01231
01232     QSqlQuery updateStatus;
01233     updateStatus.prepare("UPDATE Orders SET status = 'Готовится' WHERE OrderId = :id AND status !=
'Закрыт'");
01234     updateStatus.bindValue(":id", OrderId);
01235     updateStatus.exec();
01236
01237     logger->info("Заказ " + QString::number(OrderId) + " синхронизирован, новая сумма " +
QString::number(newTotal));
01238     return true;
01239 } catch (const DatabaseEx &e) {
01240     logger->error(QString("syncOrderItems: ") + e.what());
01241     emit errorOccurred(e.what());
01242     return false;
01243 } catch (const ValidateEx &e) {
01244     logger->warning(QString("syncOrderItems: ") + e.what());
01245     emit errorOccurred(e.what());
01246     return false;
01247 } catch (const std::exception &e) {
01248     logger->error(QString("syncOrderItems: неизвестная ошибка - ") + e.what());
01249     emit errorOccurred("Внутренняя ошибка при синхронизации заказа");
01250     return false;
01251 }
01252 }
01253
01254 QVariantList DBWorker::getAllOrdersForReport()
01255 {
01256     return getOrders(QString());
01257 }
01258
01259 QVariantList DBWorker::getClosedOrders()
01260 {
01261     return getOrders("Закрыт");
01262 }
01263
01264 void DBWorker::updateConnectionParams(const QString &host, int port, const QString &dbName, const
QString &user, const QString &password)
01265 {
01266     db.setHostName(host);
01267     db.setPort(port);
01268     db.setDatabaseName(dbName);
01269     db.setUserName(user);
01270     db.setPassword(password);
01271     logger->info("Параметры подключения к БД обновлены");
01272 }
01273
01274 QVariantList DBWorker::getChecksByDateRange(const QString &from, const QString &to)
01275 {
01276     try {
01277         if (!db.isOpen())
01278             throw DatabaseEx("База данных не открыта");
01279
01280         QVariantList result;
01281         QDate fromDate = QDate::fromString(from, "yyyy-MM-dd");
01282         QDate toDate = QDate::fromString(to, "yyyy-MM-dd");
01283         if (!fromDate.isValid() || !toDate.isValid())
01284             throw ValidateEx("Неверный формат даты. Используйте ГГГГ-ММ-ДД");
01285
01286         QSqlQuery query;
01287         query.prepare(
01288             "SELECT c.check_number, c.order_number, c.total_sum, c.created_at, "
01289             "o.TableNumber, u.FullName AS waiterName, o.OrderId "

```

```

01290         "FROM Checks c "
01291         "JOIN Orders o ON c.order_id = o.OrderId "
01292         "LEFT JOIN Users u ON o.UserId = u.UserId "
01293         "WHERE DATE(c.created_at) BETWEEN :from AND :to "
01294         "ORDER BY c.created_at DESC"
01295     );
01296     query.bindValue(":from", fromDate);
01297     query.bindValue(":to", toDate);
01298     if (!query.exec())
01299         throw DatabaseEx("Ошибка getChecksByDateRange: " + query.lastError().text());
01300
01301     while (query.next()) {
01302         QVariantMap check;
01303         check["checkNumber"] = query.value("check_number").toString();
01304         check["orderNumber"] = query.value("order_number").toInt();
01305         check["totalSum"] = query.value("total_sum").toDouble();
01306         check["createdAt"] = query.value("created_at").toDateTime();
01307         check["tableNumber"] = query.value("TableNumber").toInt();
01308         check["waiterName"] = query.value("waiterName").toString();
01309         check["orderId"] = query.value("OrderId").toInt();    // для просмотра чека
01310         result.append(check);
01311     }
01312     logger->info("getChecksByDateRange: найдено " + QString::number(result.size()) + " чеков");
01313     return result;
01314 }
01315 catch (const DatabaseEx &e) {
01316     logger->error(QString("getChecksByDateRange: ") + e.what());
01317     emit errorOccurred(e.what());
01318     return QVariantList();
01319 }
01320 catch (const ValidateEx &e) {
01321     logger->warning(QString("getChecksByDateRange: ") + e.what());
01322     emit errorOccurred(e.what());
01323     return QVariantList();
01324 }
01325 catch (const std::exception &e) {
01326     logger->error(QString("getChecksByDateRange: неизвестная ошибка - ") + e.what());
01327     emit errorOccurred("Внутренняя ошибка при загрузке чеков");
01328     return QVariantList();
01329 }
01330 }

```

6.8. exceptions.cpp

```

00001 #include "exceptions.h"
00002
00003 Exceptions::Exceptions(const QString &message)
00004     : msg(message.toStdString())
00005 {
00006 }
00007
00008 const char* Exceptions::what() const noexcept
00009 {
00010     return msg.c_str();
00011 }

```

6.9. logs.cpp

```

00001 #include "logs.h"
00002 #include <QDateTime>
00003 #include <QDebug>
00004 #include <QStandardPaths>
00005 #include <QDir>
00006
00007 Logs::Logs(QObject *parent) : QObject(parent), m_logPath("logs.txt")
00008 {
00009     openLogFile();
00010 }
00011
00012 Logs::~Logs()
00013 {
00014     if (logFile.isOpen())
00015         logFile.close();
00016 }
00017
00018 void Logs::setLogPath(const QString &path)
00019 {
00020     if (m_logPath == path) return;
00021     m_logPath = path;
00022     openLogFile();
00023 }

```



```

00024
00025 void Logs::openLogFile()
00026 {
00027     if (logFile.isOpen())
00028         logFile.close();
00029
00030     QString actualPath = m_logPath;
00031     QFileInfo info(actualPath);
00032     if (info.isRelative()) {
00033         actualPath = QDir::current().absoluteFilePath(actualPath);
00034     }
00035
00036     logFile.setFileName(actualPath);
00037     if (logFile.open(QIODevice::Append | QIODevice::Text)) {
00038         out.setDevice(&logFile);
00039         writeLog("INF", "Лог-файл открыт: " + actualPath);
00040         return;
00041     }
00042
00043     QString fallbackDir = QStandardPaths::writableLocation(QStandardPaths::AppLocalDataLocation);
00044     if (fallbackDir.isEmpty())
00045         fallbackDir = QDir::tempPath();
00046     else
00047         QDir().mkpath(fallbackDir);
00048
00049     QString fallbackPath = fallbackDir + "/logs.txt";
00050     logFile.setFileName(fallbackPath);
00051     if (logFile.open(QIODevice::Append | QIODevice::Text)) {
00052         out.setDevice(&logFile);
00053         writeLog("WARN", "Не удалось открыть лог по пути " + actualPath + ", используется " +
00054             fallbackPath);
00055     } else {
00056         qWarning() << "Критическая ошибка: не удалось открыть лог-файл даже в системной папке";
00057     }
00058
00059 void Logs::writeLog(const QString &level, const QString &message)
00060 {
00061     if (!logFile.isOpen()) return;
00062     QString timestamp = QDateTime::currentDateTime().toString("yyyy-MM-dd hh:mm:ss");
00063     QString line = QString("%1 [%2] %3\n").arg(timestamp, level, message);
00064     out << line;
00065     out.flush();
00066
00067     if (level == "ERR") qCritical() << message;
00068     else if (level == "WARN") qWarning() << message;
00069     else qInfo() << message;
00070 }
00071
00072 void Logs::info(const QString &msg) { writeLog("INF", msg); }
00073 void Logs::warning(const QString &msg) { writeLog("WARN", msg); }
00074 void Logs::error(const QString &msg) { writeLog("ERR", msg); }

```

6.10. main.cpp

```

00001 #include <QGuiApplication>
00002 #include <QIcon>
00003 #include <QQmlApplicationEngine>
00004 #include <QQmlContext>
00005 #include <QSettings>
00006 #include <QDirIterator>
00007 #include "dbworker.h"
00008 #include "logs.h"
00009 #include "settings.h"
00010 #include "tcpclient.h"
00011 #include "tcpserver.h"
00012
00013 int main(int argc, char *argv[])
00014 {
00015     QGuiApplication app(argc, argv);
00016     QQmlApplicationEngine engine;
00017
00018     app.setWindowIcon(QIcon(":/resources/icons/logo.png"));
00019
00020     Logs logger;
00021     Logs *loger = &logger;
00022
00023     Settings set(loger);
00024     engine.rootContext()->setContextProperty("appSettings", &set);
00025
00026     loger->setLogPath(set.logPath());
00027     loger->info("Запуск приложения");
00028     engine.rootContext()->setContextProperty("loger", loger);
00029     loger->info("Настройки загружены");

```

```

00030
00031 QSettings qsettings("RestaurantApp", "Settings");
00032 bool serverMode = qsettings.value("isServerMode", false).toBool();
00033 logger->info("Режим сервера: " + QString::number(serverMode));
00034
00035 if (serverMode) {
00036     logger->info("Запуск в серверном режиме");
00037     DBWorker *worker = new DBWorker(logger, &app);
00038     engine.rootContext()->setContextProperty("dbWorker", worker);
00039     logger->info("DBWorker создан");
00040
00041     TcpServer *tcpServer = new TcpServer(logger, worker, &app);
00042     quint16 port = static_cast<quint16>(set.serverPort());
00043     if (!tcpServer->start(port)) {
00044         logger->error("Не удалось запустить сервер");
00045         delete tcpServer;
00046         delete worker;
00047         return -1;
00048     }
00049     logger->info("Сервер запущен на порту " + QString::number(port));
00050 } else {
00051     logger->info("Запуск в клиентском режиме");
00052     TcpClient *client = new TcpClient(logger, &app);
00053     engine.rootContext()->setContextProperty("dbWorker", client);
00054
00055     QString host = set.serverHost();
00056     quint16 port = static_cast<quint16>(set.serverPort());
00057     if (!client->connectToServer(host, port)) {
00058         logger->error("Не удалось подключиться к серверу");
00059     } else {
00060         logger->info("Подключено к серверу " + host + ":" + QString::number(port));
00061     }
00062 }
00063
00064 QObject::connect(&engine, &QCoreApplicationEngine::objectCreationFailed,
00065                 &app, []() { QCoreApplication::exit(-1); }, Qt::QueuedConnection);
00066 engine.loadFromModule("app", "Main");
00067 logger->info("QML загружен");
00068
00069 return app.exec();
00070 }

```

6.11. settings.cpp

```

00001 #include "settings.h"
00002 #include <QSettings>
00003
00004 Settings::Settings(Logs *logger, QObject *parent) : QObject(parent), logger(logger)
00005 {
00006     load();
00007 }
00008
00009 void Settings::setDbHost(const QString &host)
00010 {
00011     if (m_dbHost == host) return;
00012     m_dbHost = host;
00013     emit dbHostChanged();
00014 }
00015
00016 void Settings::setDbPort(int port)
00017 {
00018     if (m_dbPort == port) return;
00019     m_dbPort = port;
00020     emit dbPortChanged();
00021 }
00022
00023 void Settings::setDbName(const QString &name)
00024 {
00025     if (m_dbName == name) return;
00026     m_dbName = name;
00027     emit dbNameChanged();
00028 }
00029
00030 void Settings::setDbUser(const QString &user)
00031 {
00032     if (m_dbUser == user) return;
00033     m_dbUser = user;
00034     emit dbUserChanged();
00035 }
00036
00037 void Settings::setDbPassword(const QString &password)
00038 {
00039     if (m_dbPassword == password) return;
00040     m_dbPassword = password;

```

```

00041     emit dbPasswordChanged();
00042 }
00043
00044 void Settings::setReportsPath(const QString &path)
00045 {
00046     if (m_reportsPath == path) return;
00047     m_reportsPath = path;
00048     emit reportsPathChanged();
00049 }
00050
00051 void Settings::save()
00052 {
00053     QSettings settings(getSettingsFilePath(), QSettings::IniFormat);
00054     settings.setValue("dbHost", m_dbHost);
00055     settings.setValue("dbPort", m_dbPort);
00056     settings.setValue("dbName", m_dbName);
00057     settings.setValue("dbUser", m_dbUser);
00058     settings.setValue("dbPassword", m_dbPassword);
00059     settings.setValue("isServerMode", m_isServerMode);
00060     settings.setValue("logPath", m_logPath);
00061     settings.setValue("serverHost", m_serverHost);
00062     settings.setValue("serverPort", m_serverPort);
00063     settings.setValue("reportsPath", m_reportsPath);
00064     settings.sync();
00065     logger->info("Настройки сохранены в " + getSettingsFilePath());
00066 }
00067
00068 void Settings::load()
00069 {
00070     QSettings settings(getSettingsFilePath(), QSettings::IniFormat);
00071     m_dbHost = settings.value("dbHost", "127.0.0.1").toString();
00072     m_dbPort = settings.value("dbPort", 5432).toInt();
00073     m_dbName = settings.value("dbName", "RestN").toString();
00074     m_dbUser = settings.value("dbUser", "R").toString();
00075     m_dbPassword = settings.value("dbPassword", "").toString();
00076     m_isServerMode = settings.value("isServerMode", true).toBool();
00077     m_logPath = settings.value("logPath", "logs.txt").toString();
00078     m_serverHost = settings.value("serverHost", "127.0.0.1").toString();
00079     m_serverPort = settings.value("serverPort", 54321).toInt();
00080     m_reportsPath = settings.value("reportsPath",
QDir::current().absoluteFilePath("reports")).toString();
00081     logger->info("Настройки загружены из " + getSettingsFilePath());
00082 }
00083
00084 void Settings::setIsServerMode(bool mode)
00085 {
00086     if (m_isServerMode == mode) return;
00087     m_isServerMode = mode;
00088     emit isServerModeChanged();
00089 }
00090
00091 void Settings::setLogPath(const QString &path)
00092 {
00093     if (m_logPath == path) return;
00094     m_logPath = path;
00095     emit logPathChanged();
00096 }
00097
00098 void Settings::setServerHost(const QString &host)
00099 {
00100     if (m_serverHost == host) return;
00101     m_serverHost = host;
00102     emit serverHostChanged();
00103 }
00104 void Settings::setServerPort(int port)
00105 {
00106     if (m_serverPort == port) return;
00107     m_serverPort = port;
00108     emit serverPortChanged();
00109 }
00110
00111 QString Settings::getSettingsFilePath() const
00112 {
00113     QString appData = QStandardPaths::writableLocation(QStandardPaths::AppLocalDataLocation);
00114     QDir dir(appData);
00115     if (!dir.exists())
00116         dir.mkpath(".");
00117     return appData + "/settings.ini";
00118 }

```

6.12. tcpclient.cpp

```

00001 #include "tcpclient.h"
00002 #include <QEventLoop>

```

```

00003 #include <QJsonDocument>
00004 #include <QJsonObject>
00005 #include <QNetworkInterface>
00006 #include <QDebug>
00007 #include <QCoreApplication>
00008 #include <QTimer>
00009
00010 TcpClient::TcpClient(Logs *logger, QObject *parent)
00011     : QObject(parent), socket(nullptr), nextRequestId(1), userId(-1), logger(logger)
00012 {
00013     logger->info("TcpClient: создан");
00014 }
00015
00016 TcpClient::~TcpClient()
00017 {
00018     logger->info("TcpClient: уничтожение");
00019     disconnectFromServer();
00020 }
00021
00022 bool TcpClient::connectToServer(const QString &host, quint16 port)
00023 {
00024     logger->info("TcpClient: попытка подключения к " + host + ":" + QString::number(port));
00025     try {
00026         if (socket) disconnectFromServer();
00027         socket = new QTcpSocket(this);
00028         socket->connectToHost(host, port);
00029         if (!socket->waitForConnected(3000))
00030             throw NetworkEx("Ошибка подключения: " + socket->errorString());
00031
00032         logger->info("TcpClient: подключено к серверу");
00033
00034         connect(socket, &QTcpSocket::disconnected, this, [this]() {
00035             logger->error("TcpClient: соединение с сервером разорвано");
00036             emit errorOccurred("Соединение с сервером потеряно. Проверьте сервер и перезапустите
приложение.");
        });
00037
00038         connect(socket, &QTcpSocket::readyRead, this, [this]() {
00039             QByteArray newData = socket->readAll();
00040             logger->info("TcpClient: получено " + QString::number(newData.size()) + " байт");
00041             buffer.append(newData);
00042             while (true) {
00043                 int nl = buffer.indexOf('\n');
00044                 if (nl == -1) break;
00045                 QByteArray line = buffer.left(nl);
00046                 buffer.remove(0, nl + 1);
00047                 QJsonParseError err;
00048                 QJsonDocument doc = QJsonDocument::fromJson(line, &err);
00049                 if (err.error != QJsonParseError::NoError) {
00050                     logger->warning("TcpClient: ошибка парсинга JSON: " + err.errorString());
00051                     continue;
00052                 }
00053                 if (!doc.isObject()) continue;
00054                 QJsonObject obj = doc.object();
00055                 if (obj.contains("response")) {
00056                     int reqId = obj.value("requestId").toInt();
00057                     bool success = obj.value("response").toBool();
00058                     QVariantMap data = obj.value("data").toObject().toVariantMap();
00059                     data["_success"] = success;
00060                     pendingResponses[reqId] = data;
00061                     emit responseReceived(reqId, success, data);
00062                     logger->info("TcpClient: получен ответ на запрос " + QString::number(reqId) + ",
ycnex=" + QString(success ? "true" : "false"));
00063                 } else if (obj.contains("notification")) {
00064                     QString type = obj.value("notification").toString();
00065                     logger->info("TcpClient: получено уведомление: " + type);
00066                     if (type == "data_changed")
00067                         emit dataChanged();
00068                 }
00069             }
00070         });
00071     } catch (const NetworkEx &e) {
00072         logger->error(QString("connectToServer: ") + e.what());
00073         emit errorOccurred(e.what());
00074         if (socket) {
00075             socket->deleteLater();
00076             socket = nullptr;
00077         }
00078         return false;
00079     } catch (const std::exception &e) {
00080         logger->error(QString("connectToServer: неизвестная ошибка - ") + e.what());
00081         emit errorOccurred("Ошибка подключения к серверу");
00082         return false;
00083     }
00084 }
00085
00086 }
00087

```

```

00088
00089 void TcpClient::disconnectFromServer()
00090 {
00091     logger->info("TcpClient: отключение от сервера");
00092     if (socket) {
00093         socket->disconnectFromHost();
00094         socket->deleteLater();
00095         socket = nullptr;
00096     }
00097     buffer.clear();
00098     pendingResponses.clear();
00099 }
00100
00101 bool TcpClient::isConnected() const
00102 {
00103     return socket && socket->state() == QAbstractSocket::ConnectedState;
00104 }
00105
00106 int TcpClient::sendRequest(const QString &command, const QVariantMap &data)
00107 {
00108     logger->info("TcpClient: отправка запроса '" + command + "'");
00109     if (!isConnected())
00110         throw NetworkEx("Нет соединения с сервером");
00111
00112     int reqId = nextRequestId++;
00113     QJsonObject obj;
00114     obj["command"] = command;
00115     obj["requestId"] = reqId;
00116     if (!data.isEmpty())
00117         obj["data"] = QJsonObject::fromVariantMap(data);
00118     QByteArray msg = QJsonDocument(obj).toJson(QJsonDocument::Compact) + "\n";
00119     qint64 bytes = socket->write(msg);
00120     if (bytes == -1)
00121         throw NetworkEx("Ошибка записи в сокет: " + socket->errorString());
00122     socket->flush();
00123     logger->info("TcpClient: запрос " + QString::number(reqId) + " отправлен, байт: " +
00124         QString::number(bytes));
00125     return reqId;
00126 }
00127
00128 bool TcpClient::waitForResponse(int requestId, QVariantMap &resultData)
00129 {
00130     logger->info("TcpClient: ожидание ответа на запрос " + QString::number(requestId));
00131     if (requestId == -1) return false;
00132     QEventLoop loop;
00133     QTimer timer;
00134     timer.setSingleShot(true);
00135     timer.start(60000);
00136     connect(this, &TcpClient::responseReceived, &loop, [&](int id, bool, const QVariantMap&) {
00137         if (id == requestId) loop.quit();
00138     });
00139     connect(&timer, &QTimer::timeout, &loop, &QEventLoop::quit);
00140     loop.exec();
00141     if (!timer.isActive()) {
00142         logger->error("TcpClient: таймаут ожидания ответа на запрос " + QString::number(requestId));
00143         throw NetworkEx("Таймаут ожидания ответа от сервера");
00144     }
00145     resultData = pendingResponses.take(requestId);
00146     bool success = resultData.value("_success").toBool();
00147     logger->info("TcpClient: получен ответ на запрос " + QString::number(requestId) + ", успех=" +
00148         QString::number(success));
00149     return success;
00150 }
00151
00152 bool TcpClient::checkUser(const QString &login, const QString &password)
00153 {
00154     logger->info("TcpClient::checkUser: логин " + login);
00155     try {
00156         QVariantMap data;
00157         data["login"] = login;
00158         data["password"] = password;
00159         int reqId = sendRequest("checkUser", data);
00160         QVariantMap res;
00161         if (waitForResponse(reqId, res)) {
00162             userRole = res["role"].toString();
00163             userName = res["name"].toString();
00164             userId = res["userId"].toInt();
00165             logger->info("TcpClient::checkUser: успешно, роль=" + userRole);
00166             return true;
00167         }
00168         return false;
00169     } catch (const NetworkEx &e) {
00170         logger->error(QString("checkUser: ") + e.what());
00171         emit errorOccurred(e.what());
00172         return false;
00173     } catch (const std::exception &e) {
00174         logger->error(QString("checkUser: неизвестная ошибка - ") + e.what());
00175     }
00176 }

```

```

00173         emit errorOccurred("Ошибка связи с сервером при авторизации");
00174         return false;
00175     }
00176 }
00177
00178 QString TcpClient::getUserName() const { return userName; }
00179 QString TcpClient::getUserRole() const { return userRole; }
00180 int TcpClient::getUserId() const { return userId; }
00181
00182 QVariantList TcpClient::getFoods()
00183 {
00184     logger->info("TcpClient::getFoods");
00185     try {
00186         int reqId = sendRequest("getFoods");
00187         QVariantMap res;
00188         if (waitForResponse(reqId, res)) {
00189             QVariantList list = res["foods"].toList();
00190             logger->info("TcpClient::getFoods: получено " + QString::number(list.size()) + " блюд");
00191             return list;
00192         }
00193         return QVariantList();
00194     } catch (const NetworkEx &e) {
00195         logger->error(QString("getFoods: ") + e.what());
00196         emit errorOccurred(e.what());
00197         return QVariantList();
00198     } catch (const std::exception &e) {
00199         logger->error(QString("getFoods: неизвестная ошибка - ") + e.what());
00200         emit errorOccurred("Ошибка получения меню с сервера");
00201         return QVariantList();
00202     }
00203 }
00204
00205 QVariantList TcpClient::getProducts()
00206 {
00207     logger->info("TcpClient::getProducts");
00208     try {
00209         int reqId = sendRequest("getProducts");
00210         QVariantMap res;
00211         if (waitForResponse(reqId, res)) {
00212             QVariantList list = res["products"].toList();
00213             logger->info("TcpClient::getProducts: получено " + QString::number(list.size()) + "
продуктов");
00214             return list;
00215         }
00216         return QVariantList();
00217     } catch (const NetworkEx &e) {
00218         logger->error(QString("getProducts: ") + e.what());
00219         emit errorOccurred(e.what());
00220         return QVariantList();
00221     } catch (const std::exception &e) {
00222         logger->error(QString("getProducts: неизвестная ошибка - ") + e.what());
00223         emit errorOccurred("Ошибка получения списка продуктов с сервера");
00224         return QVariantList();
00225     }
00226 }
00227
00228 bool TcpClient::updateProductQuantity(int productId, int newQuantity)
00229 {
00230     logger->info("TcpClient::updateProductQuantity: id=" + QString::number(productId) + ", new=" +
QString::number(newQuantity));
00231     try {
00232         QVariantMap data;
00233         data["productId"] = productId;
00234         data["newQuantity"] = newQuantity;
00235         int reqId = sendRequest("updateProductQuantity", data);
00236         QVariantMap res;
00237         return waitForResponse(reqId, res);
00238     } catch (const NetworkEx &e) {
00239         logger->error(QString("updateProductQuantity: ") + e.what());
00240         emit errorOccurred(e.what());
00241         return false;
00242     } catch (const std::exception &e) {
00243         logger->error(QString("updateProductQuantity: неизвестная ошибка - ") + e.what());
00244         emit errorOccurred("Ошибка обновления количества продукта");
00245         return false;
00246     }
00247 }
00248
00249 bool TcpClient::addProductQuantity(int productId, int delta)
00250 {
00251     logger->info("TcpClient::addProductQuantity: id=" + QString::number(productId) + ", delta=" +
QString::number(delta));
00252     try {
00253         QVariantMap data;
00254         data["productId"] = productId;
00255         data["delta"] = delta;
00256         int reqId = sendRequest("addProductQuantity", data);

```

```

00257         QVariantMap res;
00258         return waitForResponse(reqId, res);
00259     } catch (const NetworkEx &e) {
00260         logger->error(QString("addProductQuantity: ") + e.what());
00261         emit errorOccurred(e.what());
00262         return false;
00263     } catch (const std::exception &e) {
00264         logger->error(QString("addProductQuantity: неизвестная ошибка - ") + e.what());
00265         emit errorOccurred("Ошибка изменения количества продукта");
00266         return false;
00267     }
00268 }
00269
00270 int TcpClient::createOrder(int tableNumber, int userId, const QString &specialRequests, double
totalCost)
00271 {
00272     logger->info("TcpClient::createOrder: table=" + QString::number(tableNumber) + ", user=" +
QString::number(userId) + ", total=" + QString::number(totalCost));
00273     try {
00274         QVariantMap data;
00275         data["tableNumber"] = tableNumber;
00276         data["userId"] = userId;
00277         data["specialRequests"] = specialRequests;
00278         data["totalCost"] = totalCost;
00279         int reqId = sendRequest("createOrder", data);
00280         QVariantMap res;
00281         if (waitForResponse(reqId, res)) {
00282             int orderId = res["orderId"].toInt();
00283             logger->info("TcpClient::createOrder: заказ создан, id=" + QString::number(orderId));
00284             return orderId;
00285         }
00286         return -1;
00287     } catch (const NetworkEx &e) {
00288         logger->error(QString("createOrder: ") + e.what());
00289         emit errorOccurred(e.what());
00290         return -1;
00291     } catch (const std::exception &e) {
00292         logger->error(QString("createOrder: неизвестная ошибка - ") + e.what());
00293         emit errorOccurred("Ошибка создания заказа на сервере");
00294         return -1;
00295     }
00296 }
00297
00298 bool TcpClient::addOrderItem(int orderId, int foodId, int quantity, double price)
00299 {
00300     logger->info("TcpClient::addOrderItem: order=" + QString::number(orderId) + ", food=" +
QString::number(foodId));
00301     try {
00302         QVariantMap data;
00303         data["orderId"] = orderId;
00304         data["foodId"] = foodId;
00305         data["quantity"] = quantity;
00306         data["price"] = price;
00307         int reqId = sendRequest("addOrderItem", data);
00308         QVariantMap res;
00309         return waitForResponse(reqId, res);
00310     } catch (const NetworkEx &e) {
00311         logger->error(QString("addOrderItem: ") + e.what());
00312         emit errorOccurred(e.what());
00313         return false;
00314     } catch (const std::exception &e) {
00315         logger->error(QString("addOrderItem: неизвестная ошибка - ") + e.what());
00316         emit errorOccurred("Ошибка добавления позиции заказа на сервере");
00317         return false;
00318     }
00319 }
00320
00321 bool TcpClient::checkAndConsume(int foodId, int quantity)
00322 {
00323     logger->info("TcpClient::checkAndConsume: food=" + QString::number(foodId) + ", qty=" +
QString::number(quantity));
00324     try {
00325         QVariantMap data;
00326         data["foodId"] = foodId;
00327         data["quantity"] = quantity;
00328         int reqId = sendRequest("checkAndConsume", data);
00329         QVariantMap res;
00330         return waitForResponse(reqId, res);
00331     } catch (const NetworkEx &e) {
00332         logger->error(QString("checkAndConsume: ") + e.what());
00333         emit errorOccurred(e.what());
00334         return false;
00335     } catch (const std::exception &e) {
00336         logger->error(QString("checkAndConsume: неизвестная ошибка - ") + e.what());
00337         emit errorOccurred("Ошибка при проверке и списании продуктов");
00338         return false;
00339     }
}

```



```

00340 }
00341
00342 bool TcpClient::returnIngredients(int foodId, int quantity)
00343 {
00344     logger->info("TcpClient::returnIngredients: food=" + QString::number(foodId) + ", qty=" +
00345         QString::number(quantity));
00346     try {
00347         QVariantMap data;
00348         data["foodId"] = foodId;
00349         data["quantity"] = quantity;
00350         int reqId = sendRequest("returnIngredients", data);
00351         QVariantMap res;
00352         return waitForResponse(reqId, res);
00353     } catch (const NetworkEx &e) {
00354         logger->error(QString("returnIngredients: ") + e.what());
00355         emit errorOccurred(e.what());
00356         return false;
00357     } catch (const std::exception &e) {
00358         logger->error(QString("returnIngredients: неизвестная ошибка - ") + e.what());
00359         emit errorOccurred("Ошибка при возврате продуктов");
00360         return false;
00361     }
00362 }
00363 QVariantList TcpClient::getLowStockWarning(int foodId, int quantity)
00364 {
00365     logger->info("TcpClient::getLowStockWarning: food=" + QString::number(foodId) + ", qty=" +
00366         QString::number(quantity));
00367     try {
00368         QVariantMap data;
00369         data["foodId"] = foodId;
00370         data["quantity"] = quantity;
00371         int reqId = sendRequest("getLowStockWarning", data);
00372         QVariantMap res;
00373         if (waitForResponse(reqId, res)) {
00374             return res["warnings"].toList();
00375         }
00376         return QVariantList();
00377     } catch (const NetworkEx &e) {
00378         logger->error(QString("getLowStockWarning: ") + e.what());
00379         emit errorOccurred(e.what());
00380         return QVariantList();
00381     } catch (const std::exception &e) {
00382         logger->error(QString("getLowStockWarning: неизвестная ошибка - ") + e.what());
00383         emit errorOccurred("Ошибка получения предупреждения о малых остатках");
00384         return QVariantList();
00385     }
00386 }
00387 QVariantMap TcpClient::getReportRevenue(const QString &from, const QString &to, int dishId, int
00388     tableNumber, int waiterId)
00389 {
00390     logger->info("TcpClient::getReportRevenue: " + from + " - " + to);
00391     try {
00392         QVariantMap data;
00393         data["from"] = from;
00394         data["to"] = to;
00395         data["dishId"] = dishId;
00396         data["tableNumber"] = tableNumber;
00397         data["waiterId"] = waiterId;
00398         int reqId = sendRequest("getReportRevenue", data);
00399         QVariantMap res;
00400         if (waitForResponse(reqId, res)) {
00401             return res;
00402         }
00403         return QVariantMap();
00404     } catch (const NetworkEx &e) {
00405         logger->error(QString("getReportRevenue: ") + e.what());
00406         emit errorOccurred(e.what());
00407         return QVariantMap();
00408     } catch (const std::exception &e) {
00409         logger->error(QString("getReportRevenue: неизвестная ошибка - ") + e.what());
00410         emit errorOccurred("Ошибка получения отчёта о выручке");
00411         return QVariantMap();
00412     }
00413 }
00414 QVariantList TcpClient::getRevenueByDay(const QString &from, const QString &to, int dishId, int
00415     tableNumber, int waiterId)
00416 {
00417     logger->info("TcpClient::getRevenueByDay: " + from + " - " + to);
00418     try {
00419         QVariantMap data;
00420         data["from"] = from;
00421         data["to"] = to;
00422         data["dishId"] = dishId;
00423         data["tableNumber"] = tableNumber;

```



```

00423         data["waiterId"] = waiterId;
00424         int reqId = sendRequest("getRevenueByDay", data);
00425         QVariantMap res;
00426         if (waitForResponse(reqId, res)) {
00427             return res["days"].toList();
00428         }
00429         return QVariantList();
00430     } catch (const NetworkEx &e) {
00431         logger->error(QString("getRevenueByDay: ") + e.what());
00432         emit errorOccurred(e.what());
00433         return QVariantList();
00434     } catch (const std::exception &e) {
00435         logger->error(QString("getRevenueByDay: неизвестная ошибка - ") + e.what());
00436         emit errorOccurred("Ошибка получения выручки по дням");
00437         return QVariantList();
00438     }
00439 }
00440
00441 QVariantList TcpClient::getFoodPopularity(const QString &from, const QString &to)
00442 {
00443     logger->info("TcpClient::getFoodPopularity: " + from + " - " + to);
00444     try {
00445         QVariantMap data;
00446         data["from"] = from;
00447         data["to"] = to;
00448         int reqId = sendRequest("getFoodPopularity", data);
00449         QVariantMap res;
00450         if (waitForResponse(reqId, res)) {
00451             return res["popularity"].toList();
00452         }
00453         return QVariantList();
00454     } catch (const NetworkEx &e) {
00455         logger->error(QString("getFoodPopularity: ") + e.what());
00456         emit errorOccurred(e.what());
00457         return QVariantList();
00458     } catch (const std::exception &e) {
00459         logger->error(QString("getFoodPopularity: неизвестная ошибка - ") + e.what());
00460         emit errorOccurred("Ошибка получения популярности блюд");
00461         return QVariantList();
00462     }
00463 }
00464
00465 QVariantList TcpClient::getReportLowStock()
00466 {
00467     logger->info("TcpClient::getReportLowStock");
00468     try {
00469         int reqId = sendRequest("getReportLowStock");
00470         QVariantMap res;
00471         if (waitForResponse(reqId, res)) {
00472             return res["lowStock"].toList();
00473         }
00474         return QVariantList();
00475     } catch (const NetworkEx &e) {
00476         logger->error(QString("getReportLowStock: ") + e.what());
00477         emit errorOccurred(e.what());
00478         return QVariantList();
00479     } catch (const std::exception &e) {
00480         logger->error(QString("getReportLowStock: неизвестная ошибка - ") + e.what());
00481         emit errorOccurred("Ошибка получения отчёта о малых остатках");
00482         return QVariantList();
00483     }
00484 }
00485
00486 QVariantList TcpClient::getDishesList()
00487 {
00488     logger->info("TcpClient::getDishesList");
00489     try {
00490         int reqId = sendRequest("getDishesList");
00491         QVariantMap res;
00492         if (waitForResponse(reqId, res)) {
00493             return res["dishes"].toList();
00494         }
00495         return QVariantList();
00496     } catch (const NetworkEx &e) {
00497         logger->error(QString("getDishesList: ") + e.what());
00498         emit errorOccurred(e.what());
00499         return QVariantList();
00500     } catch (const std::exception &e) {
00501         logger->error(QString("getDishesList: неизвестная ошибка - ") + e.what());
00502         emit errorOccurred("Ошибка получения списка блюд");
00503         return QVariantList();
00504     }
00505 }
00506
00507 QVariantList TcpClient::getWaitersList()
00508 {
00509     logger->info("TcpClient::getWaitersList");

```

```

00510     try {
00511         int reqId = sendRequest("getWaitersList");
00512         QVariantMap res;
00513         if (waitForResponse(reqId, res)) {
00514             return res["waiters"].toList();
00515         }
00516         return QVariantList();
00517     } catch (const NetworkEx &e) {
00518         logger->error(QString("getWaitersList: ") + e.what());
00519         emit errorOccurred(e.what());
00520         return QVariantList();
00521     } catch (const std::exception &e) {
00522         logger->error(QString("getWaitersList: неизвестная ошибка - ") + e.what());
00523         emit errorOccurred("Ошибка получения списка официантов");
00524         return QVariantList();
00525     }
00526 }
00527
00528 QStringList TcpClient::getLocalIps()
00529 {
00530     logger->info("TcpClient::getLocalIps");
00531     QStringList result;
00532     const QList<QNetworkInterface> ifaces = QNetworkInterface::allInterfaces();
00533     for (const QNetworkInterface &iface : ifaces) {
00534         if (!(iface.flags() & QNetworkInterface::IsUp) || (iface.flags() &
00535             QNetworkInterface::IsLoopBack))
00536             continue;
00537         for (const QNetworkAddressEntry &entry : iface.addressEntries()) {
00538             if (entry.ip().protocol() == QAbstractSocket::IPv4Protocol)
00539                 result « entry.ip().toString();
00540         }
00541     }
00542     logger->info("TcpClient::getLocalIps: найдено " + QString::number(result.size()) + " IP");
00543     return result;
00544 }
00545 bool TcpClient::pingServer(const QString &host, int port)
00546 {
00547     logger->info("TcpClient::pingServer: проверка " + host + ":" + QString::number(port));
00548     QTcpSocket socket;
00549     socket.connectToHost(host, port);
00550     bool ok = socket.waitForConnected(2000);
00551     logger->info("TcpClient::pingServer: результат " + QString(ok ? "успех" : "неудача"));
00552     return ok;
00553 }
00554
00555 bool TcpClient::reconnect(const QString &host, quint16 port)
00556 {
00557     disconnectFromServer();
00558     return connectToServer(host, port);
00559 }
00560
00561 bool TcpClient::saveReportToCSV(const QString &reportType,
00562     const QString &from,
00563     const QString &to,
00564     int dishId,
00565     int tableNumber,
00566     int waiterId,
00567     const QString &fileName)
00568 {
00569     logger->info("TcpClient::saveReportToCSV: тип=" + reportType);
00570     try {
00571         QVariantMap data;
00572         data["reportType"] = reportType;
00573         data["from"] = from;
00574         data["to"] = to;
00575         data["dishId"] = dishId;
00576         data["tableNumber"] = tableNumber;
00577         data["waiterId"] = waiterId;
00578         data["fileName"] = fileName; // передаём полный путь
00579         int reqId = sendRequest("saveReportToCSV", data);
00580         QVariantMap res;
00581         return waitForResponse(reqId, res);
00582     }
00583     catch (const NetworkEx &e) {
00584         logger->error("saveReportToCSV: " + QString(e.what()));
00585         emit errorOccurred(e.what());
00586         return false;
00587     }
00588 }
00589
00590 QVariantList TcpClient::getOrders(const QString &status)
00591 {
00592     logger->info("TcpClient::getOrders, status=" + status);
00593     try {
00594         QVariantMap data;
00595         if (!status.isEmpty())

```

```

00596         data["status"] = status;
00597         int reqId = sendRequest("getOrders", data);
00598         QVariantMap res;
00599         if (waitForResponse(reqId, res)) {
00600             return res["orders"].toList();
00601         }
00602         return QVariantList();
00603     } catch (const NetworkEx &e) {
00604         logger->error(QString("getOrders: ") + e.what());
00605         emit errorOccurred(e.what());
00606         return QVariantList();
00607     }
00608 }
00609
00610 bool TcpClient::updateOrderStatus(int orderId, const QString &newStatus)
00611 {
00612     logger->info("TcpClient::updateOrderStatus: " + QString::number(orderId) + " -> " + newStatus);
00613     try {
00614         QVariantMap data;
00615         data["orderId"] = orderId;
00616         data["newStatus"] = newStatus;
00617         int reqId = sendRequest("updateOrderStatus", data);
00618         QVariantMap res;
00619         bool ok = waitForResponse(reqId, res);
00620         if (ok)
00621             emit ordersChanged(); // уведомить QML об изменении
00622         return ok;
00623     } catch (const NetworkEx &e) {
00624         logger->error(QString("updateOrderStatus: ") + e.what());
00625         emit errorOccurred(e.what());
00626         return false;
00627     }
00628 }
00629
00630 QVariantList TcpClient::getOrderItems(int orderId)
00631 {
00632     logger->info("TcpClient::getOrderItems, orderId=" + QString::number(orderId));
00633     try {
00634         QVariantMap data;
00635         data["orderId"] = orderId;
00636         int reqId = sendRequest("getOrderItems", data);
00637         QVariantMap res;
00638         if (waitForResponse(reqId, res)) {
00639             return res["items"].toList();
00640         }
00641         return QVariantList();
00642     } catch (const NetworkEx &e) {
00643         logger->error(QString("getOrderItems: ") + e.what());
00644         emit errorOccurred(e.what());
00645         return QVariantList();
00646     }
00647 }
00648
00649 bool TcpClient::closeOrder(int orderId, double totalSum, const QVariantList &items)
00650 {
00651     logger->info("TcpClient::closeOrder, orderId=" + QString::number(orderId));
00652     try {
00653         QVariantMap data;
00654         data["orderId"] = orderId;
00655         data["totalSum"] = totalSum;
00656         data["items"] = items;
00657         int reqId = sendRequest("closeOrder", data);
00658         QVariantMap res;
00659         bool ok = waitForResponse(reqId, res);
00660         if (ok)
00661             emit ordersChanged();
00662         return ok;
00663     } catch (const NetworkEx &e) {
00664         logger->error(QString("closeOrder: ") + e.what());
00665         emit errorOccurred(e.what());
00666         return false;
00667     }
00668 }
00669
00670 QVariantMap TcpClient::getCheckByOrderId(int orderId)
00671 {
00672     logger->info("TcpClient::getCheckByOrderId, orderId=" + QString::number(orderId));
00673     try {
00674         QVariantMap data;
00675         data["orderId"] = orderId;
00676         int reqId = sendRequest("getCheckByOrderId", data);
00677         QVariantMap res;
00678         if (waitForResponse(reqId, res)) {
00679             return res;
00680         }
00681         return QVariantMap();
00682     } catch (const NetworkEx &e) {

```

```

00683         logger->error(QString("getCheckByOrderId: ") + e.what());
00684         emit errorOccurred(e.what());
00685         return QVariantMap();
00686     }
00687 }
00688
00689 bool TcpClient::syncOrderItems(int orderId, const QVariantList &newItems)
00690 {
00691     logger->info("TcpClient::syncOrderItems, orderId=" + QString::number(orderId));
00692     try {
00693         QVariantMap data;
00694         data["orderId"] = orderId;
00695         data["newItems"] = newItems;
00696         int reqId = sendRequest("syncOrderItems", data);
00697         QVariantMap res;
00698         bool ok = waitForResponse(reqId, res);
00699         if (ok)
00700             emit ordersChanged();
00701         return ok;
00702     } catch (const NetworkEx &e) {
00703         logger->error(QString("syncOrderItems: ") + e.what());
00704         emit errorOccurred(e.what());
00705         return false;
00706     }
00707 }
00708
00709 QVariantList TcpClient::getClosedOrders()
00710 {
00711     return getOrders("Закрыт");
00712 }

```

6.13. tcpserver.cpp

```

00001 #include "tcpserver.h"
00002 #include "dbworker.h"
00003 #include <QJsonDocument>
00004 #include <QJsonObject>
00005 #include <QDebug>
00006
00007 ClientConnection::ClientConnection(Logs *logger, qintptr socketDescriptor, DBWorker *dbWorker, QObject
    *parent)
00008     : QObject(parent), dbWorker(dbWorker), logger(logger)
00009 {
00010     socket = new QTcpSocket(this);
00011     socket->setSocketDescriptor(socketDescriptor);
00012     connect(socket, &QTcpSocket::readyRead, this, &ClientConnection::onReadyRead);
00013     connect(socket, &QTcpSocket::disconnected, this, &ClientConnection::onDisconnected);
00014 }
00015
00016 ClientConnection::~ClientConnection()
00017 {
00018     socket->deleteLater();
00019 }
00020
00021 void ClientConnection::sendResponse(int requestId, bool success, const QVariantMap &data)
00022 {
00023     QJsonObject resp;
00024     resp["response"] = success;
00025     resp["requestId"] = requestId;
00026     if (!data.isEmpty())
00027         resp["data"] = QJsonObject::fromVariantMap(data);
00028     QByteArray msg = QJsonDocument(resp).toJson(QJsonDocument::Compact) + "\n";
00029     qint64 bytes = socket->write(msg);
00030     socket->flush();
00031     logger->info("ClientConnection: отправлен ответ на запрос " + QString::number(requestId) +
00032         ", успеш=" + QString::number(success) + ", байт=" + QString::number(bytes));
00033 }
00034
00035 void ClientConnection::onReadyRead()
00036 {
00037     QByteArray data = socket->readAll();
00038     logger->info("ClientConnection: получено " + QString::number(data.size()) + " байт");
00039     buffer.append(data);
00040     while (true) {
00041         int nl = buffer.indexOf('\n');
00042         if (nl == -1) break;
00043         QByteArray line = buffer.left(nl);
00044         buffer.remove(0, nl + 1);
00045         QJsonParseError err;
00046         QJsonDocument doc = QJsonDocument::fromJson(line, &err);
00047         if (err.error != QJsonParseError::NoError) continue;
00048         if (!doc.isObject()) continue;
00049         QJsonObject obj = doc.object();
00050         int requestId = obj.value("requestId").toInt();

```

```

00051     QString command = obj.value("command").toString();
00052     QVariantMap data = obj.value("data").toObject().toVariantMap();
00053
00054     if (command == "checkUser") {
00055         try {
00056             bool ok = dbWorker->checkUser(data["login"].toString(), data["password"].toString());
00057             if (ok) {
00058                 QVariantMap result;
00059                 result["role"] = dbWorker->getUserRole();
00060                 result["name"] = dbWorker->getUserName();
00061                 result["userId"] = dbWorker->getUserId();
00062                 sendResponse(requestId, true, result);
00063             } else {
00064                 sendResponse(requestId, false);
00065             }
00066         } catch (const std::exception &e) {
00067             logger->error("Исключение в checkUser: " + QString(e.what()));
00068             sendResponse(requestId, false, {"error", e.what()});
00069         }
00070     }
00071     else if (command == "getFoods") {
00072         try {
00073             QVariantList foods = dbWorker->getFoods();
00074             sendResponse(requestId, true, {"foods", foods});
00075         } catch (const std::exception &e) {
00076             logger->error("Исключение в getFoods: " + QString(e.what()));
00077             sendResponse(requestId, false, {"error", e.what()});
00078         }
00079     }
00080     else if (command == "getProducts") {
00081         try {
00082             QVariantList products = dbWorker->getProducts();
00083             sendResponse(requestId, true, {"products", products});
00084         } catch (const std::exception &e) {
00085             logger->error("Исключение в getProducts: " + QString(e.what()));
00086             sendResponse(requestId, false, {"error", e.what()});
00087         }
00088     }
00089     else if (command == "updateProductQuantity") {
00090         try {
00091             bool ok = dbWorker->updateProductQuantity(data["productId"].toInt(),
00092 data["newQuantity"].toInt());
00093             sendResponse(requestId, ok);
00094         } catch (const std::exception &e) {
00095             logger->error("Исключение в updateProductQuantity: " + QString(e.what()));
00096             sendResponse(requestId, false, {"error", e.what()});
00097         }
00098     }
00099     else if (command == "addProductQuantity") {
00100         try {
00101             bool ok = dbWorker->addProductQuantity(data["productId"].toInt(),
00102 data["delta"].toInt());
00103             sendResponse(requestId, ok);
00104         } catch (const std::exception &e) {
00105             logger->error("Исключение в addProductQuantity: " + QString(e.what()));
00106             sendResponse(requestId, false, {"error", e.what()});
00107         }
00108     }
00109     else if (command == "createOrder") {
00110         try {
00111             int orderId = dbWorker->createOrder(data["tableNumber"].toInt(),
00112 data["userId"].toInt(),
00113 data["specialRequests"].toString(),
00114 data["totalCost"].toDouble());
00115             if (orderId > 0) {
00116                 sendResponse(requestId, true, {"orderId", orderId});
00117             } else {
00118                 sendResponse(requestId, false);
00119             }
00120         } catch (const std::exception &e) {
00121             logger->error("Исключение в createOrder: " + QString(e.what()));
00122             sendResponse(requestId, false, {"error", e.what()});
00123         }
00124     }
00125     else if (command == "addOrderItem") {
00126         try {
00127             bool ok = dbWorker->addOrderItem(data["orderId"].toInt(), data["foodId"].toInt(),
00128 data["quantity"].toInt(), data["price"].toDouble());
00129             sendResponse(requestId, ok);
00130         } catch (const std::exception &e) {
00131             logger->error("Исключение в addOrderItem: " + QString(e.what()));
00132             sendResponse(requestId, false, {"error", e.what()});
00133         }
00134     }
00135     else if (command == "checkAndConsume") {
00136         try {
00137             bool ok = dbWorker->checkAndConsume(data["foodId"].toInt(), data["quantity"].toInt());

```

```

00134         sendResponse(requestId, ok);
00135     } catch (const std::exception &e) {
00136         logger->error("Исключение в checkAndConsume: " + QString(e.what()));
00137         sendResponse(requestId, false, {"error", e.what()});
00138     }
00139 }
00140 else if (command == "returnIngredients") {
00141     try {
00142         bool ok = dbWorker->returnIngredients(data["foodId"].toInt(),
data["quantity"].toInt());
00143         sendResponse(requestId, ok);
00144     } catch (const std::exception &e) {
00145         logger->error("Исключение в returnIngredients: " + QString(e.what()));
00146         sendResponse(requestId, false, {"error", e.what()});
00147     }
00148 }
00149 else if (command == "getLowStockWarning") {
00150     try {
00151         QVariantList warnings = dbWorker->getLowStockWarning(data["foodId"].toInt(),
data["quantity"].toInt());
00152         sendResponse(requestId, true, {"warnings", warnings});
00153     } catch (const std::exception &e) {
00154         logger->error("Исключение в getLowStockWarning: " + QString(e.what()));
00155         sendResponse(requestId, false, {"error", e.what()});
00156     }
00157 }
00158 else if (command == "getReportRevenue") {
00159     try {
00160         QVariantMap rev = dbWorker->getReportRevenue(data["from"].toString(),
data["to"].toString(),
00161                                                     data["dishId"].toInt(),
data["tableNumber"].toInt(),
00162                                                     data["waiterId"].toInt());
00163         sendResponse(requestId, true, rev);
00164     } catch (const std::exception &e) {
00165         logger->error("Исключение в getReportRevenue: " + QString(e.what()));
00166         sendResponse(requestId, false, {"error", e.what()});
00167     }
00168 }
00169 else if (command == "getRevenueByDay") {
00170     try {
00171         QVariantList days = dbWorker->getRevenueByDay(data["from"].toString(),
data["to"].toString(),
00172                                                     data["dishId"].toInt(),
data["tableNumber"].toInt(),
00173                                                     data["waiterId"].toInt());
00174         sendResponse(requestId, true, {"days", days});
00175     } catch (const std::exception &e) {
00176         logger->error("Исключение в getRevenueByDay: " + QString(e.what()));
00177         sendResponse(requestId, false, {"error", e.what()});
00178     }
00179 }
00180 else if (command == "getFoodPopularity") {
00181     try {
00182         QVariantList pop = dbWorker->getFoodPopularity(data["from"].toString(),
data["to"].toString());
00183         sendResponse(requestId, true, {"popularity", pop});
00184     } catch (const std::exception &e) {
00185         logger->error("Исключение в getFoodPopularity: " + QString(e.what()));
00186         sendResponse(requestId, false, {"error", e.what()});
00187     }
00188 }
00189 else if (command == "getReportLowStock") {
00190     try {
00191         QVariantList low = dbWorker->getReportLowStock();
00192         sendResponse(requestId, true, {"lowStock", low});
00193     } catch (const std::exception &e) {
00194         logger->error("Исключение в getReportLowStock: " + QString(e.what()));
00195         sendResponse(requestId, false, {"error", e.what()});
00196     }
00197 }
00198 else if (command == "getDishesList") {
00199     try {
00200         QVariantList dishes = dbWorker->getDishesList();
00201         sendResponse(requestId, true, {"dishes", dishes});
00202     } catch (const std::exception &e) {
00203         logger->error("Исключение в getDishesList: " + QString(e.what()));
00204         sendResponse(requestId, false, {"error", e.what()});
00205     }
00206 }
00207 else if (command == "getWaitersList") {
00208     try {
00209         QVariantList waiters = dbWorker->getWaitersList();
00210         sendResponse(requestId, true, {"waiters", waiters});
00211     } catch (const std::exception &e) {
00212         logger->error("Исключение в getWaitersList: " + QString(e.what()));
00213         sendResponse(requestId, false, {"error", e.what()});

```



```

00214     }
00215 }
00216 else if (command == "saveReportToCSV") {
00217     try {
00218         bool ok = dbWorker->saveReportToCSV(
00219             data["reportType"].toString(),
00220             data["from"].toString(),
00221             data["to"].toString(),
00222             data["dishId"].toInt(),
00223             data["tableNumber"].toInt(),
00224             data["waiterId"].toInt(),
00225             data["fileName"].toString()
00226         );
00227         sendResponse(requestId, ok);
00228     } catch (const std::exception &e) {
00229         logger->error("Исключение в saveReportToCSV: " + QString(e.what()));
00230         sendResponse(requestId, false, {"error", e.what()});
00231     }
00232 }
00233 else if (command == "getOrders") {
00234     try {
00235         QString status = data["status"].toString();
00236         QVariantList orders = dbWorker->getOrders(status);
00237         sendResponse(requestId, true, {"orders", orders});
00238     } catch (const std::exception &e) {
00239         logger->error("Исключение в getOrders: " + QString(e.what()));
00240         sendResponse(requestId, false, {"error", e.what()});
00241     }
00242 }
00243 else if (command == "updateOrderStatus") {
00244     try {
00245         bool ok = dbWorker->updateOrderStatus(data["orderId"].toInt(), data["newStatus"].toString());
00246         sendResponse(requestId, ok);
00247     } catch (const std::exception &e) {
00248         logger->error("Исключение в updateOrderStatus: " + QString(e.what()));
00249         sendResponse(requestId, false, {"error", e.what()});
00250     }
00251 }
00252 else if (command == "getOrderItems") {
00253     try {
00254         QVariantList items = dbWorker->getOrderItems(data["orderId"].toInt());
00255         sendResponse(requestId, true, {"items", items});
00256     } catch (const std::exception &e) {
00257         logger->error("Исключение в getOrderItems: " + QString(e.what()));
00258         sendResponse(requestId, false, {"error", e.what()});
00259     }
00260 }
00261 else if (command == "closeOrder") {
00262     try {
00263         bool ok = dbWorker->closeOrder(data["orderId"].toInt(), data["totalSum"].toDouble(),
00264             data["items"].toList());
00265         sendResponse(requestId, ok);
00266     } catch (const std::exception &e) {
00267         logger->error("Исключение в closeOrder: " + QString(e.what()));
00268         sendResponse(requestId, false, {"error", e.what()});
00269     }
00270 }
00271 else if (command == "getCheckByOrderId") {
00272     try {
00273         QVariantMap check = dbWorker->getCheckByOrderId(data["orderId"].toInt());
00274         sendResponse(requestId, true, check);
00275     } catch (const std::exception &e) {
00276         logger->error("Исключение в getCheckByOrderId: " + QString(e.what()));
00277         sendResponse(requestId, false, {"error", e.what()});
00278     }
00279 }
00280 else if (command == "syncOrderItems") {
00281     try {
00282         bool ok = dbWorker->syncOrderItems(data["orderId"].toInt(), data["newItems"].toList());
00283         sendResponse(requestId, ok);
00284     } catch (const std::exception &e) {
00285         logger->error("Исключение в syncOrderItems: " + QString(e.what()));
00286         sendResponse(requestId, false, {"error", e.what()});
00287     }
00288 }
00289 else {
00290     sendResponse(requestId, false, {"error", "Unknown command"});
00291 }
00292 }
00293 void ClientConnection::onDisconnected()
00294 {
00295     deleteLater();
00296 }
00297 }
00298
00299 TcpServer::TcpServer(Logs *logger, DBWorker *dbWorker, QObject *parent)

```

```

00300 : QObject(parent), server(nullptr), dbWorker(dbWorker), logger(logger)
00301 {
00302     logger->info("Инициализация TCP-сервера");
00303 }
00304
00305 TcpServer::~TcpServer()
00306 {
00307     stop();
00308     logger->info("TCP-сервер уничтожен");
00309 }
00310
00311 bool TcpServer::start(quint16 port)
00312 {
00313     if (server) {
00314         logger->warning("Сервер уже запущен");
00315         return false;
00316     }
00317     server = new QTcpServer(this);
00318     connect(server, &QTcpServer::newConnection, this, &TcpServer::onNewConnection);
00319     if (!server->listen(QHostAddress::Any, port)) {
00320         logger->error("Ошибка запуска сервера: " + server->errorString());
00321         delete server;
00322         server = nullptr;
00323         return false;
00324     }
00325     logger->info("Сервер запущен на порту " + QString::number(port));
00326     return true;
00327 }
00328
00329 void TcpServer::stop()
00330 {
00331     if (server) {
00332         server->close();
00333         delete server;
00334         server = nullptr;
00335         logger->info("Сервер остановлен");
00336     }
00337     qDeleteAll(connections);
00338     connections.clear();
00339 }
00340
00341 void TcpServer::onNewConnection()
00342 {
00343     QTcpSocket *clientSocket = server->nextPendingConnection();
00344     ClientConnection *conn = new ClientConnection(logger, clientSocket->socketDescriptor(), dbWorker,
00345     this);
00346     connections.append(conn);
00347     logger->info("Новое подключение, всего клиентов: " + QString::number(connections.size()));
00348 }

```